

Análisis de redes en R

Un enfoque práctico y didáctico

Roberto Alvarez

2026-04-12

Table of contents

Prefacio	7
¿Para quién es este libro?	7
¿Qué vas a aprender?	7
Estructura del libro	7
Prerrequisitos	8
Paquetes necesarios	8
Convenciones del libro	9
Cómo usar este libro	9
1 Fundamentos de igraph	10
1.1 Preparación del entorno	12
1.2 Crear redes desde cero	13
1.2.1 Paso 1: Grafo vacío	13
1.2.2 Paso 2: Agregar conexiones	14
1.2.3 Paso 3: Agregar nodos con atributos	14
1.2.4 Paso 4: Nombrar los nodos	15
1.2.5 Paso 5: Modificar conexiones	16
1.3 grado (degree) y distribución de grado (degree) {#sec-grado (degree)-basico} .	17
1.3.1 grado (degree) total	18
1.3.2 grado (degree) de entrada	18
1.3.3 grado (degree) de salida	18
1.3.4 Visualizar el grado (degree)	18
1.4 Matriz de adyacencia	20
1.5 Redes especiales	22
1.5.1 Estrella	22
1.5.2 Anillo	23
1.5.3 Árbol binario	23
1.5.4 Lattice	24
1.6 Ejercicios	25
Ejercicios resueltos	25
Ejercicios con pistas	29
Ejercicios propuestos	31
1.7 Resumen del capítulo	32
Autoevaluación	34

2	Grado y distribuciones	36
2.1	Preparación del entorno	39
2.2	Dos modelos, dos mundos	39
2.2.1	Modelo Erdős–Rényi (ER): redes aleatorias	39
2.2.2	Modelo Barabási–Albert (BA): enlace preferencial	40
2.2.3	Comparación rápida	40
2.3	Crear y visualizar ambas redes	40
2.4	Distribuciones de grado	42
2.4.1	Histogramas	42
2.4.2	Gráfico log-log	43
2.5	Ajuste de ley de potencia con <code>powerLaw</code>	44
2.5.1	Paso 1: Crear el modelo	45
2.5.2	Paso 2: Estimar parámetros	45
2.5.3	Paso 3: Visualizar el ajuste	46
2.6	Identificar <i>hubs</i>	47
2.7	Comparación completa: ER vs. BA vs. <i>Small-World</i>	48
2.8	Ejercicios	50
	Ejercicios resueltos	50
	Ejercicios con pistas	55
	Ejercicios propuestos	57
2.9	Resumen del capítulo	59
	Autoevaluación	60
3	Distancias y eficiencia	62
3.1	Preparación del entorno	63
3.2	¿Qué es la distancia en una red?	63
3.3	Caminos más cortos	64
3.3.1	Entre dos nodos específicos	64
3.3.2	La matriz de distancias completa	66
3.4	Métricas globales de distancia	68
3.4.1	Distancia media	68
3.4.2	Diámetro	69
3.4.3	Excentricidad	69
3.5	Eficiencia de la red	71
3.5.1	Densidad de aristas	72
3.6	El efecto de los atajos	72
3.7	Comparación entre topologías	75
3.8	Distancias en redes grandes	77
3.9	Ejercicios	79
	Ejercicios resueltos	79
	Ejercicios con pistas	83
	Ejercicios propuestos	85
3.10	Resumen del capítulo	86

Autoevaluación	88
4 Clustering y transitividad	90
4.1 Preparación del entorno	91
4.2 ¿Qué es el clustering?	91
4.2.1 Ejemplo visual: triángulos	91
4.3 Coeficiente de clustering local	92
4.3.1 ¿Por qué A tiene ese clustering?	95
4.3.2 ¿Por qué F tiene NaN?	95
4.3.3 ¿Por qué B tiene clustering 1?	95
4.4 Coeficiente de clustering global (transitividad)	95
4.4.1 Transitividad global	96
4.4.2 Clustering promedio	96
4.5 Triángulos en la red	96
4.6 Clustering en distintos modelos de red	97
4.7 El fenómeno <i>small-world</i> : clustering + distancia corta	100
4.8 Relación entre grado y clustering local	102
4.9 Visualización por clustering	103
4.10 Ejercicios	105
Ejercicios resueltos	105
Ejercicios con pistas	110
Ejercicios propuestos	112
4.11 Resumen del capítulo	113
Autoevaluación	114
5 Robustez de redes	116
5.1 Preparación del entorno	117
5.2 ¿Qué significa que una red sea robusta?	117
5.3 Herramienta fundamental: <code>delete_vertices()</code>	118
5.4 El componente gigante	120
5.5 Simulación de fallos aleatorios	121
5.6 Experimento: ER vs. BA bajo fallos y ataques	123
5.6.1 Resultado 1: Componente gigante	125
5.6.2 Resultado 2: Distancia media	126
5.6.3 Resultado 3: Diámetro	128
5.7 Incluyendo la red <i>Small-World</i>	129
5.8 Tabla resumen de robustez	131
5.9 Visualización del proceso de fragmentación	132
5.10 Múltiples realizaciones: promediar la incertidumbre	133
5.11 Ejercicios	136
Ejercicios resueltos	136
Ejercicios con pistas	140
Ejercicios propuestos	142

5.12	Resumen del capítulo	144
	Autoevaluación	145
6	Medidas de centralidad	147
6.1	Preparación del entorno	148
6.2	¿Qué es la centralidad?	148
6.3	Red de ejemplo	149
6.4	Las cuatro medidas clásicas	151
6.4.1	1. Centralidad de grado (<i>Degree Centrality</i>)	151
6.4.2	2. Centralidad de cercanía (<i>Closeness Centrality</i>)	153
6.4.3	3. Centralidad de intermediación (<i>Betweenness Centrality</i>)	154
6.4.4	4. Centralidad de vector propio (<i>Eigenvector Centrality</i>)	157
6.5	Comparación visual: las 4 clásicas	159
6.6	Medidas avanzadas de centralidad	161
6.6.1	5. PageRank	161
6.6.2	6. HITS: Hubs y Authorities	162
6.6.3	7. Centralidad armónica (<i>Harmonic Centrality</i>)	163
6.6.4	8. Centralidad de subgrafo (<i>Subgraph Centrality</i>)	164
6.6.5	9. Alpha Centrality	165
6.6.6	10. Centralidad de poder de Bonacich (<i>Power Centrality</i>)	166
6.6.7	11. Fuerza (<i>Strength</i>) — Centralidad para redes ponderadas	167
6.7	Tabla comparativa completa	168
6.8	Correlación entre medidas	169
6.9	Centralización: del nodo a la red	172
6.9.1	Comparación con redes de referencia	174
6.10	Ejercicios	175
	Ejercicios resueltos	175
	Ejercicios con pistas	178
	Ejercicios propuestos	180
6.11	Resumen del capítulo	182
	Autoevaluación	183
7	Detección de comunidades	185
7.1	Preparación del entorno	186
7.2	¿Qué es una comunidad?	186
7.3	Modularidad: la métrica de calidad	188
7.4	Red de ejemplo: Club de Karate de Zachary	188
7.5	Los 11 algoritmos de igraph	189
7.5.1	1. Edge Betweenness (<code>cluster_edge_betweenness</code>)	189
7.5.2	2. Fast Greedy (<code>cluster_fast_greedy</code>)	190
7.5.3	3. Walktrap (<code>cluster_walktrap</code>)	191
7.5.4	4. Leading Eigenvector (<code>cluster_leading_eigen</code>)	192
7.5.5	5. Louvain (<code>cluster_louvain</code>)	193

7.5.6	6. Leiden (<code>cluster_leiden</code>)	194
7.5.7	7. Label Propagation (<code>cluster_label_prop</code>)	195
7.5.8	8. Spinglass (<code>cluster_spinglass</code>)	196
7.5.9	9. Infomap (<code>cluster_infomap</code>)	197
7.5.10	10. Fluid Communities (<code>cluster_fluid_communities</code>)	198
7.5.11	11. Optimal (<code>cluster_optimal</code>)	199
7.6	Comparación de todos los algoritmos	200
7.6.1	Tabla resumen	200
7.6.2	Visualización comparativa	202
7.6.3	Gráfico de barras: modularidad	203
7.7	Comparar dos particiones: NMI y Rand Index	204
7.8	Guía de selección de algoritmo	207
7.9	Ejercicios	208
	Ejercicios resueltos	208
	Ejercicios con pistas	211
	Ejercicios propuestos	214
7.10	Resumen del capítulo	215
	Autoevaluación	216
8	VisNetwork y networkD3	218
8.0.1	Redes con <code>networkD3</code>	218
8.0.2	Cómo obtener ayuda en <code>visNetwork</code>	220
8.0.3	Personalización de la visualización	220
9	Ejercicios integradores	222
	Guía de dificultad	222
9.1	Preparación del entorno	222
	Ejercicio integrador 1: Perfil completo de una red	222
	Ejercicio integrador 2: Red social de una clase	226
	Ejercicio integrador 3: Epidemia en tres topologías	227
	Ejercicio integrador 4: Análisis de robustez comparativo completo	228
	Ejercicio integrador 5: Red de ingredientes	229
	Ejercicio integrador 6: Construir desde archivo	230
	Ejercicio integrador 7: Power-law en el mundo real	231
	Ejercicio integrador 8: Diseña tu propia red	232
	Rúbrica de autoevaluación general	233
	Referencias	234

Prefacio

Bienvenido a **Análisis de redes en R**, un libro diseñado para que aprendas, de forma práctica y progresiva, los fundamentos del análisis de redes complejas utilizando el lenguaje R y el paquete `igraph` y otros como `visNetwork`.

¿Para quién es este libro?

Este material está pensado para **estudiantes universitarios** de ciencias, ingeniería o áreas afines que deseen comprender cómo modelar, analizar y visualizar sistemas conectados: desde redes sociales y biológicas hasta infraestructuras tecnológicas. No se requieren conocimientos previos en teoría de grafos, pero sí es útil tener una base en estadística descriptiva y programación en R.

¿Qué vas a aprender?

Objetivos generales del libro

Al finalizar estas notas serás capaz de:

1. **Crear y manipular** redes (grafos) en R usando `igraph`.
2. **Calcular e interpretar** métricas fundamentales: grado, distancia, clustering y centralidad.
3. **Distinguir** modelos de redes (Erdős–Rényi, Barabási–Albert, *small-world*) y sus propiedades.
4. **Evaluar** la robustez de una red ante fallos aleatorios y ataques dirigidos.
5. **Comunicar** hallazgos mediante visualizaciones claras y reproducibles.

Estructura del libro

Table 1: Mapa del libro

Capítulo	Tema	Pregunta guía
1	Fundamentos de igraph	¿Cómo se construye y representa una red en R?
2	Grado y distribuciones	¿Por qué algunas redes tienen “superhubs” y otras no?
3	Distancias y eficiencia	¿Qué tan fácil es ir de un punto a otro en una red?
4	Clustering y transitividad	¿Los amigos de tus amigos son tus amigos?
5	Robustez de redes	¿Qué pasa si falla un nodo crítico?
Integradores	Ejercicios finales	¿Puedes analizar una red completa de principio a fin?

Prerrequisitos

! Antes de empezar necesitas

- **R** (versión 4.1) y **RStudio** instalados.
- Conocimientos básicos de R: vectores, funciones, gráficos con `plot()`.
- Nociones elementales de estadística descriptiva (media, varianza, histogramas).
- Familiaridad básica con álgebra de matrices (útil, pero no indispensable).

Paquetes necesarios

Ejecuta el siguiente bloque para instalar todos los paquetes que usaremos a lo largo del libro:

```
paquetes <- c("igraph", "tidyverse", "powerlaw", "gt", "knitr")

# Instalar solo los que no tengas
nuevos <- paquetes[!(paquetes %in% installed.packages()[, "Package"])]
if (length(nuevos)) install.packages(nuevos)
```


Convenciones del libro

A lo largo del texto encontrarás estos bloques informativos:



Concepto clave

Resalta un concepto o idea fundamental que debes recordar.



Error frecuente

Señala errores comunes que cometen lxs estudiantes y cómo evitarlos.



Pista

Pistas progresivas para resolver ejercicios. Haz clic para expandir.

Cómo usar este libro

1. **Lee la teoría** y ejecuta los ejemplos en tu sesión de R.
2. **Intenta los ejercicios** antes de ver las soluciones o pistas.
3. **Experimenta**: modifica parámetros, cambia redes, rompe cosas.
4. Al final de cada capítulo, responde la **autoevaluación** para verificar tu comprensión.

¡Comencemos!

1 Fundamentos de igraph

i Objetivos de aprendizaje

Al finalizar este capítulo serás capaz de:

1. **Crear** redes vacías, agregar nodos y conexiones usando **igraph**.
2. **Manipular** atributos de vértices y conexiones (color, nombre, forma).
3. **Visualizar** redes con distintos *layouts* y personalizaciones.
4. **Construir** la matriz de adyacencia y representarla como mapa de calor.
5. **Generar** redes especiales: estrella, anillo, árbol, lattice.

Pregunta motivadora: Si representaras tu grupo de amigos como una red, donde cada persona es un punto y cada amistad es una línea, ¿cómo se vería? ¿Quién estaría más “conectado”? ¿Habría grupos aislados?

```
flowchart TD
    A([ Fundamentos de igraph ])
    subgraph CREAR [ Crear redes ]
        direction LR
        B1[Nodos y<br/>conexiones]
        B2[Atributos]
        B3[Visualización]
    end
    subgraph GRADO [ Grado y distribución ]
        direction LR
        C1[Grado de<br/>entrada / salida]
        C2[Histogramas]
    end
    subgraph MATRIZ [ Matriz de adyacencia ]
        direction LR
        D1[Heatmap]
    end
```

```

subgraph ESPECIALES[" Redes especiales"]
    direction LR
    E1[Estrella]:::esp
    E2[Anillo]:::esp
    E3[Árbol]:::esp
end

A -->|define| B1
A -->|define| B2
A -->|define| B3
A -->|calcula| C1
A -->|visualiza| C2
A -->|representa| D1
A -->|incluye| E1
A -->|incluye| E2
A -->|incluye| E3
C1 -.->|se grafica en| C2
D1 -.->|refleja| B1

classDef central fill:#F39C12,stroke:#D68910,color:#fff,font-weight:bold
classDef crear fill:#2E86C1,stroke:#1A5276,color:#fff
classDef grado fill:#1E8449,stroke:#145A32,color:#fff
classDef matriz fill:#7D3C98,stroke:#6C3483,color:#fff
classDef esp fill:#C0392B,stroke:#922B21,color:#fff

```

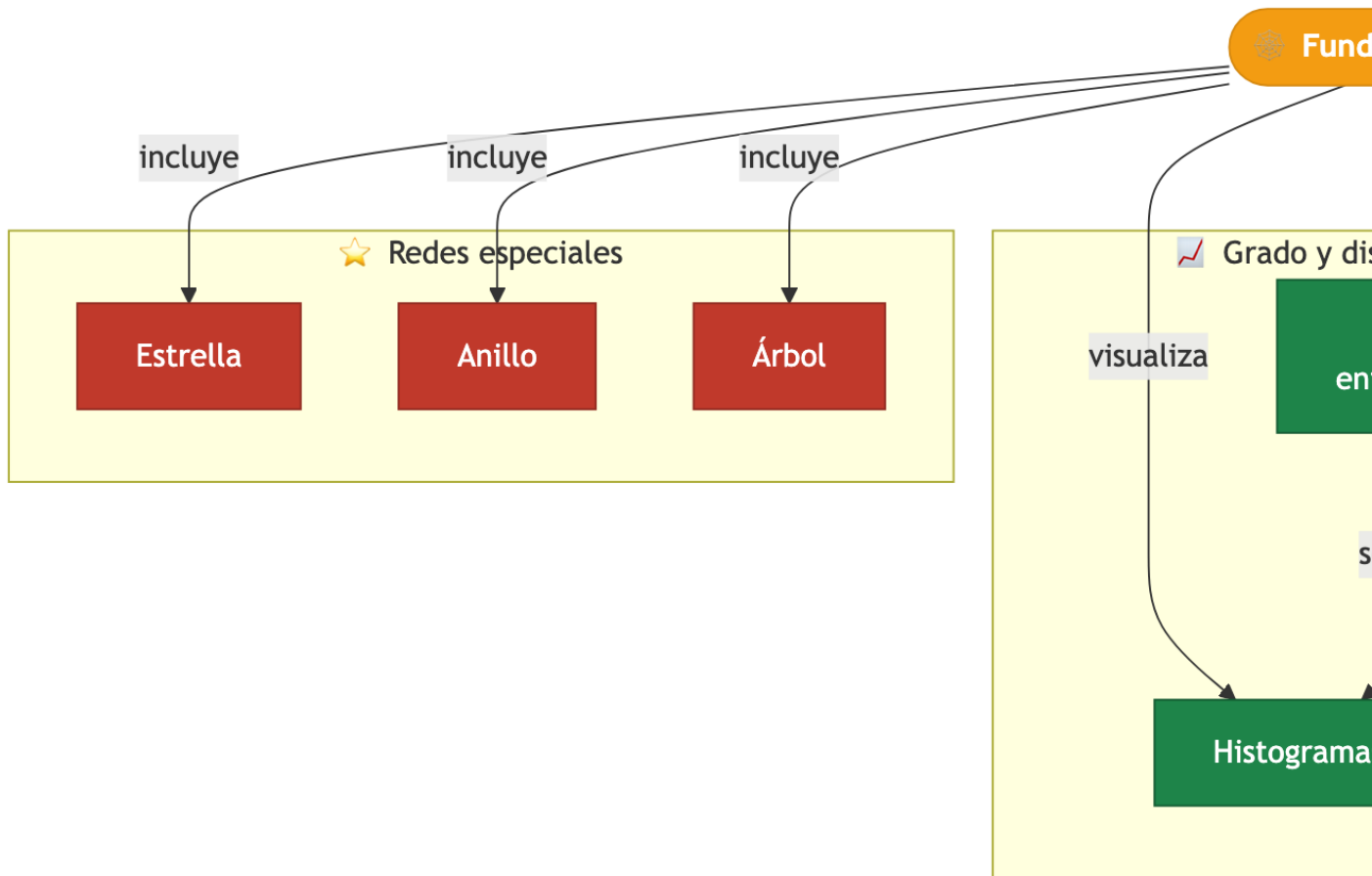


Figure 1.1: Mapa conceptual del capítulo

1.1 Preparación del entorno

```
library(igraph)
```

i Paquete requerido

Si no tienes instalado `igraph`, ejecuta:

```
install.packages("igraph")
```

1.2 Crear redes desde cero

Un **grafo** (o red) se compone de **vértices** (nodos) y **conexiones** (conexiones entre nodos). En **igraph**, todo comienza creando un grafo vacío y luego añadiendo elementos.

💡 Concepto clave

En **igraph**, los vértices se acceden con $V(g)$ y las conexiones con $E(g)$. Piensa en V de *vertices* y E de *edges*.

1.2.1 Paso 1: Grafo vacío

```
# Crear un grafo dirigido vacío con 5 nodos
g <- make_empty_graph(n = 5, directed = TRUE)

# Asignar atributos visuales a los nodos
V(g)$color <- "yellow"

# Visualizar
plot(g,
      main = "Red vacía con 5 nodos",
      vertex.size = 30)
```

Red vacía con 5 nodos



Figure 1.2: Red vacía con 5 nodos y sin conexiones

1.2.2 Paso 2: Agregar conexiones

Las conexiones se agregan como un vector de pares: `c(origen1, destino1, origen2, destino2, ...)`.

```
# Agregar conexiones: 1→2, 1→3, 2→4, 3→4, 4→5
g <- add_edges(g, c(1,2, 1,3, 2,4, 3,4, 4,5))

plot(g,
      main = "Red con conexiones agregadas",
      vertex.size = 30,
      edge.arrow.size = 0.6)
```

Red con conexiones agregadas

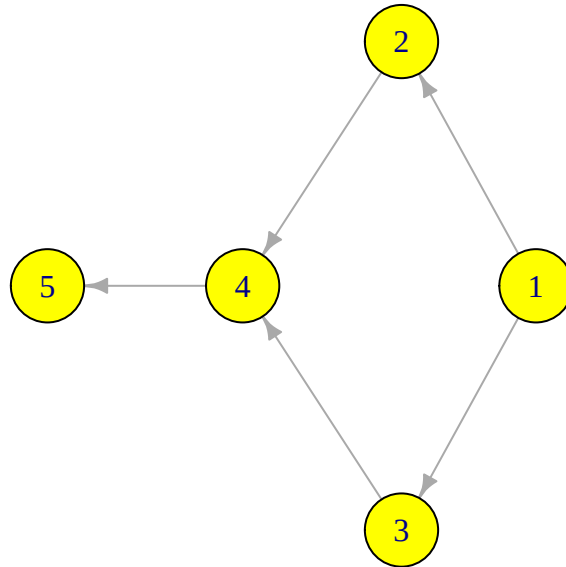


Figure 1.3: Red con conexiones agregadas manualmente

1.2.3 Paso 3: Agregar nodos con atributos

```
# Agregar un nodo nuevo con color rojo
g <- add_vertices(g, 1, color = "red")

# Conectar el nuevo nodo (nodo 6)
```

```
g <- add_edges(g, c(3,6, 6,5))

plot(g,
      main = "Nodo rojo y nuevas conexiones",
      vertex.size = 30,
      edge.arrow.size = 0.6)
```

Nodo rojo y nuevas conexiones

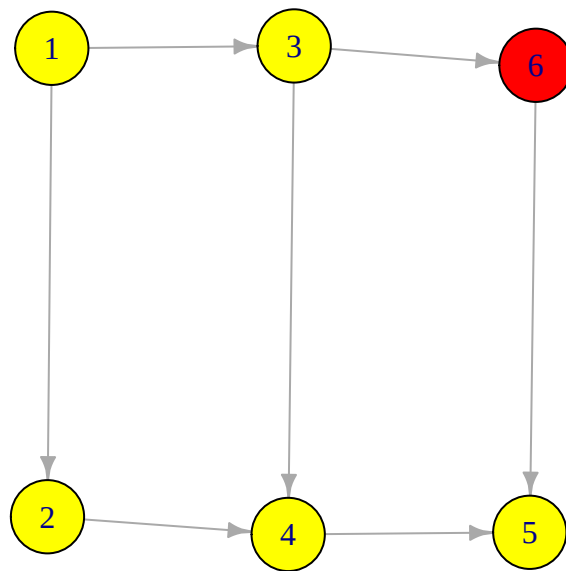


Figure 1.4: Nodo rojo agregado con nuevas conexiones

1.2.4 Paso 4: Nombrar los nodos

```
# Asignar nombres a los vértices
V(g)$name <- LETTERS[1:6]

plot(g,
      main = "Nodos con nombres A-F",
      vertex.size = 30,
      edge.arrow.size = 0.6)
```

Nodos con nombres A-F

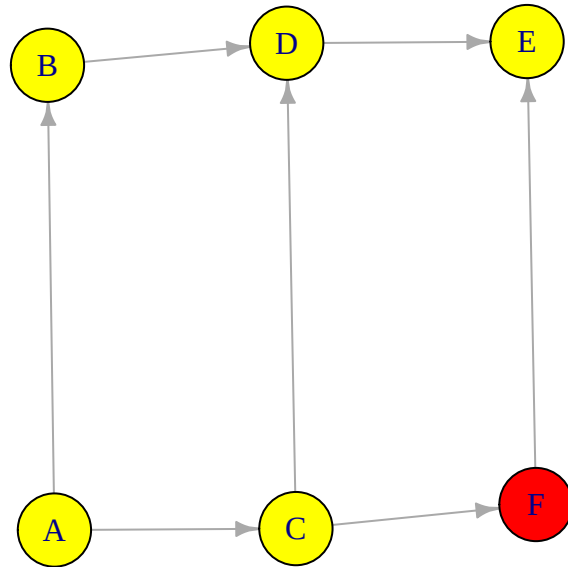


Figure 1.5: Red con nodos etiquetados de A a F

1.2.5 Paso 5: Modificar conexiones

```
# Eliminar la segunda conexión (A→C) y agregar C→A
g <- delete_edges(g, 2)
g <- add_edges(g, c(3, 1)) # C → A

plot(g,
     main = "Cambio en dirección de conexión",
     vertex.size = 30,
     edge.arrow.size = 0.6)
```


Cambio en dirección de conexión

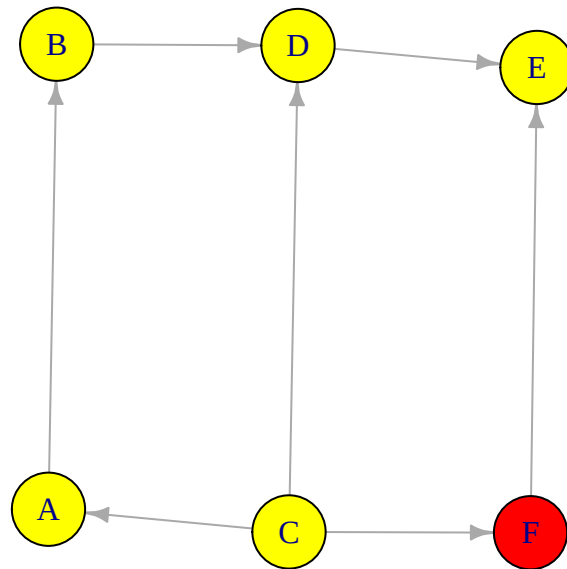


Figure 1.6: Red con dirección de conexión modificada

⚠ Error frecuente

Cuando eliminas conexiones con `delete_edges()`, los **índices se reorganizan**. Si eliminas la conexión 2, la que antes era la 3 ahora es la 2. Siempre verifica con `E(g)` después de eliminar.

1.3 grado (degree) y distribución de grado (degree) {#sec-grado (degree)-basico}

El **grado (degree)** de un nodo es el número de conexiones que tiene. En redes dirigidas, distinguimos entre grado (degree) de entrada (*in-degree*) y de salida (*out-degree*).

1.3.1 grado (degree) total

```
# grado (degree) total de cada nodo  
degree(g)
```

```
A B C D E F  
2 2 3 3 2 2
```

1.3.2 grado (degree) de entrada

```
# ¿Cuántas conexiones llegan a cada nodo?  
degree(g, mode = "in")
```

```
A B C D E F  
1 1 0 2 2 1
```

1.3.3 grado (degree) de salida

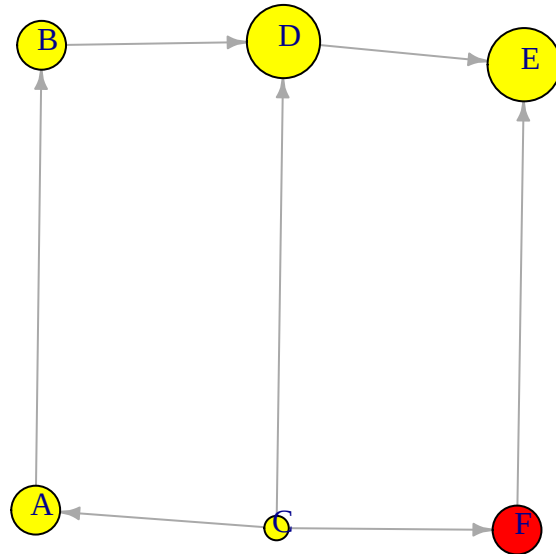
```
# ¿Cuántas conexiones salen de cada nodo?  
degree(g, mode = "out")
```

```
A B C D E F  
1 1 3 1 0 1
```

1.3.4 Visualizar el grado (degree)

```
plot(g,  
      layout = layout_nicely(g),  
      vertex.size = degree(g, mode = "in") * 10 + 10,  
      vertex.label.dist = 0.5,  
      edge.arrow.size = 0.5,  
      main = "Tamaño proporcional al grado (degree) de entrada")
```

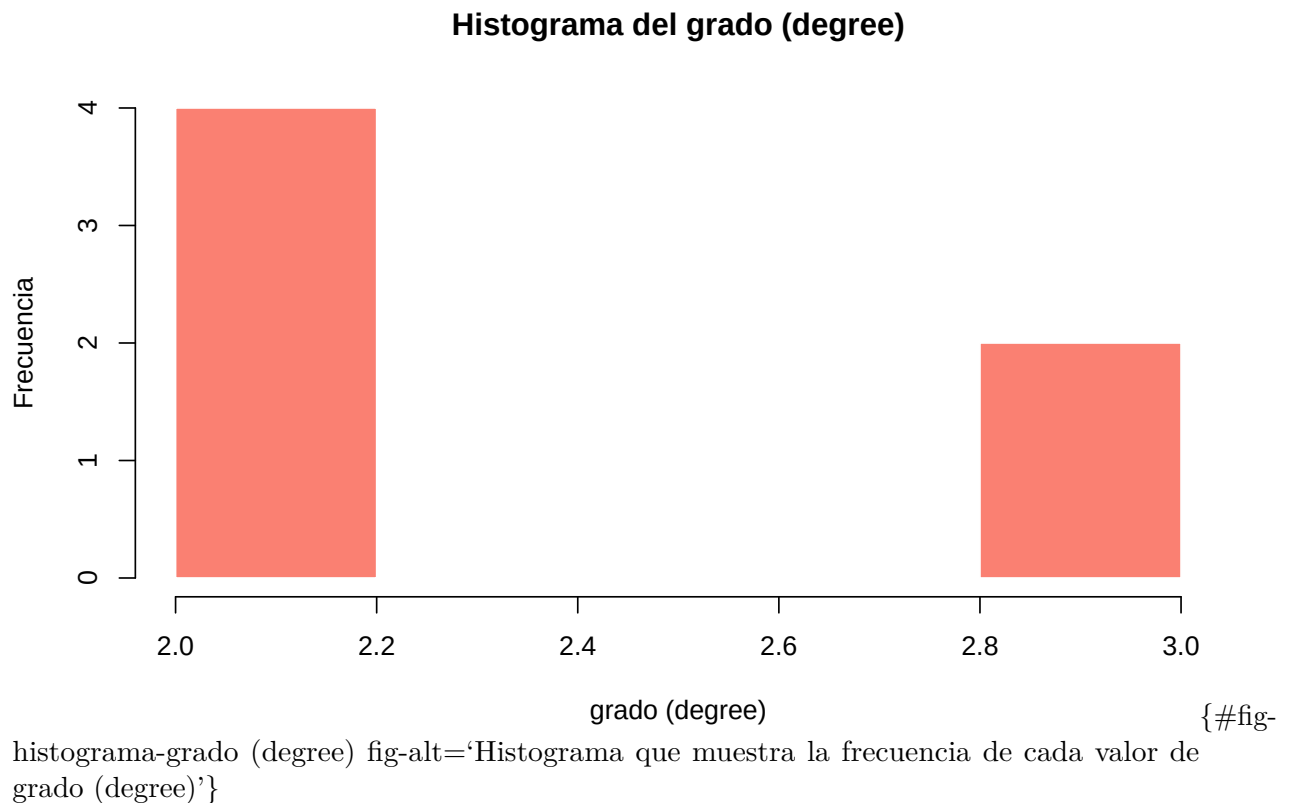
Tamaño proporcional al grado (degree) de entrada



{#fig-tamano-grado

(degree) fig-alt='Red donde los nodos más grandes reciben más conexiones'}

```
hist(degree(g),  
      col = "salmon",  
      main = "Histograma del grado (degree)",  
      xlab = "grado (degree)",  
      ylab = "Frecuencia",  
      border = "white")
```



1.4 Matriz de adyacencia

La **matriz de adyacencia** es una representación numérica de la red: un 1 en la posición (i, j) indica que existe una conexión del nodo i al nodo j .

$$A_{ij} = \begin{cases} 1 & \text{si existe conexión de } i \text{ a } j \\ 0 & \text{en otro caso} \end{cases}$$

```
# Obtener la matriz de adyacencia
adj_mat <- as.matrix(get.adjacency(g))

# Mostrar como tabla
knitr::kable(adj_mat, caption = "Matriz de adyacencia de la red")
```

Table 1.1: Matriz de adyacencia de la red

	A	B	C	D	E	F
A	0	1	0	0	0	0
B	0	0	0	1	0	0
C	1	0	0	1	0	1
D	0	0	0	0	1	0
E	0	0	0	0	0	0
F	0	0	0	0	1	0

```
heatmap(adj_mat,
  Rowv = NA, Colv = NA,
  col = c("white", "steelblue"),
  main = "Mapa de calor - Matriz de adyacencia",
  scale = "none")
```

Mapa de calor – Matriz de adyacencia

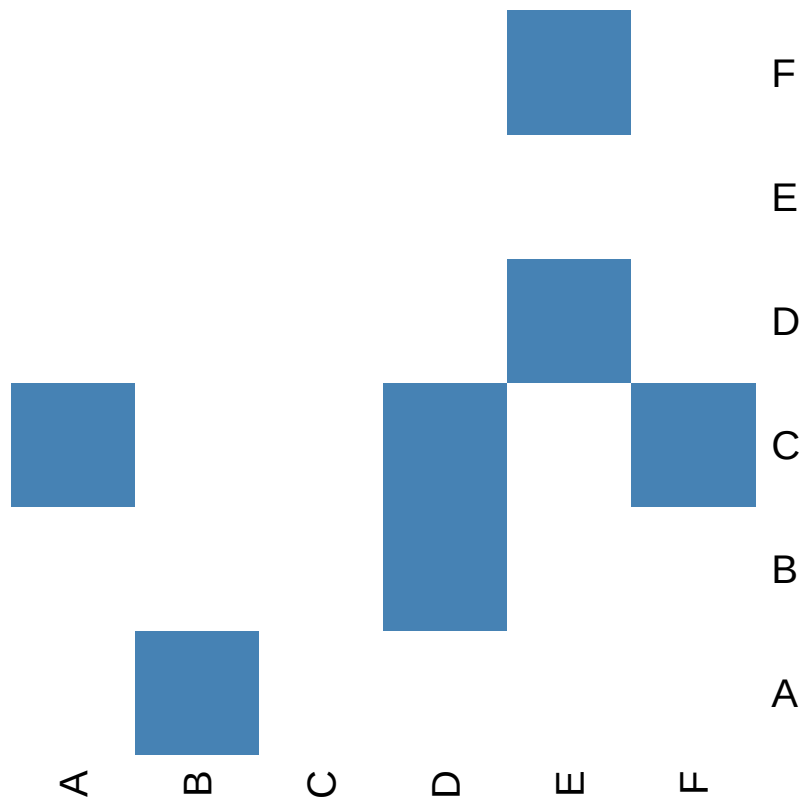


Figure 1.7: Mapa de calor de la matriz de adyacencia

💡 Concepto clave

En una red **no dirigida**, la matriz de adyacencia es **simétrica** ($A_{ij} = A_{ji}$). En una red dirigida, no necesariamente lo es. Verifica esto convirtiendo la red con `as.undirected(g)`.

1.5 Redes especiales

`igraph` incluye funciones para crear topologías clásicas que aparecen frecuentemente en la teoría de redes.

1.5.1 Estrella

```
plot(make_star(10),  
     vertex.color = c("tomato", rep("gold", 9)),  
     vertex.size = 20,  
     main = "Red estrella")
```

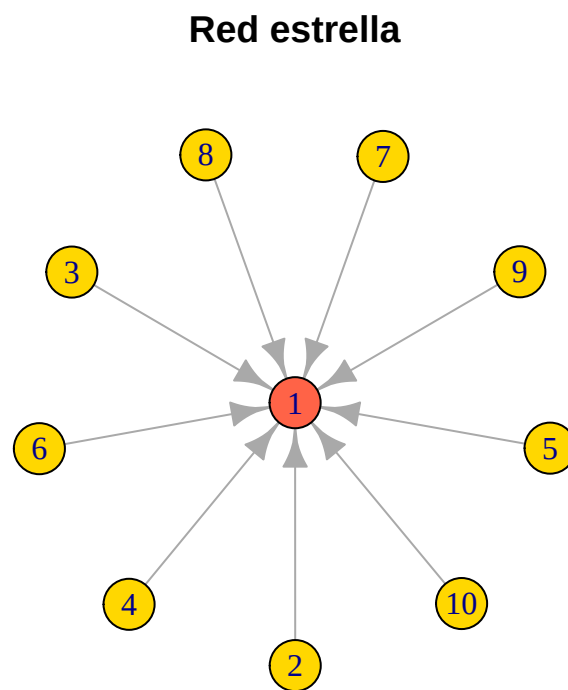


Figure 1.8: Red tipo estrella con 10 nodos

1.5.2 Anillo

```
plot(make_ring(10),  
     vertex.color = "skyblue",  
     vertex.size = 20,  
     main = "Red anillo")
```

Red anillo

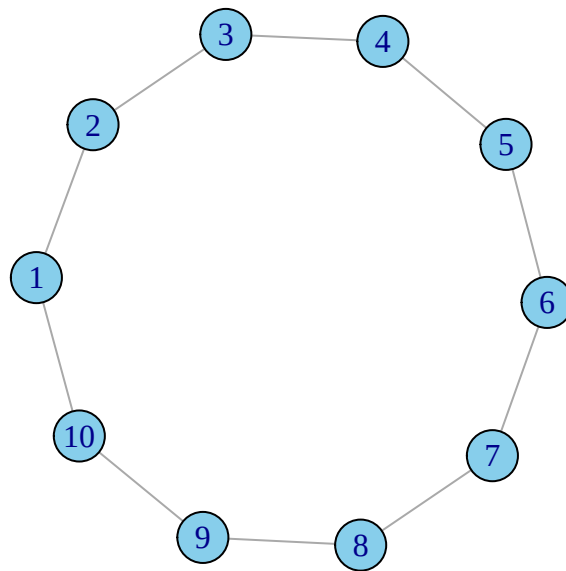


Figure 1.9: Red tipo anillo con 10 nodos

1.5.3 Árbol binario

```
plot(make_tree(15, children = 2),  
     vertex.color = "lightgreen",  
     vertex.size = 15,  
     layout = layout_as_tree,  
     main = "Árbol binario")
```

Árbol binario

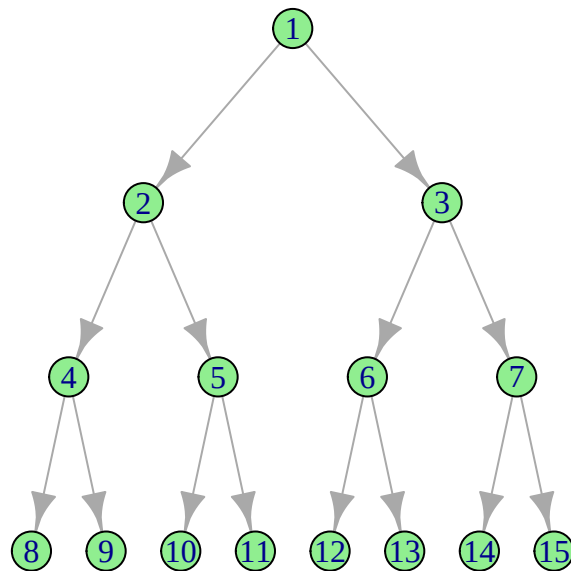


Figure 1.10: Árbol binario con 15 nodos

1.5.4 Lattice

```
plot(make_lattice(c(4, 4)),  
     vertex.color = "plum",  
     vertex.size = 20,  
     main = "Lattice 4×4")
```


Lattice 4×4

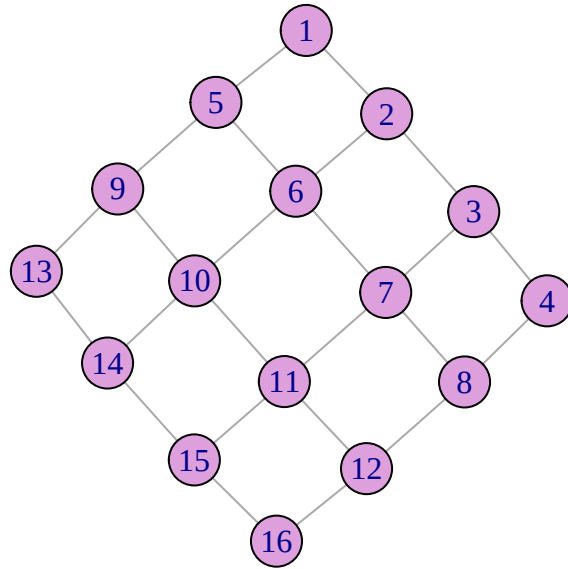


Figure 1.11: Red tipo rejilla (lattice) de 4×4

1.6 Ejercicios

Ejercicios resueltos

Ejercicio resuelto 1: Crea una red no dirigida de 7 nodos con colores aleatorios de una paleta, conéctalos formando una cadena (*path*), y visualiza el resultado.

💡 Solución

Paso 1: Crear el grafo tipo cadena y asignar colores.

```

set.seed(42)

# Crear cadena de 7 nodos
#g_cadena <- make_graph(~ 1-2-3-4-5-6-7, directed = FALSE)
g_cadena <- graph_from_literal(1-2-3-4-5-6-7)
# Asignar colores aleatorios de una paleta
paleta <- c("tomato", "gold", "skyblue", "lightgreen", "plum", "salmon", "steelblue")
V(g_cadena)$color <- sample(paleta, 7)
V(g_cadena)$name <- paste0("N", 1:7)

```

Paso 2: Visualizar la red.

```

plot(g_cadena,
     vertex.size = 30,
     main = "Red tipo cadena (path)")

```

Red tipo cadena (path)

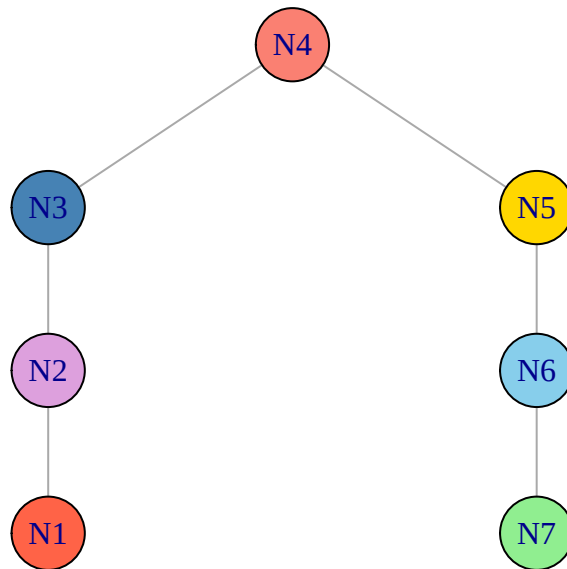


Figure 1.12: Red tipo cadena con 7 nodos y colores aleatorios

Paso 3: Verificar propiedades.

```
cat("Número de nodos:", vcount(g_cadena), "\n")
```

Número de nodos: 7

```
cat("Número de conexiones:", ecoun(g_cadena), "\n")
```

Número de conexiones: 6

```
cat("grado (degree) de cada nodo:", degree(g_cadena), "\n")
```

grado (degree) de cada nodo: 1 2 2 2 2 2 1

```
cat("¿Es conexa?", is_connected(g_cadena), "\n")
```

¿Es conexa? TRUE

Interpretación: La cadena tiene 7 nodos y 6 conexiones. Los nodos extremos tienen grado (degree) 1 y los internos grado (degree) 2. La red es conexa (todos los nodos se pueden alcanzar entre sí).

Ejercicio resuelto 2: Construye manualmente una red donde los nodos representen 5 ciudades mexicanas y las conexiones indiquen si comparten frontera estatal. Muestra la matriz de adyacencia.

 Solución

Paso 1: Definir las ciudades y sus conexiones fronterizas.

```
# Crear la red
ciudades <- graph_from_literal(
  Querétaro -- Guanajuato,
  Querétaro -- "San Luis Potosí",
  Querétaro -- Hidalgo,
  Guanajuato -- "San Luis Potosí",
  Hidalgo -- "Estado de México"
)

V(ciudades)$color <- "gold"
V(ciudades)$size <- 25
```

Paso 2: Visualizar y mostrar la matriz.

```
plot(ciudades,
     vertex.label.cex = 0.8,
     main = "Red de vecindad entre estados")
```

Red de vecindad entre estados

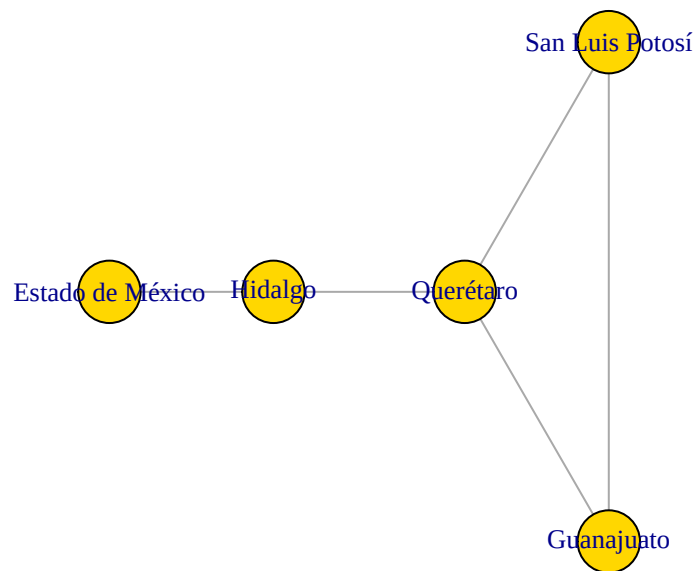


Figure 1.13: Red de ciudades que comparten frontera estatal

```
knitr::kable(
  as.matrix(get.adjacency(ciudades)),
  caption = "Matriz de adyacencia de la red de estados"
)
```

Table 1.2: Matriz de adyacencia de la red de estados

	Queré- taro	Guanaju- ato	San Luis Potosí	Hidalgo	Estado de México
Querétaro	0	1	1	1	0
Guanajuato	1	0	1	0	0
San Luis Potosí	1	1	0	0	0
Hidalgo	1	0	0	0	1
Estado de México	0	0	0	1	0

Interpretación: Querétaro es el nodo más conectado (grado (degree) 3) porque comparte frontera con tres estados. La matriz es simétrica porque la red es no dirigida.

Ejercicios con pistas

Ejercicio 1: Crea una red vacía de 7 nodos no dirigidos. Agrega conexiones para formar un triángulo entre los nodos 1, 2 y 3, y otro triángulo entre 5, 6 y 7. Conecta ambos triángulos con una conexión entre 3 y 5. Visualiza el resultado y calcula el grado (degree) de cada nodo.

i Pista 1

Usa `make_empty_graph(n = 7, directed = FALSE)` y luego `add_edges()` con el vector de pares.

i Pista 2

El vector de conexiones sería: `c(1,2, 2,3, 1,3, 5,6, 6,7, 5,7, 3,5)`. Son 7 conexiones en total.

i Pista 3 — Pseudocódigo

```
1. g <- make_empty_graph(7, directed = FALSE)
2. g <- add_edges(g, c(1,2, 2,3, ...))
3. plot(g)
4. degree(g)
```

Ejercicio 2: Genera un `make_tree()` con 3 hijos por nodo y profundidad suficiente para tener al menos 40 nodos. ¿Cuántos nodos tiene exactamente? ¿Cuántas hojas (nodos con grado (degree) 1)?

i Pista 1

En `make_tree(n, children)`, `n` es el número total de nodos. Un árbol con 3 hijos y `n = 40` tendrá exactamente 40 nodos.

i Pista 2

Las hojas son los nodos con grado (degree) 1. Filtra con: `sum(degree(arbol) == 1)`.

i Pista 3 — Pseudocódigo

```
1. arbol <- make_tree(40, children = 3)
2. vcount(arbol)
3. sum(degree(arbol) == 1)
4. plot(arbol, layout = layout_as_tree, vertex.size = 5)
```

Ejercicio 3: ¿Qué pasa si agregas un *self-loop* (conexión de un nodo a sí mismo) y una conexión duplicada? Crea una red, agrégalos, visualiza, y luego “limpia” la red con `simplify()`. Compara el antes y después.

i Pista 1

Usa `add_edges(g, c(1,1))` para un *self-loop* y repite una conexión existente para duplicarla.

i Pista 2

`simplify(g, remove.multiple = TRUE, remove.loops = TRUE)` elimina duplicados y *self-loops*.

i Pista 3 — Pseudocódigo

```
1. g <- make_ring(5)
2. g <- add_edges(g, c(1,1, 1,2)) # loop + duplicada
3. ecount(g) # contar conexiones antes
4. plot(g)
5. g_limpio <- simplify(g)
6. ecount(g_limpio) # contar conexiones después
7. plot(g_limpio)
```

Ejercicios propuestos

Ejercicio propuesto 1

Crea una red estrella de 15 nodos. Calcula su grado (degree) máximo y mínimo. ¿Cuál es la densidad de conexiones (`edge_density()`)?

Autoevaluación: El grado (degree) máximo debe ser 14 (el nodo central) y el mínimo 1 (nodos periféricos).

Ejercicio propuesto 2

Genera tres redes con `sample_gnp(50, p)` usando $p = 0.05$, $p = 0.20$ y $p = 0.50$. Compara sus histogramas de grado (degree) en un gráfico con `par(mfrow = c(1, 3))`. ¿Qué patrón observas a medida que p aumenta?

Autoevaluación: A mayor p , la distribución de grado (degree) se desplaza a la derecha y se vuelve más concentrada. Con $p = 0.50$ el grado (degree) medio debe estar cerca de 25.

Ejercicio propuesto 3

Diseña una red hipotética de 8 ingredientes de cocina donde las conexiones representen que dos ingredientes aparecen juntos en una misma receta. Usa `graph_from_literal()` para definirla. Visualízala y calcula la matriz de distancias entre todos los pares.

Autoevaluación: Todos los nodos deben ser alcanzables entre sí (la red debe ser conexa). Si alguna distancia es `Inf`, tienes componentes desconectados.

Ejercicio propuesto 4

Crea una función en R llamada `resumen_red()` que reciba un grafo `g` y devuelva un *data frame* con las columnas: `nodo`, `grado_total`, `grado_entrada`, `grado_salida`. Pruébala con una red dirigida de al menos 10 nodos.

Autoevaluación: Tu función debe manejar correctamente tanto redes dirigidas como no dirigidas (en las no dirigidas, los tres grados son iguales).

1.7 Resumen del capítulo

```
flowchart LR
  A[Crear redes] --> B[Manipular nodos y conexiones]
  B --> C[Calcular grado (degree)]
  C --> D[Matriz de adyacencia]
  D --> E[Redes especiales]
  E --> F["Siguiente: Distribuciones de grado (degree) →"]
  style F fill:#e8f4f8,stroke:#3ABEF9
```

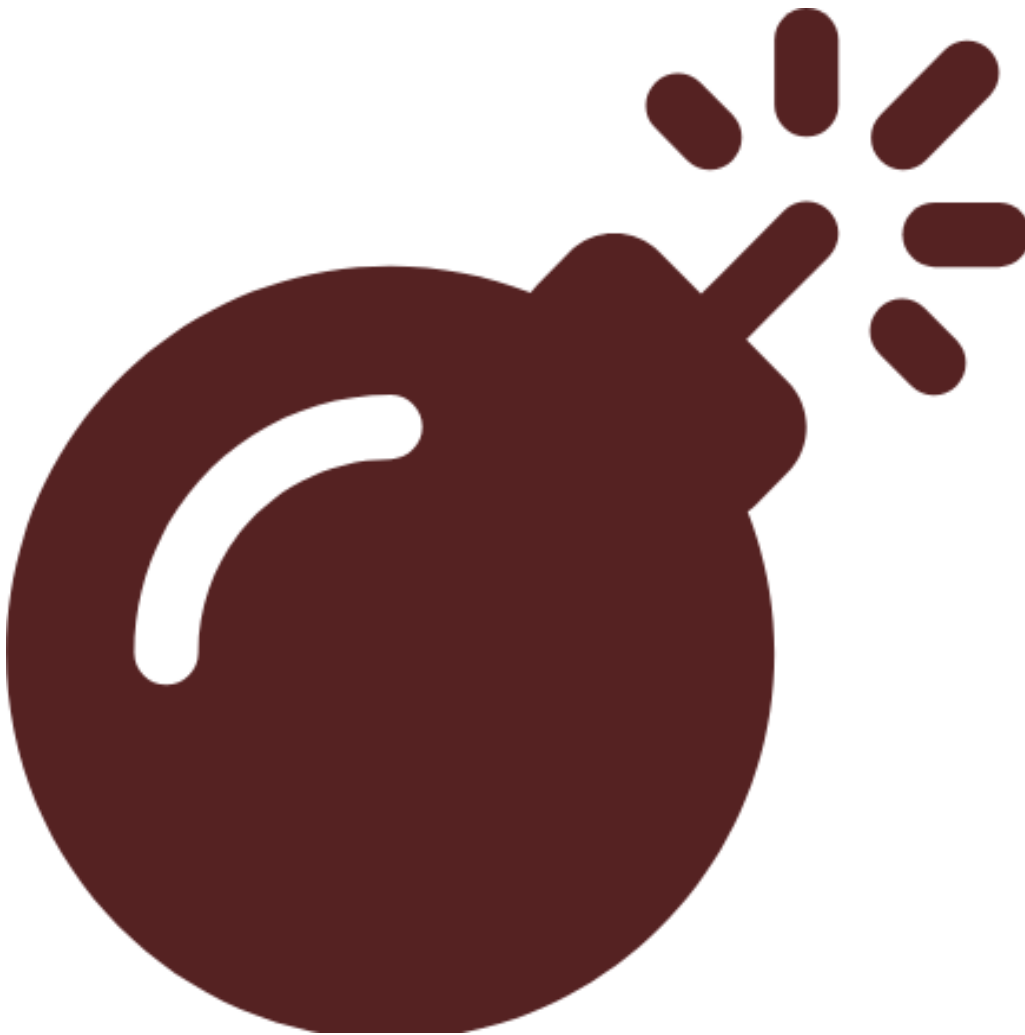



Figure 1.14: Resumen visual del capítulo

Table 1.3: Funciones clave del capítulo

Función	Descripción
<code>make_empty_graph()</code>	Crear grafo vacío
<code>add_vertices()</code> / <code>add_edges()</code>	Agregar nodos / conexiones
<code>delete_vertices()</code> / <code>delete_edges()</code>	Eliminar nodos / conexiones
<code>V(g)</code> / <code>E(g)</code>	Acceder a vértices / conexiones
<code>degree()</code>	Calcular grado (degree)
<code>get.adjacency()</code>	Obtener matriz de adyacencia
<code>make_star()</code> , <code>make_ring()</code> , <code>make_tree()</code>	Crear redes especiales

Conexión con el siguiente capítulo: Ahora que sabes crear redes y calcular grados, en el **?@sec-grado** (degree)-distribuciones exploraremos *cómo se distribuyen* esos grados y por qué algunas redes producen “superhubs” mientras que otras son más homogéneas.

Autoevaluación

Pregunta 1: Verdadero o Falso: En una red no dirigida, ¿la matriz de adyacencia siempre es simétrica?

 Respuesta 1

Verdadero. Si i está conectado con j , entonces j también está conectado con i .

Pregunta 2: ¿Cuál es el grado (degree) del nodo central en una red estrella de n nodos?

- a) 1
- b) n
- c) $n - 1$
- d) 2

 Respuesta 2

¿Cuál es el grado (degree) del nodo central en una red estrella de n nodos?

- a) 1
- b) n
- c) $n - 1$
- d) 2

El nodo central se conecta con todos los demás, así que su grado (degree) es $n - 1$.

 Pregunta 3

¿Qué función usarías para eliminar *self-loops* y conexiones duplicadas de una red?

`simplify()` con los parámetros `remove.multiple = TRUE` y `remove.loops = TRUE`.

 Pregunta 4

Verdadero o Falso: `add_edges(g, c(1,2,3))` agrega tres conexiones.

Falso. El vector debe tener un número par de elementos (pares origen-destino). Este código produciría un error.

 Pregunta 5

¿Cuántas conexiones tiene un `make_ring(10)`?

Exactamente **10**: cada nodo se conecta con su vecino y el último se conecta con el primero, formando un ciclo.

2 Grado y distribuciones

i Objetivos de aprendizaje

Al finalizar este capítulo serás capaz de:

1. **Diferenciar** entre distribuciones de grado tipo Poisson y *power-law* (ley de potencia).
2. **Generar** redes con los modelos Erdős–Rényi (ER) y Barabási–Albert (BA) en `igraph`.
3. **Visualizar y comparar** distribuciones de grado usando histogramas y gráficos log-log.
4. **Ajustar** una ley de potencia a datos de grado con el paquete `powerLaw`.
5. **Identificar** *hubs* (nodos altamente conectados) y explicar su importancia en la estructura de la red.

Pregunta motivadora: En internet, unos pocos sitios web (Google, Facebook, Wikipedia) reciben miles de millones de visitas, mientras que la inmensa mayoría recibe muy pocas. ¿Por qué esta desigualdad extrema? ¿Es azar o hay un mecanismo detrás?

```
flowchart TD
    A([Distribuciones de grado<br/>en redes complejas]):::central

    subgraph MOD[" Modelos generativos"]
        direction LR
        B[Erdős–Rényi<br/>red aleatoria]:::modelo
        C[Barabási–Albert<br/>crecimiento preferencial]:::modelo
    end

    subgraph DIST[" Distribuciones resultantes"]
        direction LR
        D1[{Distribución<br/>Poisson}]:::dist
        D2[{Distribución<br/>Power-law}]:::dist
    end

    subgraph PROP[" Propiedades estructurales"]
```

```

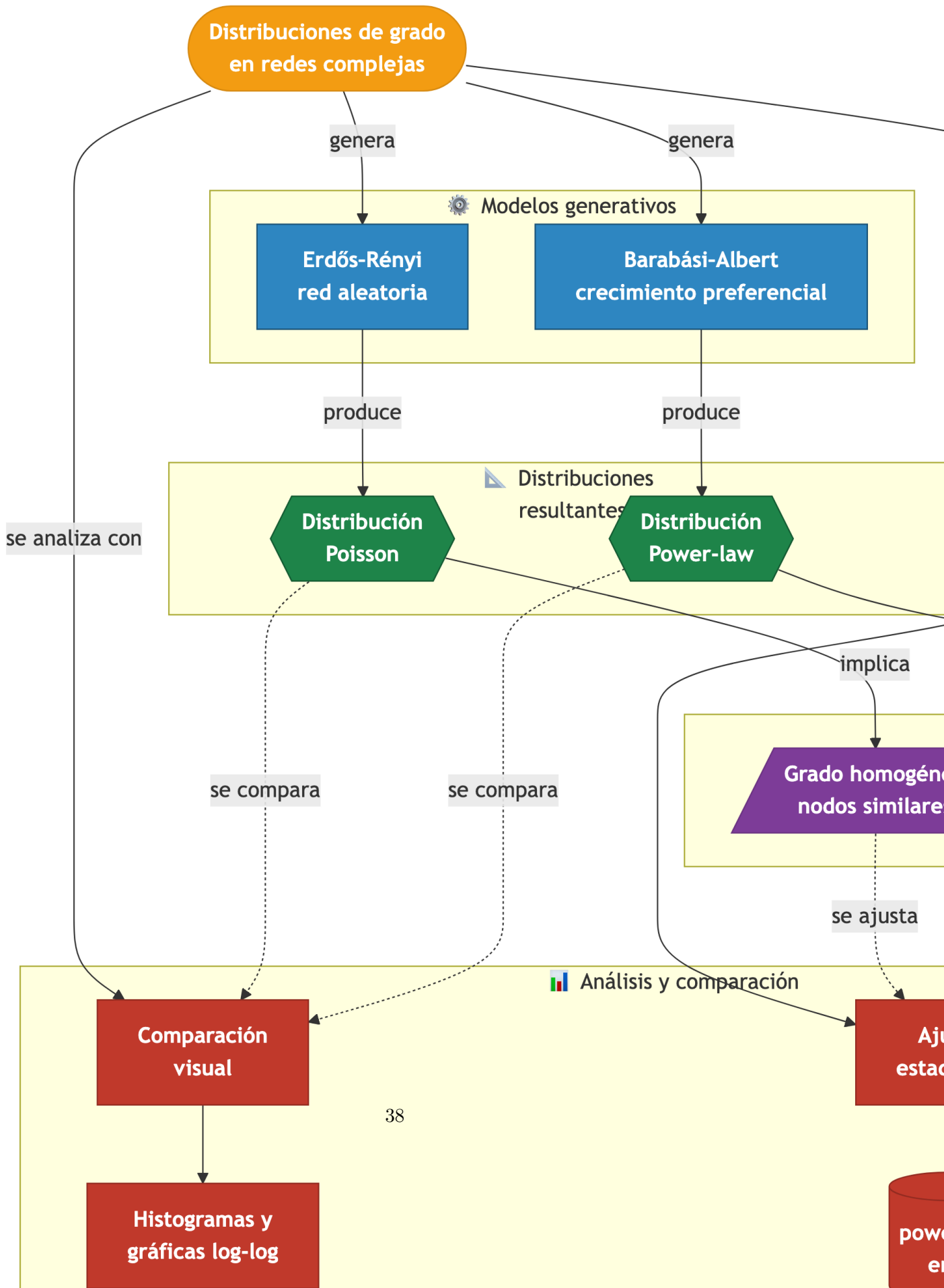
    direction LR
    F[/Grado homogéneo<br/>nodos similares/]:::prop
    G[/Hubs y colas pesadas<br/>alta conectividad/]:::prop
end

subgraph ANAL[" Análisis y comparación"]
    direction LR
    H[Comparación<br/>visual]:::anal
    I[Histogramas y<br/>gráficas log-log]:::anal
    J[Ajuste<br/>estadístico]:::anal
    K[(poweRlaw<br/>en R)]:::anal
end

A -->|genera| B
A -->|genera| C
A -->|se analiza con| H
A -->|se ajusta con| J
B -->|produce| D1
C -->|produce| D2
D1 -->|implica| F
D2 -->|implica| G
H --> I
J --> K
D1 -. se compara .-> H
D2 -. se compara .-> H
F -. se ajusta .-> J
G -. se ajusta .-> J

classDef central fill:#F39C12,stroke:#D68910,color:#fff,font-weight:bold
classDef modelo fill:#2E86C1,stroke:#1A5276,color:#fff,font-weight:bold
classDef dist fill:#1E8449,stroke:#145A32,color:#fff,font-weight:bold
classDef prop fill:#7D3C98,stroke:#6C3483,color:#fff,font-weight:bold
classDef anal fill:#C0392B,stroke:#922B21,color:#fff,font-weight:bold

```



2.1 Preparación del entorno

```
library(igraph)
library(powerLaw)
```

Paquete requerido

Si no tienes instalado `powerLaw`, ejecuta:

```
install.packages("powerLaw")
```

Este paquete permite ajustar y evaluar distribuciones de ley de potencia sobre datos empíricos.

2.2 Dos modelos, dos mundos

Antes de sumergirnos en los datos, necesitas entender dos modelos fundamentales de generación de redes. Cada uno produce redes con propiedades radicalmente distintas.

2.2.1 Modelo Erdős–Rényi (ER): redes aleatorias

En este modelo, cada par de nodos tiene una **probabilidad fija** p de estar conectado, independientemente de todo lo demás. Es como lanzar una moneda (con probabilidad p de “cara”) por cada par posible de nodos.

$$P(\text{arista entre } i \text{ y } j) = p, \quad \forall i \neq j$$

El resultado: la mayoría de los nodos terminan con un grado similar, cercano al promedio $\langle k \rangle \approx p \cdot (n - 1)$, y la distribución de grado sigue una **distribución de Poisson** (Erdős & Rényi, 1959).

Concepto clave

En una red Erdős–Rényi, la distribución de grado es tipo **Poisson**: la mayoría de los nodos tienen un grado parecido al promedio y es muy raro encontrar nodos con grado extremadamente alto o bajo. Es una red “democrática”.

2.2.2 Modelo Barabási–Albert (BA): enlace preferencial

Este modelo introduce un mecanismo más realista: **los nodos nuevos prefieren conectarse a nodos que ya tienen muchas conexiones** (“los ricos se hacen más ricos”). Esto se llama *preferential attachment* (enlace preferencial).

$$P(\text{conectar a nodo } i) = \frac{k_i}{\sum_j k_j}$$

donde k_i es el grado del nodo i . El resultado: una distribución de grado tipo **ley de potencia** (*power-law*) donde unos pocos nodos (*hubs*) acumulan una cantidad desproporcionada de conexiones (Albert & Barabási, 2002; Barabási, 2016).

Concepto clave

En una red Barabási–Albert, la distribución de grado sigue una **ley de potencia**: $P(k) \sim k^{-\gamma}$. Esto significa que hay muchos nodos con pocas conexiones y unos pocos *hubs* con muchísimas. Es una red “desigual”.

2.2.3 Comparación rápida

Table 2.1: Comparación entre modelos de redes

Característica	Erdős–Rényi (ER)	Barabási–Albert (BA)
Mecanismo	Conexión aleatoria con prob. p	Enlace preferencial
Distribución de grado	Poisson	<i>Power-law</i>
Presencia de <i>hubs</i>	Muy improbable	Sí, emergentes
Ejemplo real	Red de carreteras	Internet, redes sociales
Función en igraph	<code>sample_gnp()</code>	<code>sample_pa()</code>

2.3 Crear y visualizar ambas redes

Vamos a generar una red de cada tipo con 100 nodos y compararlas visualmente.

```
set.seed(123)

# Red Erdős–Rényi: 100 nodos, probabilidad de conexión 0.04
g_er <- sample_gnp(100, p = 0.04)
V(g_er)$name <- paste0("ER_", 1:vcount(g_er))
```



```
# Red Barabási-Albert: 100 nodos, enlace preferencial
g_ba <- sample_pa(100, directed = FALSE)
V(g_ba)$name <- paste0("BA_", 1:vcount(g_ba))
```

```
par(mfrow = c(1, 2), mar = c(1, 1, 3, 1))
```

```
# Red ER
plot(g_er,
     vertex.size = degree(g_er) * 1.5 + 3,
     vertex.label = NA,
     vertex.color = "skyblue",
     edge.color = adjustcolor("gray50", alpha = 0.4),
     main = "Red Aleatoria (Erdős-Rényi)")
```

```
# Red BA
plot(g_ba,
     vertex.size = degree(g_ba) * 1.5 + 3,
     vertex.label = NA,
     vertex.color = "tomato",
     edge.color = adjustcolor("gray50", alpha = 0.4),
     main = "Red Power-law (Barabási-Albert)")
```

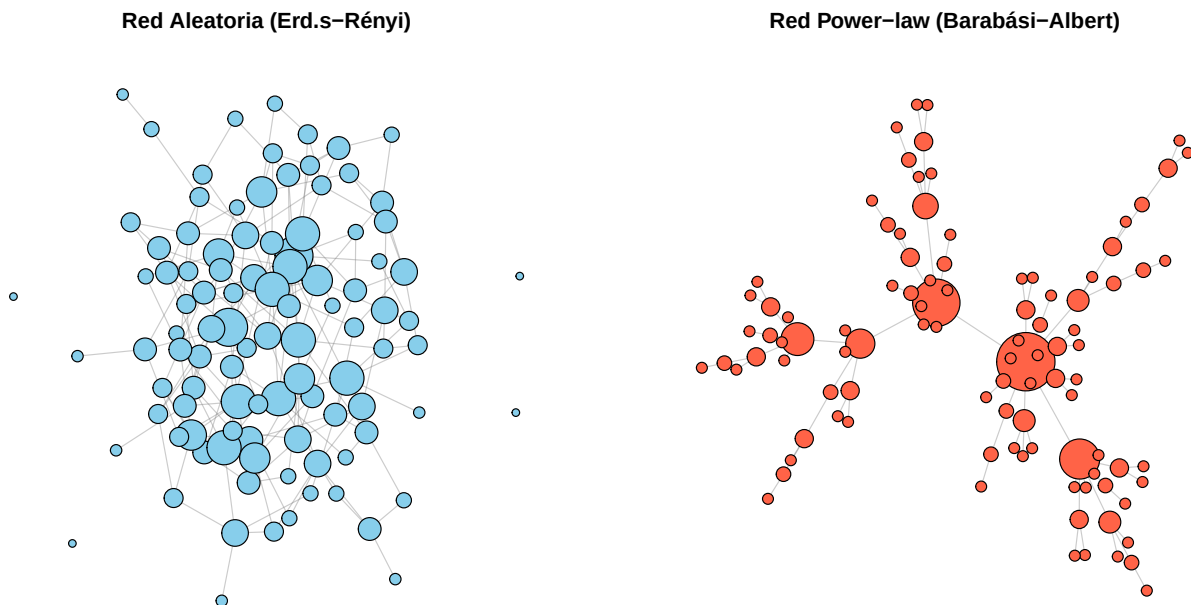


Figure 2.2: Comparación visual: red aleatoria (ER) vs. red de enlace preferencial (BA)

Observa la diferencia: en la red ER los nodos tienen tamaños similares, mientras que en la red BA unos pocos nodos destacan por ser mucho más grandes (más conectados).

2.4 Distribuciones de grado

Ahora veamos la distribución de grado de cada red. Aquí es donde la diferencia se vuelve evidente numéricamente.

2.4.1 Histogramas

```
deg_er <- degree(g_er)
deg_ba <- degree(g_ba)

par(mfrow = c(1, 2), mar = c(4, 4, 3, 1))

hist(deg_er,
      breaks = seq(-0.5, max(deg_er) + 0.5, by = 1),
      col = "skyblue", border = "white",
      main = "Grado - Red ER",
      xlab = "Grado (k)", ylab = "Frecuencia")
abline(v = mean(deg_er), col = "navy", lwd = 2, lty = 2)
legend("topright", paste("Media =", round(mean(deg_er), 1)),
      col = "navy", lty = 2, lwd = 2, bty = "n")

hist(deg_ba,
      breaks = seq(-0.5, max(deg_ba) + 0.5, by = 1),
      col = "tomato", border = "white",
      main = "Grado - Red BA",
      xlab = "Grado (k)", ylab = "Frecuencia")
abline(v = mean(deg_ba), col = "darkred", lwd = 2, lty = 2)
legend("topright", paste("Media =", round(mean(deg_ba), 1)),
      col = "darkred", lty = 2, lwd = 2, bty = "n")
```

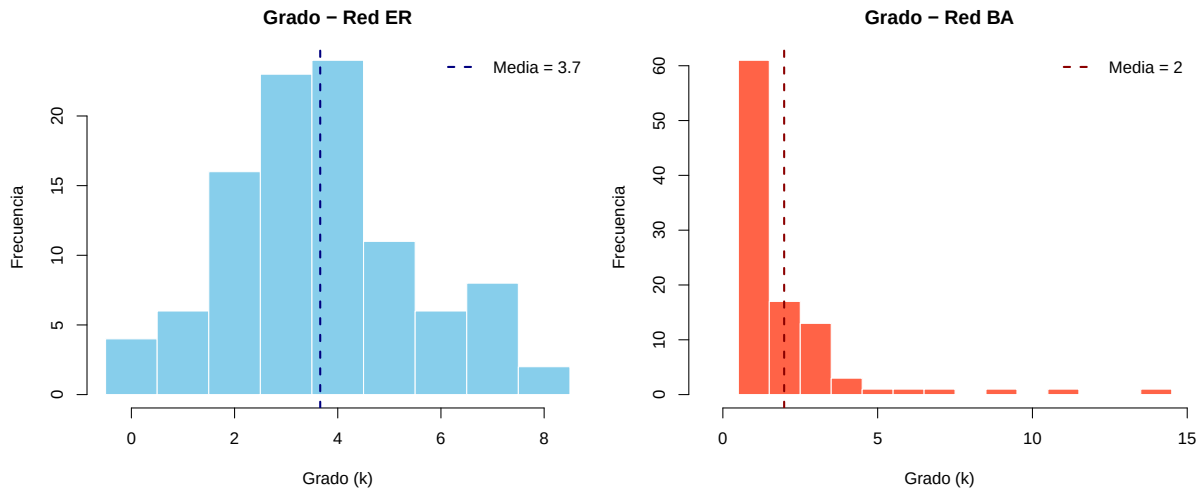


Figure 2.3: Histogramas de grado: Poisson (ER) vs. cola pesada (BA)



Concepto clave

La distribución de Poisson (ER) es simétrica y concentrada alrededor de la media. La distribución *power-law* (BA) tiene una **cola pesada** (*heavy tail*): la mayoría de nodos tienen grado bajo, pero unos pocos alcanzan grados extremadamente altos.

2.4.2 Gráfico log-log

La forma más clara de identificar una ley de potencia es graficar la distribución de grado en **escala logarítmica** en ambos ejes. Si la distribución es *power-law*, los puntos se alinean aproximadamente en una recta.

$$\log P(k) = -\gamma \log k + C \Rightarrow \text{recta en escala log-log}$$

```
par(mfrow = c(1, 2), mar = c(4, 4, 3, 1))

# ER en log-log
dd_er <- degree_distribution(g_er)
k_er <- which(dd_er > 0) - 1 # grados con frecuencia > 0
pk_er <- dd_er[dd_er > 0]

plot(k_er, pk_er,
     log = "xy", pch = 19, col = "skyblue", cex = 1.5,
     main = "Log-log - Red ER",
```

```

xlab = "log(k)", ylab = "log(P(k))")

# BA en log-log
dd_ba <- degree_distribution(g_ba)
k_ba <- which(dd_ba > 0) - 1
pk_ba <- dd_ba[dd_ba > 0]

plot(k_ba, pk_ba,
     log = "xy", pch = 19, col = "tomato", cex = 1.5,
     main = "Log-log - Red BA",
     xlab = "log(k)", ylab = "log(P(k))")

```

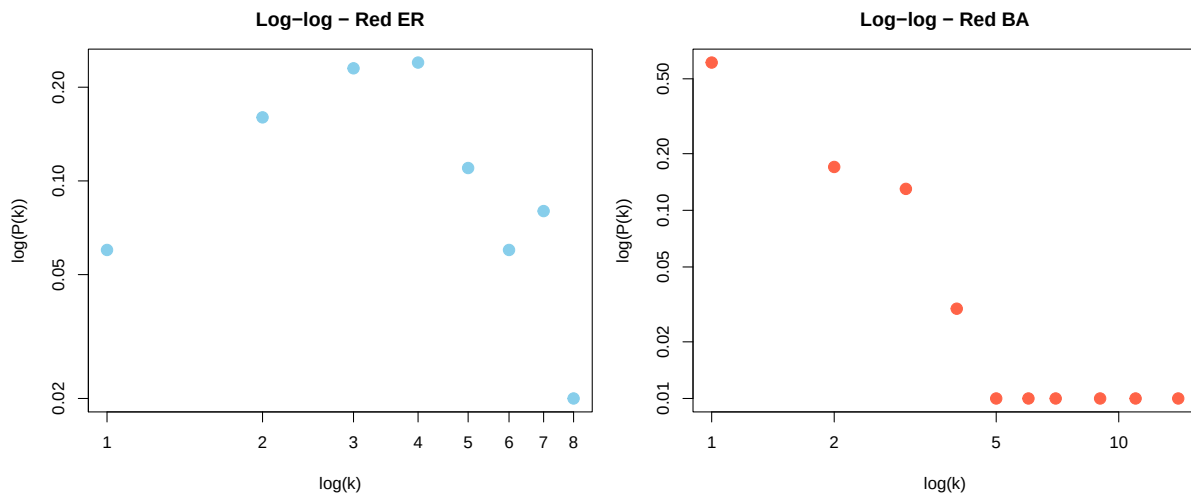


Figure 2.4: Distribución de grado en escala log-log: solo la BA muestra un patrón lineal



Error frecuente

No confundas “parece una recta en log-log” con “es una ley de potencia”. Para confirmar se necesita un **ajuste estadístico formal**, que veremos en la siguiente sección. Muchas distribuciones con cola pesada (log-normal, exponencial estirada) pueden *parecer* lineales en log-log con pocos datos.

2.5 Ajuste de ley de potencia con powerLaw

El paquete `powerLaw` implementa el método riguroso de Clauset, Shalizi y Newman para ajustar distribuciones *power-law* y estimar sus parámetros.

El proceso tiene tres pasos:

2.5.1 Paso 1: Crear el modelo

```
# Obtener los grados de la red BA
deg_ba <- degree(g_ba)

# Crear un objeto de distribución discreta power-law
m <- displ$new(deg_ba)

cat("Modelo creado con", length(deg_ba), "observaciones\n")
```

Modelo creado con 100 observaciones

2.5.2 Paso 2: Estimar parámetros

```
# Estimar el valor mínimo de k a partir del cual aplica la ley de potencia
est <- estimate_xmin(m)
m$setXmin(est)

# Estimar el exponente gamma
m$setPars(estimate_pars(m))

cat("x_min estimado:", m$getXmin(), "\n")
```

x_min estimado: 1

```
cat("Exponente gamma:", round(m$getPars(), 3), "\n")
```

Exponente gamma: 2.196



Concepto clave

El parámetro `x_min` indica el grado mínimo a partir del cual la distribución se comporta como ley de potencia. Los nodos con grado menor a `x_min` no siguen este patrón. El exponente γ típicamente está entre 2 y 3 en redes reales.

2.5.3 Paso 3: Visualizar el ajuste

```
plot(m,
     pch = 19, col = "gray40",
     xlab = "Grado (k)", ylab = "P(X ≥ k)",
     main = "Ajuste Power-law - Red BA")
lines(m, col = "red", lwd = 2)
legend("bottomleft",
     legend = paste0("α = ", round(m$getPars(), 2),
                     ", x_min = ", m$getXmin()),
     col = "red", lwd = 2, bty = "n")
```

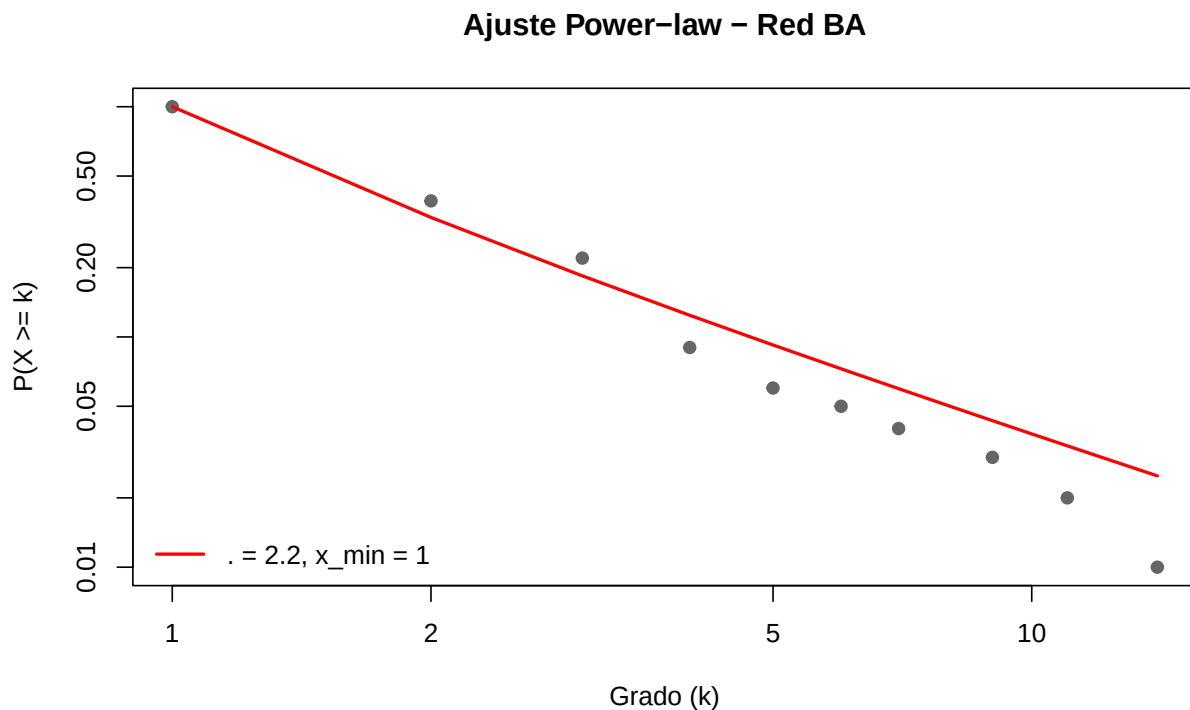


Figure 2.5: Ajuste de ley de potencia sobre la distribución de grado de la red BA

! Recuerda

El gráfico del ajuste muestra la **función de distribución acumulada complementaria** (CCDF), es decir, $P(X \geq k)$, no el histograma. La CCDF es más estable y confiable para evaluar colas pesadas que el histograma, especialmente con pocos datos.

2.6 Identificar *hubs*

Los *hubs* son los nodos con grado extremadamente alto en una red *power-law*. Actúan como conectores centrales y su eliminación puede fragmentar la red (esto lo veremos a fondo en el Chapter 5).

```
# Top 5 nodos más conectados en la red BA
top_hubs <- sort(degree(g_ba), decreasing = TRUE)[1:5]

# Mostrar como tabla
knitr::kable(
  data.frame(
    Nodo = names(top_hubs),
    Grado = as.integer(top_hubs)
  ),
  caption = "Los 5 hubs de la red Barabási-Albert",
  col.names = c("Nodo", "Grado")
)
```

Table 2.2: Los 5 hubs de la red Barabási–Albert

Nodo	Grado
BA_2	14
BA_1	11
BA_9	9
BA_13	7
BA_4	6

```
# Colores: rojo para hubs, gris para el resto
es_hub <- degree(g_ba) >= min(top_hubs)
V(g_ba)$color <- ifelse(es_hub, "tomato", "gray80")
V(g_ba)$size <- ifelse(es_hub, 15, 3)

plot(g_ba,
  vertex.label = ifelse(es_hub, V(g_ba)$name, NA),
  vertex.label.cex = 0.7,
  edge.color = adjustcolor("gray60", alpha = 0.3),
  main = "Red BA - Hubs destacados")
```

Red BA – Hubs destacados

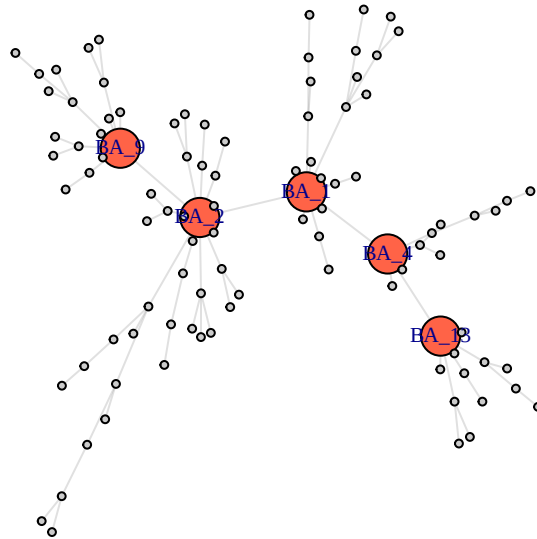


Figure 2.6: Red BA con los 5 hubs destacados en rojo y mayor tamaño

2.7 Comparación completa: ER vs. BA vs. *Small-World*

Agreguemos un tercer modelo: la red *small-world* de Watts y Strogatz (Watts & Strogatz, 1998), que combina alto clustering con distancias cortas.

```
set.seed(42)

redes <- list(
  "Erdős-Rényi"      = sample_gnp(100, p = 0.04),
  "Barabási-Albert"  = sample_pa(100, directed = FALSE),
  "Small-World (WS)" = sample_smallworld(1, 100, nei = 3, p = 0.1)
)
```

```
colores <- c("skyblue", "tomato", "mediumpurple")

par(mfrow = c(1, 3), mar = c(4, 4, 3, 1))

for (i in seq_along(redes)) {
  deg <- degree(redes[[i]])
```



```

hist(deg,
     breaks = seq(-0.5, max(deg) + 0.5, by = 1),
     col = colores[i], border = "white",
     main = names(redes)[i],
     xlab = "Grado (k)", ylab = "Frecuencia")
abline(v = mean(deg), col = "black", lwd = 2, lty = 2)
}

```

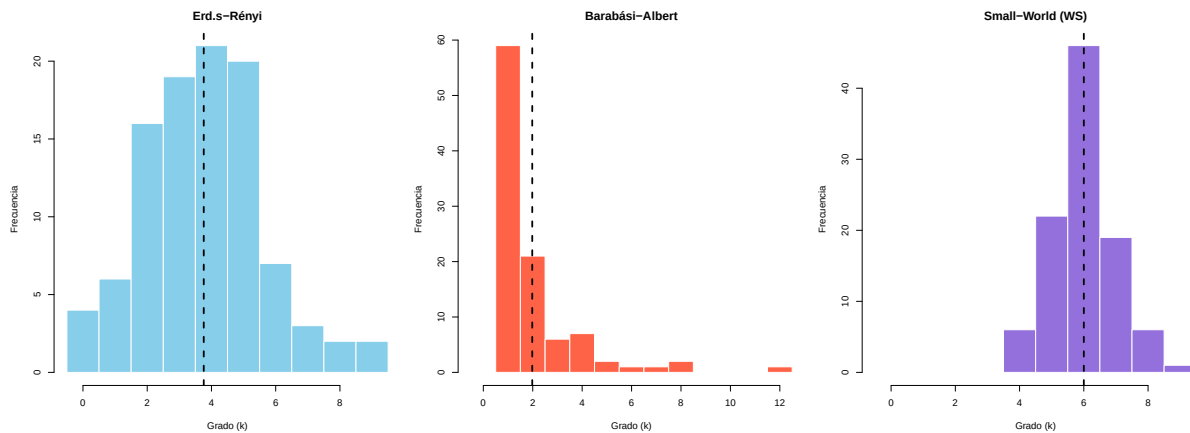


Figure 2.7: Distribuciones de grado de tres modelos de red con 100 nodos

```

# Resumen de métricas
resumen <- data.frame(
  Modelo = names(redes),
  Nodos = sapply(redes, vcount),
  Aristas = sapply(redes, ecount),
  Grado_medio = round(sapply(redes, function(g) mean(degree(g))), 2),
  Grado_max = sapply(redes, function(g) max(degree(g))),
  Grado_min = sapply(redes, function(g) min(degree(g))),
  Densidad = round(sapply(redes, edge_density), 4)
)

knitr::kable(resumen,
              caption = "Comparación de métricas básicas entre tres modelos de red",
              col.names = c("Modelo", "Nodos", "Aristas", "Grado medio",
                           "Grado máx.", "Grado mín.", "Densidad"))

```

Table 2.3: Comparación de métricas básicas entre tres modelos de red

	Modelo	No- dos	Aris- tas	Grado medio	Grado máx.	Grado mín.	Densi- dad
Erdős–Rényi	Erdős–Rényi	100	188	3.76	9	0	0.0380
Barabási– Albert	Barabási– Albert	100	99	1.98	12	1	0.0200
Small-World (WS)	Small-World (WS)	100	300	6.00	9	4	0.0606



Error frecuente

No compares distribuciones de grado entre redes con diferente número de aristas sin tener en cuenta la **densidad**. Una red con el doble de aristas naturalmente tendrá grados más altos, sin que eso indique un mecanismo diferente.

2.8 Ejercicios

Ejercicios resueltos

Ejercicio resuelto 1: Genera una red ER con 500 nodos y $p = 0.01$. Calcula el grado medio teórico y compáralo con el empírico. Visualiza la distribución de grado superpuesta con la distribución de Poisson teórica.



Solución

Paso 1: Generar la red y calcular los grados.

```
set.seed(99)

n <- 500
p <- 0.01

g_er500 <- sample_gnp(n, p)
deg <- degree(g_er500)

# Grado medio teórico vs empírico
grado_teorico <- p * (n - 1)
grado_empirico <- mean(deg)

cat("Grado medio teórico: ", round(grado_teorico, 2), "\n")
```

Grado medio teórico: 4.99

```
cat("Grado medio empírico:", round(grado_empirico, 2), "\n")
```

Grado medio empírico: 5.03

Paso 2: Superponer la distribución empírica con la Poisson teórica.

```

# Histograma normalizado (densidad)
hist(deg,
      breaks = seq(-0.5, max(deg) + 0.5, by = 1),
      col = adjustcolor("skyblue", alpha = 0.6), border = "white",
      probability = TRUE,
      main = "Red ER (n=500, p=0.01): Empírico vs. Poisson",
      xlab = "Grado (k)", ylab = "Densidad de probabilidad",
      xlim = c(0, max(deg) + 2))

# Curva de Poisson teórica
k_vals <- 0:max(deg)
lines(k_vals, dpois(k_vals, lambda = grado_teorico),
      col = "navy", lwd = 2.5, type = "b", pch = 16, cex = 0.8)

legend("topright",
      legend = c("Empírico", paste0("Poisson(=", round(grado_teorico, 1), ")")),
      fill = c(adjustcolor("skyblue", 0.6), NA),
      border = c("white", NA),
      col = c(NA, "navy"), lwd = c(NA, 2.5), pch = c(NA, 16),
      bty = "n", merge = TRUE)

```

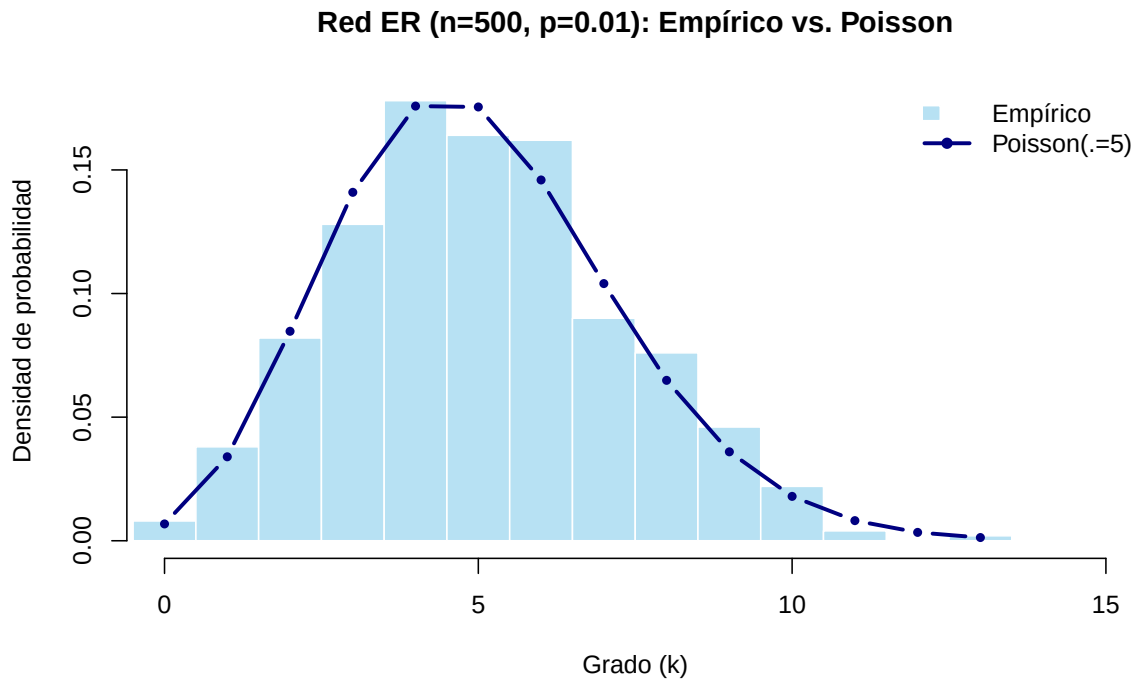


Figure 2.8: Distribución empírica de grado vs. Poisson teórica

Interpretación: El grado medio empírico es muy cercano al teórico $\lambda = p(n-1) \approx 5$. La curva de Poisson se ajusta bien al histograma, confirmando que la distribución de grado en redes ER es efectivamente Poisson.

Ejercicio resuelto 2: Genera una red BA con 1000 nodos y ajusta una ley de potencia con `powerLaw`. Reporta el exponente γ y el valor de $x_{\min}$.

💡 Solución

Paso 1: Generar la red y extraer los grados.

```
set.seed(77)

g_ba1000 <- sample_pa(1000, directed = FALSE)
deg_ba1000 <- degree(g_ba1000)

cat("Nodos:", vcount(g_ba1000), "\n")
```

Nodos: 1000

```
cat("Aristas:", ecount(g_ba1000), "\n")
```

Aristas: 999

```
cat("Grado máximo:", max(deg_ba1000), "\n")
```

Grado máximo: 37

Paso 2: Ajustar la ley de potencia.

```
m_1000 <- displ$new(deg_ba1000)
est_1000 <- estimate_xmin(m_1000)
m_1000$setXmin(est_1000)
m_1000$setPars(estimate_pars(m_1000))

cat("x_min estimado:", m_1000$getXmin(), "\n")
```

x_min estimado: 2

```
cat("Exponente :", round(m_1000$getPars(), 3), "\n")
```

Exponente : 2.641

Paso 3: Visualizar.

```
plot(m_1000, pch = 19, col = "gray40",
      xlab = "Grado (k)", ylab = "P(X = k)",
      main = "Ajuste Power-law - BA (n=1000)")
lines(m_1000, col = "red", lwd = 2)
legend("bottomleft",
      legend = paste0(" = ", round(m_1000$getPars(), 2),
                      ", x_min = ", m_1000$getXmin()),
      col = "red", lwd = 2, bty = "n")
```

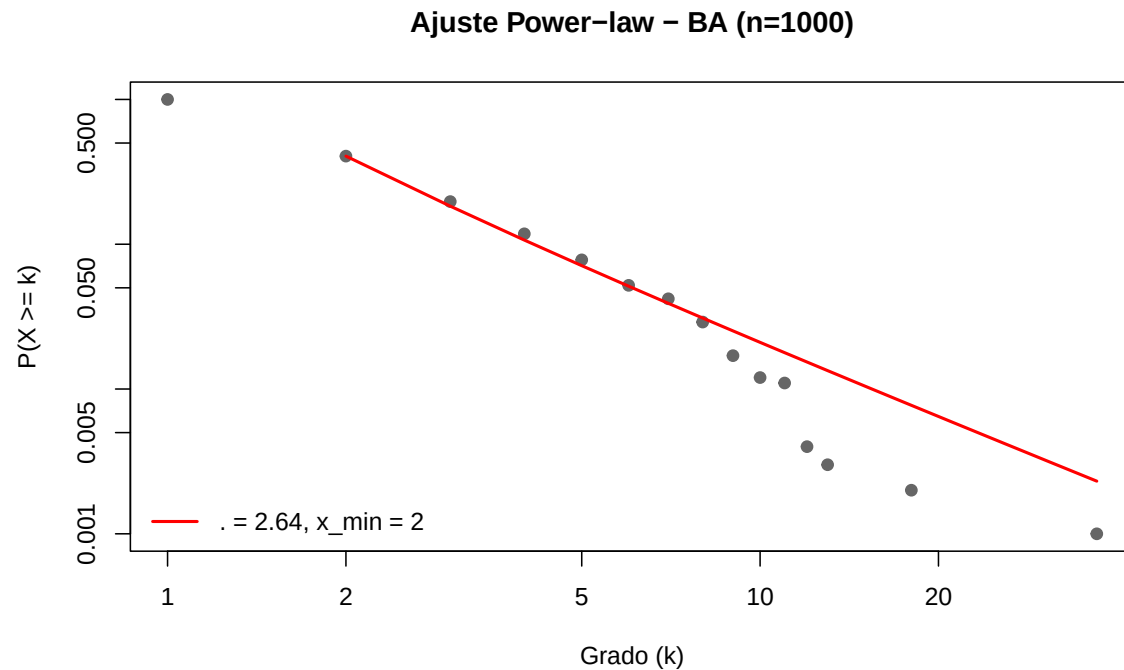


Figure 2.9: Ajuste power-law sobre red BA de 1000 nodos

Interpretación: El exponente γ debería estar cerca de 3 (el valor teórico para redes BA generadas con $m = 1$). El ajuste visual muestra que la cola de la distribución sigue razonablemente bien la recta en escala log-log a partir de $x_{\min}$.

Ejercicios con pistas

Ejercicio 1: Genera redes BA con 100, 500, 1000 y 5000 nodos. ¿Cómo cambia el grado del *hub* más conectado a medida que crece la red? Haz un gráfico de dispersión de n (tamaño de red) vs. grado máximo.

i Pista 1

Usa un `for` o `sapply()` sobre un vector de tamaños: `tamanos <- c(100, 500, 1000, 5000)`. Para cada tamaño, genera la red y extrae `max(degree(g))`.

i Pista 2

Teóricamente, el grado máximo en una red BA crece como $k_{\max} \sim \sqrt{n}$. Puedes superponer esta curva teórica sobre tus datos empíricos.

i Pista 3 — Pseudocódigo

```
1. tamanos <- c(100, 500, 1000, 5000)
2. set.seed(42)
3. grado_max <- sapply(tamanos, function(n) {
4.   g <- sample_pa(n, directed = FALSE)
5.   max(degree(g))
6. })
7. plot(tamanos, grado_max, type = "b", log = "xy")
8. # Superponer curva teórica: sqrt(n)
```

Ejercicio 2: Crea una red ER con 200 nodos y $p = 0.03$. Identifica los nodos con grado mayor al promedio + 2 desviaciones estándar. ¿Cuántos son? Coloréalos de rojo en la visualización.

i Pista 1

Calcula el umbral como `mean(deg) + 2 * sd(deg)`. Luego usa `which(deg > umbral)` para identificar los nodos.

i Pista 2

En una distribución Poisson, aproximadamente el 2.5% de los valores están por encima de $\mu + 2\sigma$. Para 200 nodos, espera alrededor de 5 nodos “extremos”.

i Pista 3 — Pseudocódigo

```
1. set.seed(10)
2. g <- sample_gnp(200, 0.03)
3. deg <- degree(g)
4. umbral <- mean(deg) + 2 * sd(deg)
5. extremos <- which(deg > umbral)
6. V(g)$color <- ifelse(deg > umbral, "red", "gray80")
7. V(g)$size <- ifelse(deg > umbral, 10, 3)
8. plot(g, vertex.label = NA)
```

Ejercicio 3: Compara visualmente la distribución de grado de una red BA y una red *small-world* (Watts-Strogatz) en un **gráfico log-log**. ¿Cuál muestra un patrón más lineal? ¿Por

qué?

i Pista 1

Genera ambas redes con `sample_pa(500)` y `sample_smallworld(1, 500, nei = 3, p = 0.1)`. Calcula `degree_distribution()` para cada una.

i Pista 2

Para graficar en log-log, usa `plot(k, pk, log = "xy")`. Recuerda filtrar los valores donde $P(k) > 0$ para evitar $\log(0)$.

i Pista 3 — Pseudocódigo

```
1. par(mfrow = c(1, 2))
2. Para cada red:
3.   dd <- degree_distribution(g)
4.   k <- which(dd > 0) - 1
5.   pk <- dd[dd > 0]
6.   plot(k, pk, log = "xy", pch = 19)
```

La red BA debería mostrar un patrón más lineal porque su distribución es *power-law*. La *small-world* tiene distribución más acotada (similar a Poisson desplazada).

Ejercicios propuestos

Ejercicio propuesto 1

Genera 5 redes ER de 100 nodos con $p \in \{0.01, 0.05, 0.10, 0.20, 0.50\}$. Para cada una, calcula el grado medio y gráficalo contra p . ¿La relación es lineal?

Autoevaluación: El grado medio teórico es $\langle k \rangle = p(n - 1) = 99p$. Para $p = 0.50$, el grado medio debe ser cercano a 49.5.

Ejercicio propuesto 2

Usa `powerLaw` para ajustar una ley de potencia a una red BA de 2000 nodos. Luego, intenta ajustar la misma distribución a una red ER de 2000 nodos con $p = 0.01$. Compara los gráficos de ajuste. ¿Tiene sentido el ajuste *power-law* para la red ER?

Autoevaluación: El ajuste sobre la red ER debería ser visualmente pobre (la línea roja no sigue los puntos). Si el $x_{\min}$ estimado es muy cercano al grado máximo, es señal de que la *power-law* no es un buen modelo para esos datos.

Ejercicio propuesto 3

Genera una red BA con 500 nodos. Encuentra los 10 *hubs* más conectados. Luego, crea un subgrafo que contenga **solo** esos 10 hubs y las aristas entre ellos (`induced_subgraph()`). ¿Están los *hubs* conectados entre sí?

Autoevaluación: En una red BA, los *hubs* suelen estar conectados entre sí (el hub más antiguo se conecta con los más nuevos). El subgrafo debería ser conexo o tener muy pocos componentes.

Ejercicio propuesto 4

Diseña un experimento numérico: genera 50 redes BA de 500 nodos cada una (con diferentes semillas). Para cada red, ajusta la ley de potencia y guarda el exponente γ . Grafica un histograma de los 50 valores de γ . ¿Alrededor de qué valor se concentran? ¿Cuál es la varianza?

Autoevaluación: Los valores de γ deberían concentrarse alrededor de 3 (el valor teórico para el modelo BA clásico con $m = 1$). La varianza debería ser pequeña pero no nula, debido a la aleatoriedad finita.

Ejercicio propuesto 5

Investiga: ¿Qué pasa con la distribución de grado de una red BA si cambias el parámetro m (número de aristas que agrega cada nodo nuevo) de 1 a 3 a 5? Genera las tres variantes con 1000 nodos, compara sus histogramas y ajusta la *power-law* a cada una. ¿Cambia el exponente γ ?

Autoevaluación: El grado medio aumenta ($\langle k \rangle \approx 2m$), pero el exponente teórico $\gamma = 3$ se mantiene independiente de m en el modelo BA estándar. Empíricamente podrías observar variaciones leves.

2.9 Resumen del capítulo

```

flowchart LR
    A[Modelos de red] --> B[ER → Poisson]
    A --> C[BA → Power-law]
    B --> D[Grado homogéneo]
    C --> E[Hubs emergentes]
    D --> F[Ajuste con dpois]
    E --> G[Ajuste con powerLaw]
    G --> H["Siguiente: Distancias →"]
    style H fill:#e8f4f8,stroke:#3ABEF9

```

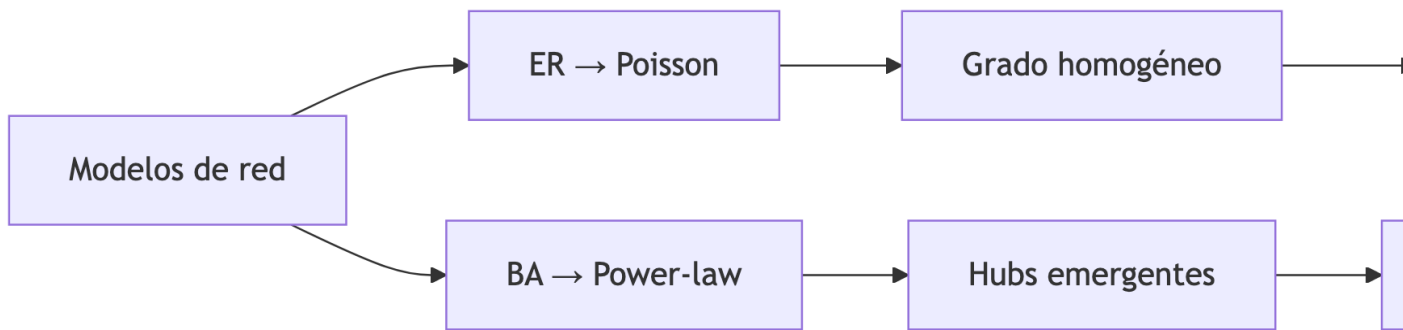


Figure 2.10: Resumen visual del capítulo

Table 2.4: Funciones y conceptos clave del capítulo

Función / Concepto	Descripción
<code>sample_gnp(n, p)</code>	Genera red Erdős–Rényi con n nodos y probabilidad p
<code>sample_pa(n)</code>	Genera red Barabási–Albert con enlace preferencial
<code>degree_distribution()</code>	Calcula la distribución de grado normalizada
<code>displ\$new()</code>	Crea modelo <i>power-law</i> discreto (paquete powerLaw)
<code>estimate_xmin()</code>	Estima el $x_{\min}$ óptimo para el ajuste
<i>Hub</i>	Nodo con grado desproporcionadamente alto
γ (exponente)	Pendiente del ajuste <i>power-law</i> ; típicamente $2 < \gamma < 3$

Conexión con el siguiente capítulo: Ahora que sabes cómo se distribuyen las conexiones

en distintos tipos de red, en el Chapter 3 exploraremos *qué tan lejos* están los nodos entre sí: distancias, caminos más cortos y eficiencia de la red.

Autoevaluación

Pregunta 1

¿Qué modelo de red produce una distribución de grado tipo ley de potencia?

- a) Erdős–Rényi
- b) Barabási–Albert
- c) Lattice regular
- d) Anillo

El modelo Barabási–Albert usa enlace preferencial, lo que genera distribuciones *power-law*.

Pregunta 2

Verdadero o Falso: En una red ER con $n = 100$ y $p = 0.10$, el grado medio esperado es 10.

Verdadero. El grado medio teórico es $\langle k \rangle = p(n - 1) = 0.10 \times 99 \approx 9.9 \approx 10$.

Pregunta 3

¿Qué representa el parámetro $x_{\min}$ en un ajuste de ley de potencia?

Es el valor de grado mínimo a partir del cual la distribución se comporta como *power-law*. Los nodos con grado menor a $x_{\min}$ no siguen este patrón.

Pregunta 4

¿Qué tipo de gráfico es el más adecuado para identificar visualmente una ley de potencia?

- a) Histograma con ejes lineales
- b) Gráfico de barras
- c) Gráfico log-log
- d) Diagrama de caja (*boxplot*)

En escala log-log, una *power-law* aparece como una línea recta con pendiente $-\gamma$.

 Pregunta 5

Verdadero o Falso: Los *hubs* son igual de probables en redes ER que en redes BA.

Falso. En redes ER la probabilidad de encontrar un nodo con grado extremo es exponencialmente baja (distribución Poisson). En redes BA, los *hubs* emergen naturalmente por el mecanismo de enlace preferencial.

3 Distancias y eficiencia

i Objetivos de aprendizaje

Al finalizar este capítulo serás capaz de:

1. **Calcular** la distancia más corta (*shortest path*) entre cualquier par de nodos.
2. **Construir e interpretar** la matriz de distancias y su representación como mapa de calor.
3. **Medir** la distancia media, el diámetro y la excentricidad de una red.
4. **Evaluar** la eficiencia global de una red y su densidad de aristas.
5. **Comparar** distintas topologías en función de sus propiedades de distancia.

Pregunta motivadora: En una red social, ¿cuántos “apretones de manos” separan a dos personas cualesquiera? La famosa idea de los “seis grados de separación” sugiere que el número es sorprendentemente bajo. ¿Es cierto para cualquier tipo de red?

flowchart TD

```
A[Distancias en redes] --> B[Caminos más cortos]
A --> C[Matriz de distancias]
A --> D[Métricas globales]
A --> E[Eficiencia]
B --> B1["shortest_paths()"]
B --> B2[Diámetro]
C --> C1[Heatmap]
D --> D1[Distancia media]
D --> D2[Excentricidad]
D --> D3[Centro y periferia]
E --> E1[Eficiencia global]
E --> E2[Densidad de aristas]
```

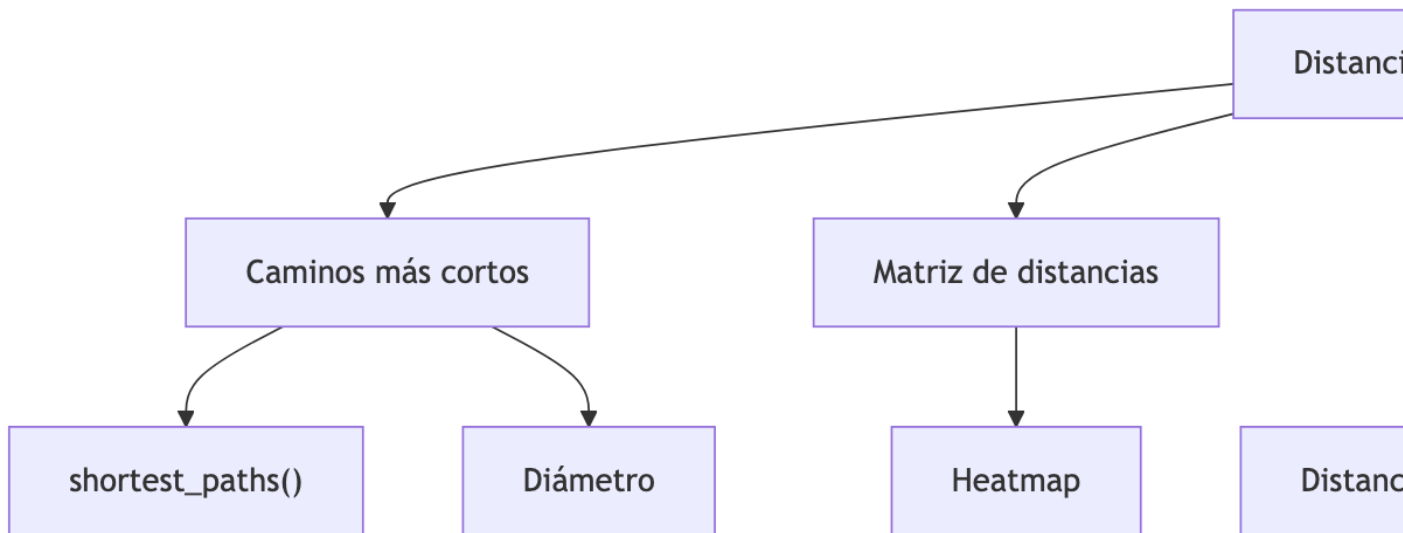


Figure 3.1: Mapa conceptual del capítulo

3.1 Preparación del entorno

```
library(igraph)
```

3.2 ¿Qué es la distancia en una red?

En una red, la **distancia** entre dos nodos u y v es el número mínimo de aristas que hay que recorrer para ir de u a v . Se denota $d(u, v)$ y también se le llama *geodésica* o *shortest path length*.

$$d(u, v) = \min\{\text{número de aristas en un camino de } u \text{ a } v\}$$



Concepto clave

La distancia en redes **no es geográfica**, es **topológica**: mide saltos entre nodos, no kilómetros. Dos personas en diferentes continentes pueden estar a distancia 1 si son amigos directos en una red social.

Comencemos con un ejemplo simple: una red tipo anillo.

```

g <- make_ring(10)
V(g)$name <- paste0("N", 1:10)

plot(g,
     vertex.color = "skyblue",
     vertex.size = 25,
     main = "Red tipo anillo (10 nodos)")

```

Red tipo anillo (10 nodos)

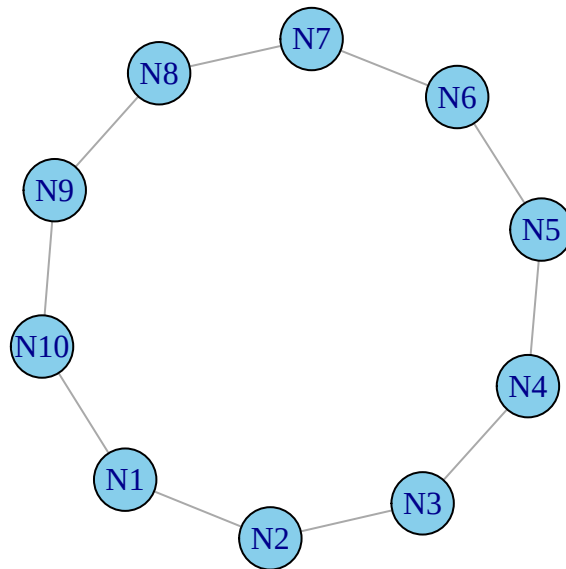


Figure 3.2: Red tipo anillo con 10 nodos: punto de partida para explorar distancias

3.3 Caminos más cortos

3.3.1 Entre dos nodos específicos

```

# ¿Cuál es el camino más corto entre N1 y N6?
camino <- shortest_paths(g, from = "N1", to = "N6")$vpath[[1]]

cat("Camino más corto de N1 a N6:", paste(V(g)$name[camino], collapse = " → "), "\n")

```

Camino más corto de N1 a N6: N1 → N2 → N3 → N4 → N5 → N6


```
cat("Distancia:", length(camino) - 1, "saltos\n")
```

Distancia: 5 saltos

```
# Colorear el camino
V(g)$color <- "skyblue"
V(g)$color[camino] <- "tomato"

# Identificar las aristas del camino
camino_ids <- as.integer(camino)
pares <- c()
for (i in 1:(length(camino_ids) - 1)) {
  pares <- c(pares, camino_ids[i], camino_ids[i + 1])
}
eids <- get.edge.ids(g, pares)

E(g)$color <- "gray70"
E(g)$width <- 1
E(g)$color[eids] <- "red"
E(g)$width[eids] <- 3

plot(g, vertex.size = 25, main = "Camino más corto: N1 → N6")

# Restaurar colores
V(g)$color <- "skyblue"
E(g)$color <- "gray70"
E(g)$width <- 1
```

Camino más corto: N1 → N6

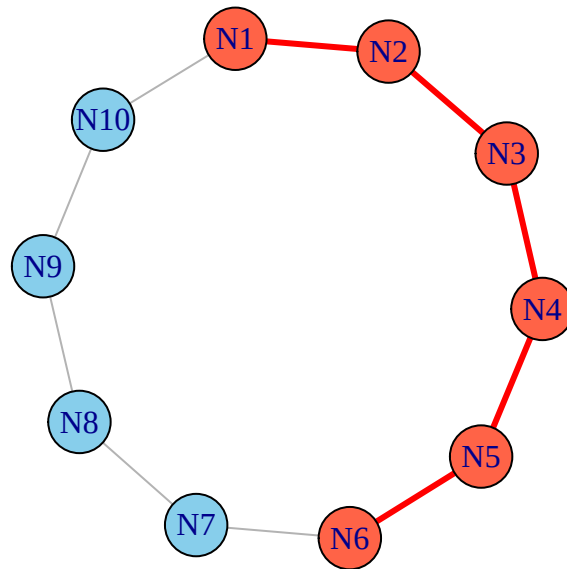


Figure 3.3: Camino más corto entre N1 y N6 destacado en rojo

3.3.2 La matriz de distancias completa

La **matriz de distancias** contiene $d(i, j)$ para todos los pares de nodos. Es una herramienta fundamental para entender la estructura global de la red.

```
dist_mat <- distances(g)

knitr::kable(dist_mat,
              caption = "Matriz de distancias de la red anillo (10 nodos)")
```

Table 3.1: Matriz de distancias de la red anillo (10 nodos)

	N1	N2	N3	N4	N5	N6	N7	N8	N9	N10
N1	0	1	2	3	4	5	4	3	2	1
N2	1	0	1	2	3	4	5	4	3	2
N3	2	1	0	1	2	3	4	5	4	3
N4	3	2	1	0	1	2	3	4	5	4
N5	4	3	2	1	0	1	2	3	4	5
N6	5	4	3	2	1	0	1	2	3	4

	N1	N2	N3	N4	N5	N6	N7	N8	N9	N10
N7	4	5	4	3	2	1	0	1	2	3
N8	3	4	5	4	3	2	1	0	1	2
N9	2	3	4	5	4	3	2	1	0	1
N10	1	2	3	4	5	4	3	2	1	0

⚠ Error frecuente

Si la red tiene **componentes desconectados**, la distancia entre nodos de diferentes componentes es **Inf** (infinito). Siempre verifica con `is_connected(g)` antes de interpretar la matriz de distancias.

```
heatmap(dist_mat,
        Rowv = NA, Colv = NA,
        col = hcl.colors(20, palette = "YlOrRd", rev = TRUE),
        scale = "none",
        main = "Mapa de calor - Distancias en red anillo")
```

Mapa de calor – Distancias en red anillo

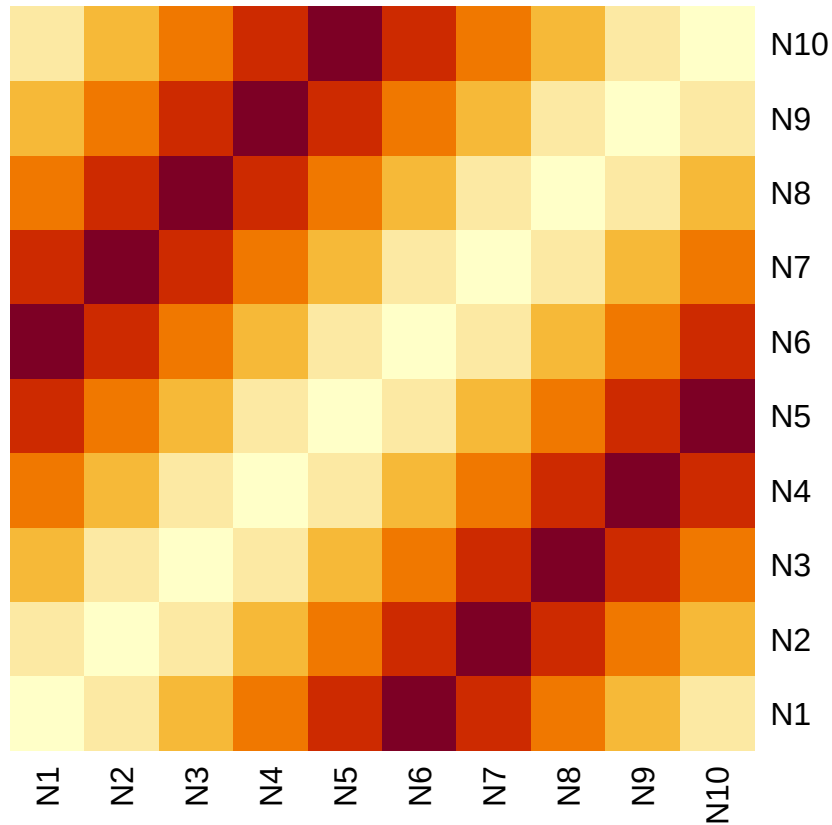


Figure 3.4: Mapa de calor de la matriz de distancias: colores cálidos indican nodos más lejanos

Observa el patrón de bandas diagonales: en una red anillo, la distancia crece simétricamente desde la diagonal (distancia 0) hasta un máximo en la posición diametralmente opuesta.

3.4 Métricas globales de distancia

3.4.1 Distancia media

La **distancia media** $\langle d \rangle$ es el promedio de las distancias más cortas entre todos los pares de nodos. Indica qué tan “compacta” es la red.

$$\langle d \rangle = \frac{1}{n(n-1)} \sum_{i \neq j} d(i, j)$$

```
cat("Distancia media de la red anillo:", mean_distance(g), "\n")
```

Distancia media de la red anillo: 2.777778

3.4.2 Diámetro

El **diámetro** es la distancia más larga entre cualquier par de nodos conectados. Representa el “peor caso” de comunicación en la red.

$$\text{diámetro} = \max_{i,j} d(i,j)$$

```
cat("Diámetro de la red anillo:", diameter(g), "\n")
```

Diámetro de la red anillo: 5

```
# El camino que corresponde al diámetro
camino_largo <- get_diameter(g)
cat("Camino más largo:", paste(V(g)$name[camino_largo], collapse = " → "), "\n")
```

Camino más largo: N1 → N2 → N3 → N4 → N5 → N6

3.4.3 Excentricidad

La **excentricidad** de un nodo v es la distancia máxima de v a cualquier otro nodo. Nos dice qué tan “periférico” es un nodo.

$$\text{ecc}(v) = \max_u d(v,u)$$

A partir de la excentricidad se definen dos conceptos importantes:

- **Centro** de la red: nodo(s) con la **menor** excentricidad.
- **Periferia**: nodo(s) con la **mayor** excentricidad (igual al diámetro).

```
ecc <- eccentricity(g)

# Tabla de excentricidad por nodo
```

```
knitr::kable(
  data.frame(Nodo = V(g)$name, Excentricidad = ecc),
  caption = "Excentricidad de cada nodo en la red anillo"
)
```

Table 3.2: Excentricidad de cada nodo en la red anillo

	Nodo	Excentricidad
N1	N1	5
N2	N2	5
N3	N3	5
N4	N4	5
N5	N5	5
N6	N6	5
N7	N7	5
N8	N8	5
N9	N9	5
N10	N10	5

```
hist(ecc,
  col = "mediumpurple", border = "white",
  main = "Distribución de excentricidad",
  xlab = "Excentricidad", ylab = "Frecuencia",
  breaks = seq(min(ecc) - 0.5, max(ecc) + 0.5, by = 1))
```

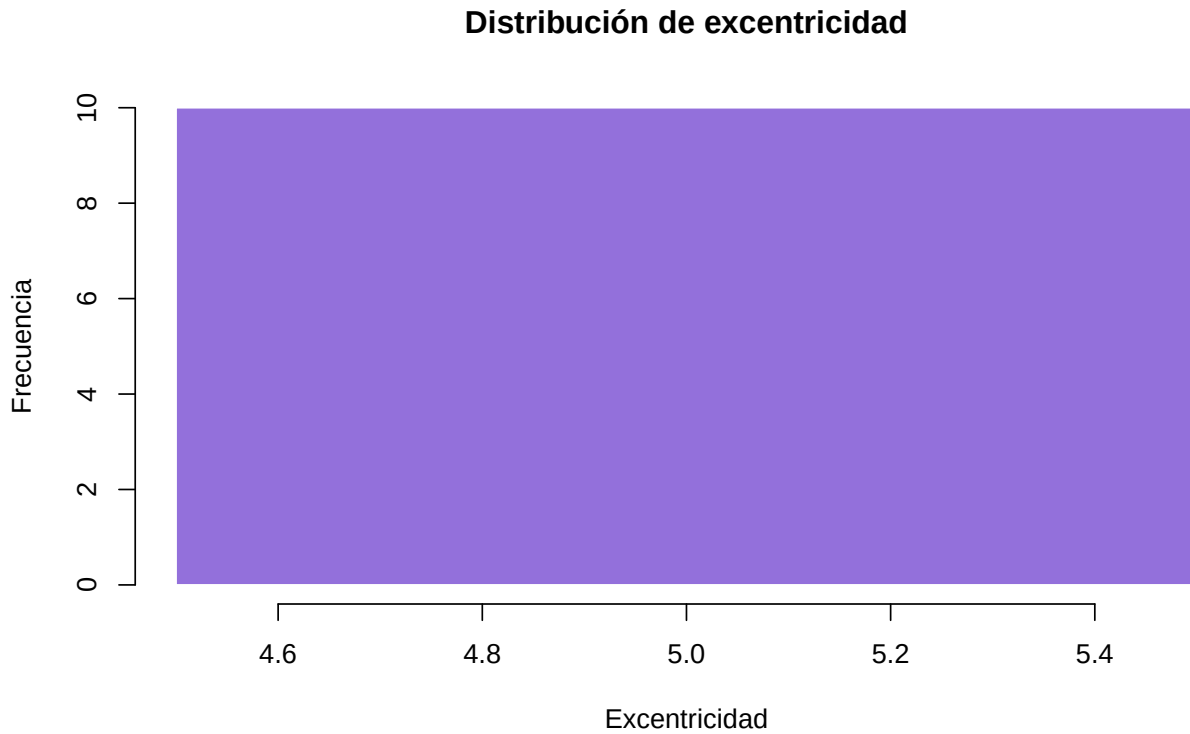


Figure 3.5: Distribución de la excentricidad en la red anillo



Concepto clave

En una red anillo con n nodos, **todos** los nodos tienen la misma excentricidad ($\lfloor n/2 \rfloor$) porque la red es perfectamente simétrica. Esto no es así en redes más complejas.

3.5 Eficiencia de la red

La **eficiencia global** mide qué tan bien se puede transmitir información en la red. Se calcula como el promedio del inverso de las distancias:

$$E_{\text{global}} = \frac{1}{n(n-1)} \sum_{i \neq j} \frac{1}{d(i, j)}$$

Una red completamente conectada tiene eficiencia 1; una red sin aristas tiene eficiencia 0.

```
# Función para calcular eficiencia global
eficiencia_global <- function(g) {
  d <- distances(g)
  n <- vcount(g)
  # Reemplazar Inf por 0 (nodos desconectados no contribuyen)
  inv_d <- ifelse(d == 0 | is.infinite(d), 0, 1 / d)
  sum(inv_d) / (n * (n - 1))
}

cat("Eficiencia global (anillo):", round(eficiencia_global(g), 4), "\n")
```

Eficiencia global (anillo): 0.4852

3.5.1 Densidad de aristas

La **densidad** indica qué fracción de todas las aristas posibles existen realmente:

$$\rho = \frac{2m}{n(n-1)} \quad (\text{red no dirigida})$$

donde m es el número de aristas y n el de nodos.

```
cat("Densidad de aristas (anillo):", round(edge_density(g), 4), "\n")
```

Densidad de aristas (anillo): 0.2222

```
cat("Aristas existentes:", ecount(g), "\n")
```

Aristas existentes: 10

```
cat("Aristas posibles:", choose(vcount(g), 2), "\n")
```

Aristas posibles: 45

3.6 El efecto de los atajos

¿Qué pasa si agregamos una sola arista “atajo” entre dos nodos distantes de una red anillo? Veamos cómo impacta las métricas.


```

g_original <- make_ring(20)
V(g_original)$name <- paste0("N", 1:20)

# Agregar un atajo entre N1 y N11 (diametralmente opuestos)
g_atajo <- add_edges(g_original, c(1, 11))

# Métricas antes y después
metricas <- data.frame(
  Métrica = c("Distancia media", "Diámetro", "Eficiencia global"),
  Sin_atajo = c(
    round(mean_distance(g_original), 3),
    diameter(g_original),
    round(eficiencia_global(g_original), 4)
  ),
  Con_atajo = c(
    round(mean_distance(g_atajo), 3),
    diameter(g_atajo),
    round(eficiencia_global(g_atajo), 4)
  )
)

knitr::kable(metricas,
              caption = "Efecto de un solo atajo en una red anillo de 20 nodos",
              col.names = c("Métrica", "Sin atajo", "Con atajo"))

```

Table 3.3: Efecto de un solo atajo en una red anillo de 20 nodos

Métrica	Sin atajo	Con atajo
Distancia media	5.263	4.2680
Diámetro	10.000	10.0000
Eficiencia global	0.303	0.3397

Figure 3.6: Impacto de un atajo en la red anillo: antes y después

```

par(mfrow = c(1, 2), mar = c(1, 1, 3, 1))

# Layout fijo para ambos gráficos
lay <- layout_in_circle(g_original)

```

```

plot(g_original, layout = lay,
     vertex.color = "skyblue", vertex.size = 15,
     vertex.label.cex = 0.6,
     main = "Anillo original")

# Destacar el atajo en la segunda red
E(g_atajo)$color <- c(rep("gray70", ecount(g_original)), "red")
E(g_atajo)$width <- c(rep(1, ecount(g_original)), 3)

plot(g_atajo, layout = lay,
     vertex.color = "skyblue", vertex.size = 15,
     vertex.label.cex = 0.6,
     main = "Anillo + 1 atajo (N1-N11)")

```

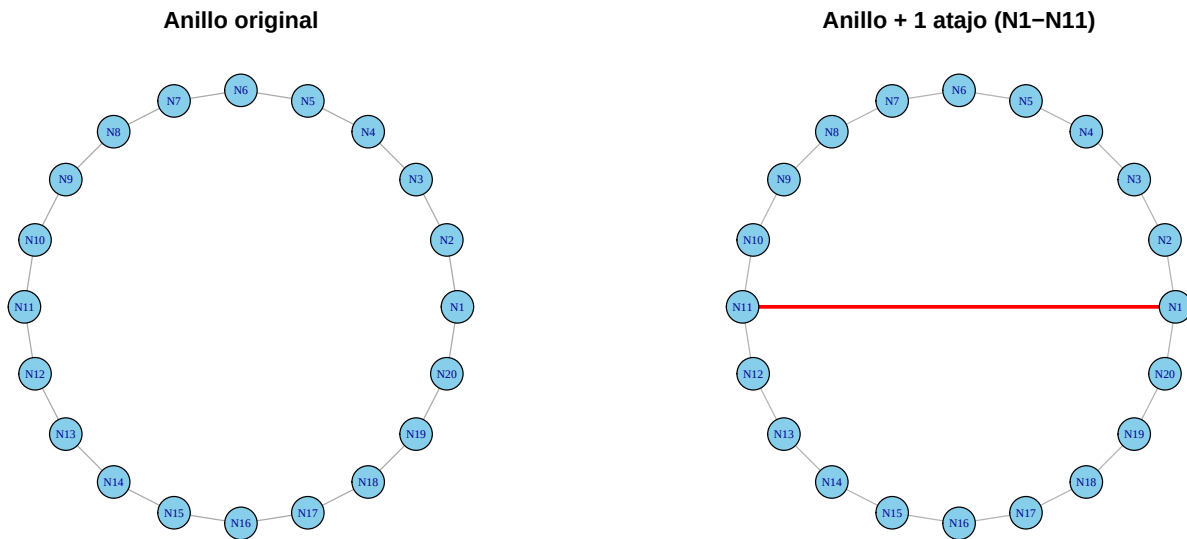


Figure 3.7: Visualización del anillo original (izq.) y con atajo (der.)

! Recuerda

Un solo atajo puede **reducir drásticamente** la distancia media de una red. Este fenómeno es la base del modelo *small-world* de Watts y Strogatz (Watts & Strogatz, 1998): unas pocas conexiones aleatorias transforman una red regular en una donde “todo el mundo está cerca de todo el mundo”.

3.7 Comparación entre topologías

Comparemos las métricas de distancia de cinco topologías clásicas con 50 nodos:

```
set.seed(42)

redes <- list(
  Anillo      = make_ring(50),
  Estrella    = make_star(50, mode = "undirected"),
  Completa    = make_full_graph(50),
  "Small-world" = sample_smallworld(1, 50, nei = 3, p = 0.1),
  "Barabási-Albert" = sample_pa(50, directed = FALSE)
)

# Calcular métricas
comparacion <- data.frame(
  Red = names(redes),
  Nodos = sapply(redes, vcount),
  Aristas = sapply(redes, ecoun),
  Dist_media = round(sapply(redes, mean_distance), 3),
  Diámetro = sapply(redes, diameter),
  Densidad = round(sapply(redes, edge_density), 4),
  Eficiencia = round(sapply(redes, eficiencia_global), 4)
)

knitr::kable(comparacion,
              caption = "Comparación de métricas de distancia entre cinco topologías (50 nodos)",
              col.names = c("Red", "Nodos", "Aristas", "Dist. media",
                           "Diámetro", "Densidad", "Eficiencia"))
```

Table 3.4: Comparación de métricas de distancia entre cinco topologías (50 nodos)

	Red	No- dos	Aristas	Dist. media	Diámetro	Densi- dad	Eficiencia
Anillo	Anillo	50	50	12.755	25	0.0408	0.1549
Estrella	Estrella	50	49	1.960	2	0.0400	0.5200
Completa	Completa	50	1225	1.000	1	1.0000	1.0000
Small-world	Small-world	50	150	2.673	5	0.1224	0.4462
Barabási-Albert	Barabási-Albert	50	49	4.358	9	0.0400	0.2851

```

par(mfrow = c(1, 2), mar = c(8, 4, 3, 1))

# Distancia media
barplot(comparacion$Dist_media,
        names.arg = comparacion$Red,
        col = c("skyblue", "gold", "lightgreen", "mediumpurple", "tomato"),
        main = "Distancia media",
        ylab = "Distancia media",
        las = 2, cex.names = 0.8)

# Eficiencia
barplot(comparacion$Eficiencia,
        names.arg = comparacion$Red,
        col = c("skyblue", "gold", "lightgreen", "mediumpurple", "tomato"),
        main = "Eficiencia global",
        ylab = "Eficiencia",
        las = 2, cex.names = 0.8)

```

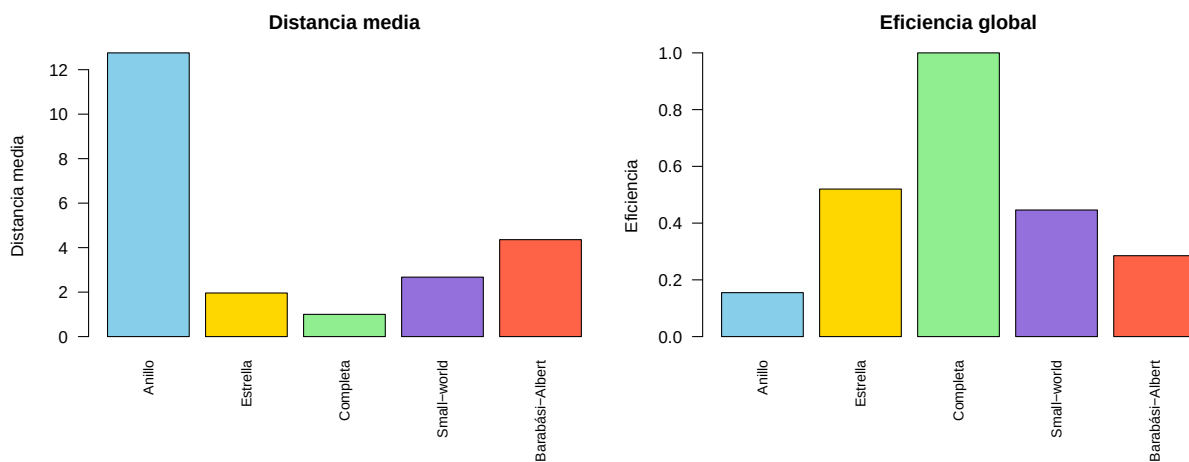


Figure 3.8: Distancia media y eficiencia global por tipo de red



Concepto clave

La red **completa** tiene la eficiencia máxima (todos están a distancia 1), pero requiere $\binom{n}{2}$ aristas. La red *small-world* logra una eficiencia alta con muchas menos aristas, lo que la convierte en un diseño “económico” para la propagación rápida de información.

3.8 Distancias en redes grandes

Veamos cómo se comportan las distancias en una red *small-world* de 500 nodos, similar a las que aparecen en sistemas reales.

```
set.seed(42)
g_large <- sample_smallworld(1, 500, nei = 3, p = 0.1)

cat("Nodos:", vcount(g_large), "\n")
```

Nodos: 500

```
cat("Aristas:", ecount(g_large), "\n")
```

Aristas: 1500

```
cat("Distancia media:", round(mean_distance(g_large), 3), "\n")
```

Distancia media: 4.556

```
cat("Diámetro:", diameter(g_large), "\n")
```

Diámetro: 8

```
# Mostrar solo un fragmento (la matriz completa sería 500×500)
dist_large <- distances(g_large)

heatmap(dist_large[1:20, 1:20],
        Rowv = NA, Colv = NA,
        col = hcl.colors(20, palette = "YlOrRd", rev = TRUE),
        scale = "none",
        main = "Distancias (primeros 20 nodos)")
```

Distancias (primeros 20 nodos)

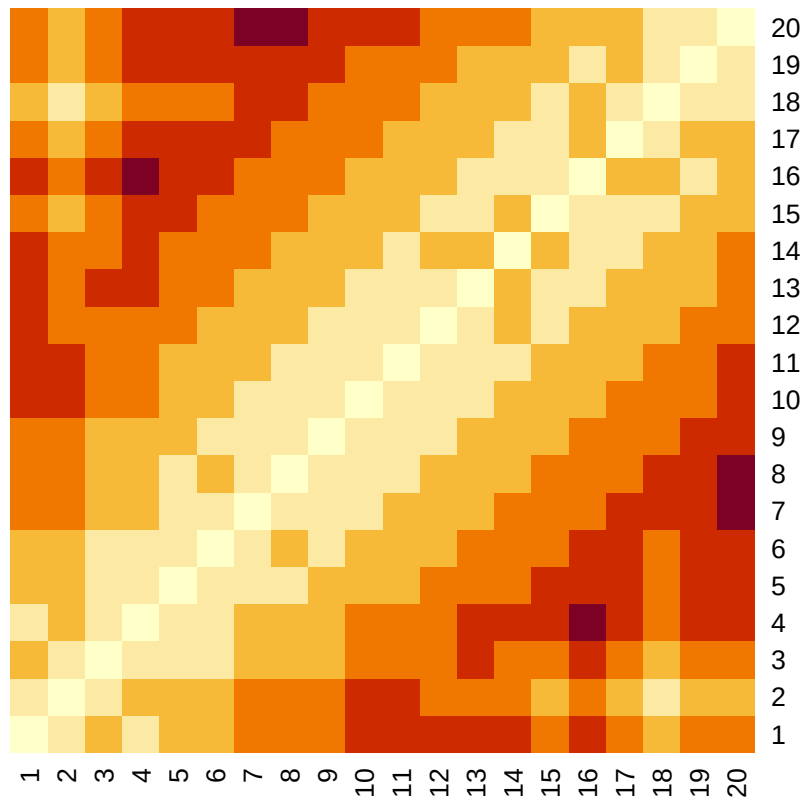


Figure 3.9: Fragmento 20×20 de la matriz de distancias de una red small-world (500 nodos)

```
# Extraer todas las distancias (triángulo superior para no duplicar)
todas_dist <- dist_large[upper.tri(dist_large)]

hist(todas_dist,
     col = "mediumpurple", border = "white",
     main = "Distribución de distancias (n = 500, small-world)",
     xlab = "Distancia (saltos)", ylab = "Frecuencia de pares",
     breaks = seq(0.5, max(todas_dist) + 0.5, by = 1))
abline(v = mean(todas_dist), col = "navy", lwd = 2, lty = 2)
legend("topright", paste("Media =", round(mean(todas_dist), 2)),
     col = "navy", lty = 2, lwd = 2, bty = "n")
```

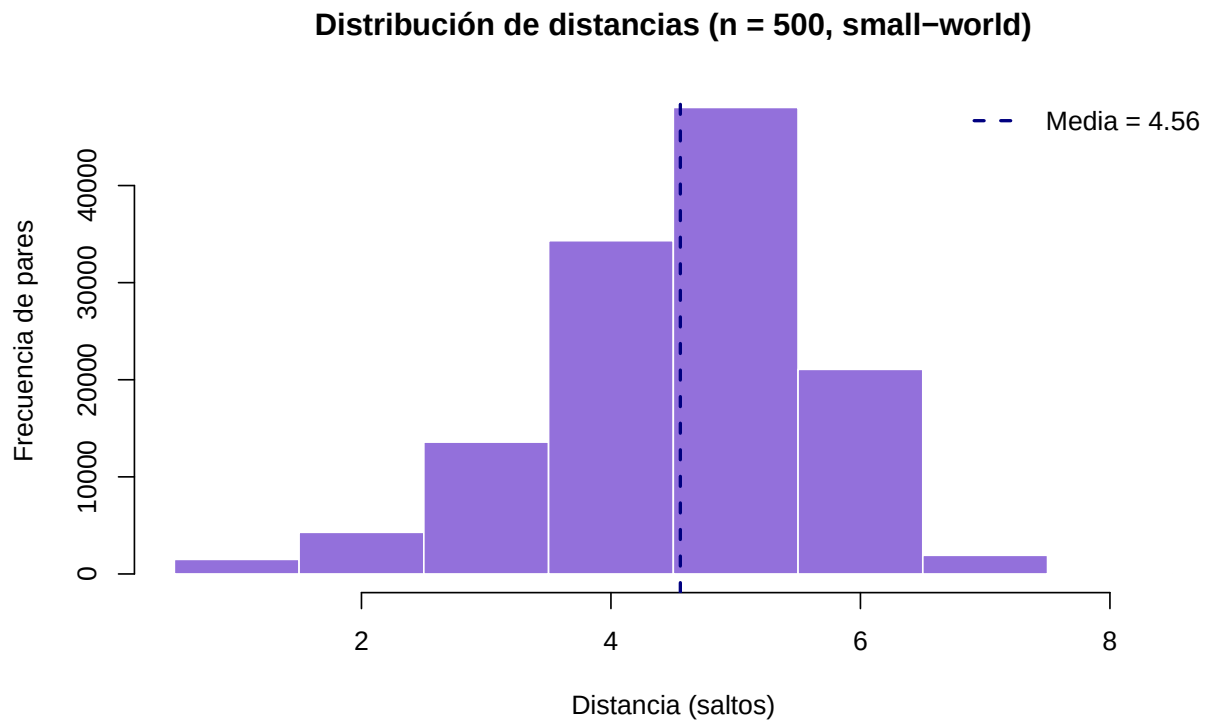


Figure 3.10: Distribución de todas las distancias en la red small-world de 500 nodos



Error frecuente

No intentes visualizar la matriz de distancias completa de una red grande con `heatmap()`. Para $n = 1000$, la matriz tiene un millón de entradas y el gráfico será ilegible. Muestra fragmentos o usa la distribución de distancias como alternativa.

3.9 Ejercicios

Ejercicios resueltos

Ejercicio resuelto 1: Compara la distancia media de tres redes: anillo, estrella y grafo completo, todos con 30 nodos. ¿Cuál sería la mejor para difundir una noticia rápidamente? Visualiza los resultados con un gráfico de barras.

💡 Solución

Paso 1: Crear las tres redes y calcular sus métricas.

```
redes_30 <- list(  
  Anillo = make_ring(30),  
  Estrella = make_star(30, mode = "undirected"),  
  Completa = make_full_graph(30)  
)  
  
metricas_30 <- data.frame(  
  Red = names(redes_30),  
  Dist_media = round(sapply(redes_30, mean_distance), 3),  
  Diámetro = sapply(redes_30, diameter),  
  Aristas = sapply(redes_30, ecount)  
)  
  
knitr::kable(metricas_30,  
  caption = "Métricas de tres topologías con 30 nodos",  
  col.names = c("Red", "Dist. media", "Diámetro", "Aristas"))
```

Table 3.5: Métricas de tres topologías con 30 nodos

	Red	Dist. media	Diámetro	Aristas
Anillo	Anillo	7.759	15	30
Estrella	Estrella	1.933	2	29
Completa	Completa	1.000	1	435

Paso 2: Visualizar con gráfico de barras.

```
barplot(metricas_30$Dist_media,  
        names.arg = metricas_30$Red,  
        col = c("skyblue", "gold", "lightgreen"),  
        main = "Distancia media (n = 30)",  
        ylab = "Distancia media",  
        ylim = c(0, max(metricas_30$Dist_media) * 1.2))  
text(x = c(0.7, 1.9, 3.1), y = metricas_30$Dist_media + 0.3,  
     labels = metricas_30$Dist_media, cex = 0.9, font = 2)
```

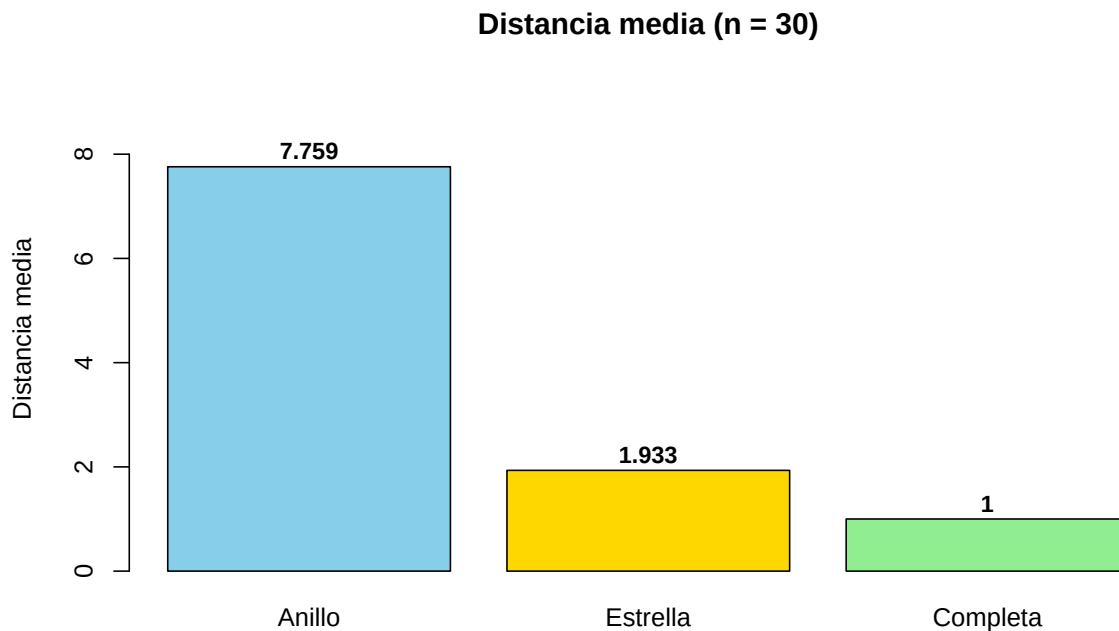


Figure 3.11: Distancia media de tres topologías con 30 nodos

Interpretación: La red **completa** tiene distancia media 1 (todos conectados directamente), pero requiere $\binom{30}{2} = 435$ aristas. La **estrella** logra una distancia media de apenas ~ 2 con solo 29 aristas (muy eficiente). El **anillo** tiene la mayor distancia media (~ 8) porque la información debe recorrer la mitad del círculo. Para difundir una noticia rápido, la completa es óptima pero costosa; la estrella es la más eficiente en aristas por unidad de distancia.

Ejercicio resuelto 2: Crea una función `resumen_distancias()` que reciba un grafo y devuelva

un *data frame* con: distancia media, diámetro, excentricidad mínima (radio), excentricidad máxima y eficiencia global. Pruébala con una red *small-world* de 100 nodos.

💡 Solución

Paso 1: Definir la función.

```
resumen_distancias <- function(g) {  
  ecc <- eccentricity(g)  
  d <- distances(g)  
  n <- vcount(g)  
  
  # Eficiencia global  
  inv_d <- ifelse(d == 0 | is.infinite(d), 0, 1 / d)  
  eficiencia <- sum(inv_d) / (n * (n - 1))  
  
  data.frame(  
    Métrica = c("Distancia media", "Diámetro", "Radio (ecc. mín.)",  
                "Ecc. máxima", "Eficiencia global", "¿Conexa?"),  
    Valor = c(  
      round(mean_distance(g), 3),  
      diameter(g),  
      min(ecc),  
      max(ecc),  
      round(eficiencia, 4),  
      is_connected(g)  
    )  
  )  
}
```

Paso 2: Probar con una red *small-world*.

```
set.seed(55)  
g_sw100 <- sample_smallworld(1, 100, nei = 3, p = 0.1)  
  
knitr::kable(resumen_distancias(g_sw100),  
              caption = "Resumen de distancias - Red small-world (100 nodos)")
```

Table 3.6: Resumen de distancias — Red small-world (100 nodos)

Métrica	Valor
Distancia media	3.2510
Diámetro	6.0000
Radio (ecc. mín.)	4.0000
Ecc. máxima	6.0000
Eficiencia global	0.3589
¿Conexa?	1.0000

Interpretación: La función encapsula todas las métricas de distancia en una sola llamada. El radio indica la excentricidad del nodo más “central” y la eficiencia global resume qué tan fácil es la comunicación en la red en un solo número entre 0 y 1.

Ejercicios con pistas

Ejercicio 1: Crea una red tipo anillo de 20 nodos. Agrega una arista entre dos nodos distantes (por ejemplo, N1 y N11). ¿Cuánto se reduce la distancia media? ¿Y el diámetro? Repite agregando 2, 3 y 5 atajos en posiciones diferentes y grafica la evolución de la distancia media.

i Pista 1

Guarda la distancia media original con `mean_distance(g)` antes de modificar. Usa `add_edges()` para agregar atajos en cada iteración.

i Pista 2

Para los atajos adicionales, elige pares de nodos alejados: (N5, N15), (N3, N18), etc. Almacena los resultados en un vector y gráficlos con `plot(0:5, distancias, type = "b")`.

i Pista 3 — Pseudocódigo

```
1. g <- make_ring(20)
2. atajos <- list(c(1,11), c(5,15), c(3,18), c(8,14), c(2,19))
3. resultados <- mean_distance(g) # sin atajos
4. for (i in 1:5) {
5.     g <- add_edges(g, atajos[[i]])
6.     resultados <- c(resultados, mean_distance(g))
}
```

```
7. }  
8. plot(0:5, resultados, type = "b",  
9.      xlab = "Número de atajos", ylab = "Distancia media")
```

Ejercicio 2: Calcula la excentricidad de cada nodo en una red tipo estrella de 15 nodos. Identifica el centro y la periferia de la red. ¿Por qué la estrella tiene un centro tan obvio?

i Pista 1

Usa `eccentricity(g)` y luego `which.min()` para encontrar el centro. En una estrella, el nodo central tiene excentricidad 1.

i Pista 2

Los nodos periféricos tienen excentricidad 2 (necesitan pasar por el centro para llegar a cualquier otro nodo periférico). El centro es el nodo 1 (el hub).

i Pista 3 — Pseudocódigo

```
1. g <- make_star(15, mode = "undirected")  
2. ecc <- eccentricity(g)  
3. cat("Centro:", which(ecc == min(ecc)))  
4. cat("Periferia:", which(ecc == max(ecc)))  
5. # Colorear: centro rojo, periferia azul  
6. V(g)$color <- ifelse(ecc == min(ecc), "tomato", "skyblue")  
7. plot(g, vertex.size = 20)
```

Ejercicio 3: Crea una red aleatoria ER (`sample_gnp(100, 0.03)`) y una red regular (anillo) del mismo tamaño. Compara sus distribuciones de distancias par-a-par usando histogramas superpuestos. ¿Cuál tiene distancias más “compactas”?

i Pista 1

Extrae las distancias del triángulo superior de la matriz: `d[upper.tri(d)]`. Esto evita contar cada par dos veces.

i Pista 2

Asegúrate de que la red ER sea conexas (`is_connected()`). Si no lo es, regenera con una semilla diferente o usa $p = 0.05$.

i Pista 3 — Pseudocódigo

```
1. set.seed(10)
2. g_er <- sample_gnp(100, 0.05)
3. g_ring <- make_ring(100)
4. d_er <- distances(g_er)[upper.tri(distances(g_er))]
5. d_ring <- distances(g_ring)[upper.tri(distances(g_ring))]
6. hist(d_ring, col = adjustcolor("skyblue", 0.5), ...)
7. hist(d_er, col = adjustcolor("tomato", 0.5), add = TRUE)
8. legend(...)
```

La red ER debería tener distancias más compactas (concentradas en valores bajos) gracias a sus conexiones aleatorias.

Ejercicios propuestos

Ejercicio propuesto 1

Usa `shortest_paths()` para encontrar el camino más corto entre los nodos 1 y 50 en una red BA de 100 nodos. ¿Cuántos saltos son? Visualiza la red destacando el camino encontrado.

Autoevaluación: En una red BA de 100 nodos, el camino más corto entre nodos arbitrarios suele tener entre 2 y 5 saltos gracias a los *hubs*.

Ejercicio propuesto 2

Crea una función `eficiencia_global()` y úsala para comparar la eficiencia de redes `sample_smallworld(1, 100, nei = 3, p)` variando $p \in \{0, 0.01, 0.05, 0.1, 0.3, 0.5, 1\}$. Grafica eficiencia vs. p . ¿En qué rango de p se obtiene el mayor “salto” de eficiencia?

Autoevaluación: La eficiencia debería aumentar rápidamente entre $p = 0.01$ y $p = 0.1$, y luego estabilizarse. Esto refleja la transición al régimen *small-world*.

Ejercicio propuesto 3

Calcula la distancia media y el diámetro de una red BA de 1000 nodos. Encuentra los dos nodos más alejados entre sí (los extremos del diámetro) y extrae el camino completo entre ellos con `get_diameter()`. ¿Pasa el camino por algún *hub*?

Autoevaluación: En una red BA de 1000 nodos, el diámetro suele estar entre 6 y 10. El camino del diámetro generalmente pasa por al menos un *hub* (nodo con grado alto).

Ejercicio propuesto 4

Diseña un experimento: genera 30 redes *small-world* con $n = 200$ y p fijo en 0.05, cada una con diferente semilla. Para cada red, calcula la distancia media y el diámetro. Grafica un *boxplot* de ambas métricas. ¿Cuánta variabilidad hay entre realizaciones del mismo modelo?

Autoevaluación: Las distancias medias deberían ser muy similares (baja varianza), mientras que el diámetro puede mostrar más variabilidad porque depende de los “extremos” de la red.

Ejercicio propuesto 5

Investiga la relación entre densidad y eficiencia: genera redes ER con 100 nodos y p variando de 0.01 a 0.50 (en pasos de 0.01). Para cada una, calcula la densidad y la eficiencia global. Grafica eficiencia vs. densidad. ¿La relación es lineal?

Autoevaluación: La relación debería ser creciente pero **no lineal**: la eficiencia crece rápido al principio (las primeras aristas tienen un impacto enorme) y luego se satura al acercarse a 1. Un gráfico con forma de curva logarítmica o raíz cuadrada es esperable.

3.10 Resumen del capítulo

```
flowchart LR
    A[Distancias] --> B[Caminos más cortos]
    B --> C[Matriz de distancias]
    C --> D[Dist. media y diámetro]
    D --> E[Excentricidad: centro/periferia]
```

```

E --> F[Eficiencia global]
F --> G["Siguiente: Clustering →"]
style G fill:#e8f4f8,stroke:#3ABEF9

```



Figure 3.12: Resumen visual del capítulo

Table 3.7: Funciones y conceptos clave del capítulo

Función / Concepto	Descripción
<code>distances(g)</code>	Matriz de distancias más cortas entre todos los pares
<code>shortest_paths(g, from, to)</code>	Camino más corto entre dos nodos específicos
<code>mean_distance(g)</code>	Distancia media de la red
<code>diameter(g)</code> / <code>get_diameter(g)</code>	Diámetro y camino correspondiente
<code>eccentricity(g)</code>	Excentricidad de cada nodo
<code>edge_density(g)</code>	Densidad de aristas
Eficiencia global	$\frac{1}{n(n-1)} \sum_{i \neq j} 1/d(i, j)$
Centro / Periferia	Nodos con menor / mayor excentricidad

Conexión con el siguiente capítulo: Las distancias nos dicen qué tan lejos están los nodos, pero no nos cuentan toda la historia. En el Chapter 4 exploraremos si los **vecinos de un nodo tienden a ser vecinos entre sí** — un fenómeno clave llamado *clustering* que distingue las redes reales de las puramente aleatorias.

Autoevaluación

Pregunta 1

¿Qué representa el diámetro de una red?

- a) El número total de aristas
- b) La distancia media entre todos los pares
- c) La distancia más larga entre cualquier par de nodos
- d) El grado del nodo más conectado

El diámetro es el “peor caso”: la distancia máxima entre dos nodos en la red.

Pregunta 2

Verdadero o Falso: Si una red tiene componentes desconectados, `mean_distance()` devuelve `Inf`.

Falso (por defecto). En `igraph`, `mean_distance()` ignora las distancias infinitas y calcula el promedio solo sobre pares de nodos conectados. Sin embargo, puedes pasar `unconnected = TRUE` para incluir los `Inf`.

Pregunta 3

¿Cuál de estas redes tiene la menor distancia media para un mismo número de nodos n ?

- a) Anillo
- b) Lattice 2D
- c) Grafo completo
- d) Árbol binario

En el grafo completo todos los nodos están a distancia 1 entre sí, por lo que la distancia media es exactamente 1.

Pregunta 4

¿Qué es la excentricidad de un nodo v ?

Es la distancia máxima de v a cualquier otro nodo de la red: $\text{ecc}(v) = \max_u d(v, u)$.
Cuanto menor la excentricidad, más “central” es el nodo.

 Pregunta 5

Verdadero o Falso: Agregar un solo atajo a una red anillo de 1000 nodos puede reducir su distancia media significativamente.

Verdadero. Este es precisamente el efecto *small-world*: unos pocos atajos reducen drásticamente las distancias en redes regulares (Watts & Strogatz, 1998).

4 Clustering y transitividad

i Objetivos de aprendizaje

Al finalizar este capítulo serás capaz de:

1. **Definir** el coeficiente de clustering local y global, y explicar su significado.
2. **Calcular** la transitividad de una red usando **igraph**.
3. **Identificar** triángulos en una red y su relación con el clustering.
4. **Comparar** el clustering entre distintos modelos de red (ER, BA, *small-world*).
5. **Analizar** la relación entre grado y clustering local en redes reales y simuladas.

Pregunta motivadora: Si tu amigo A conoce a tu amigo B, ¿es probable que A y B también se conozcan entre sí? En la vida real, sí. Este fenómeno — que los amigos de tus amigos tienden a ser tus amigos — es el corazón del *clustering*. Pero, ¿todas las redes se comportan así?

```
flowchart TD
    A[Clustering] --> B[Triángulos]
    A --> C[Coeficiente local]
    A --> D[Coeficiente global]
    A --> E[Comparar topologías]
    B --> B1["¿Se cierran los triángulos?"]
    C --> C1["C(v): por nodo"]
    D --> D1[Transitividad de la red]
    E --> E1[ER vs BA vs Small-world]
    E --> E2[Relación grado-clustering]
```

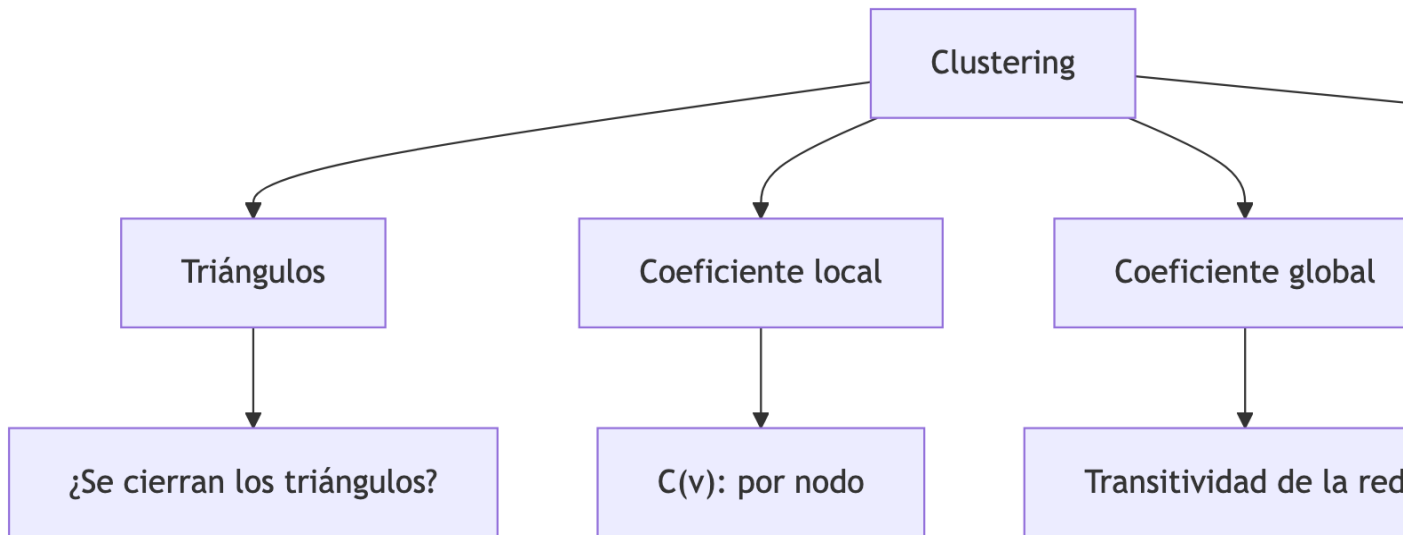


Figure 4.1: Mapa conceptual del capítulo

4.1 Preparación del entorno

```
library(igraph)
```

4.2 ¿Qué es el clustering?

El **coeficiente de clustering** mide la tendencia de los vecinos de un nodo a estar conectados entre sí. Dicho de otra forma: si v tiene vecinos a y b , ¿existe una arista entre a y b ?

Cuando esa arista existe, se forma un **triángulo** (v - a - b), que es la unidad básica del clustering.

4.2.1 Ejemplo visual: triángulos

```
par(mfrow = c(1, 2), mar = c(1, 1, 3, 1))

# Triángulo cerrado
g_cerrado <- make_full_graph(3)
V(g_cerrado)$name <- c("A", "B", "C")
plot(g_cerrado,
```

```

vertex.color = "lightgreen", vertex.size = 35,
edge.width = 3,
main = "Triángulo cerrado\n(clustering = 1)")

# Triángulo abierto (le falta una arista)
g_abierto <- graph_from_literal(A -- B, B -- C)
plot(g_abierto,
     vertex.color = "salmon", vertex.size = 35,
     edge.width = 3,
     main = "Triángulo abierto\n(clustering de B = 0)")

```

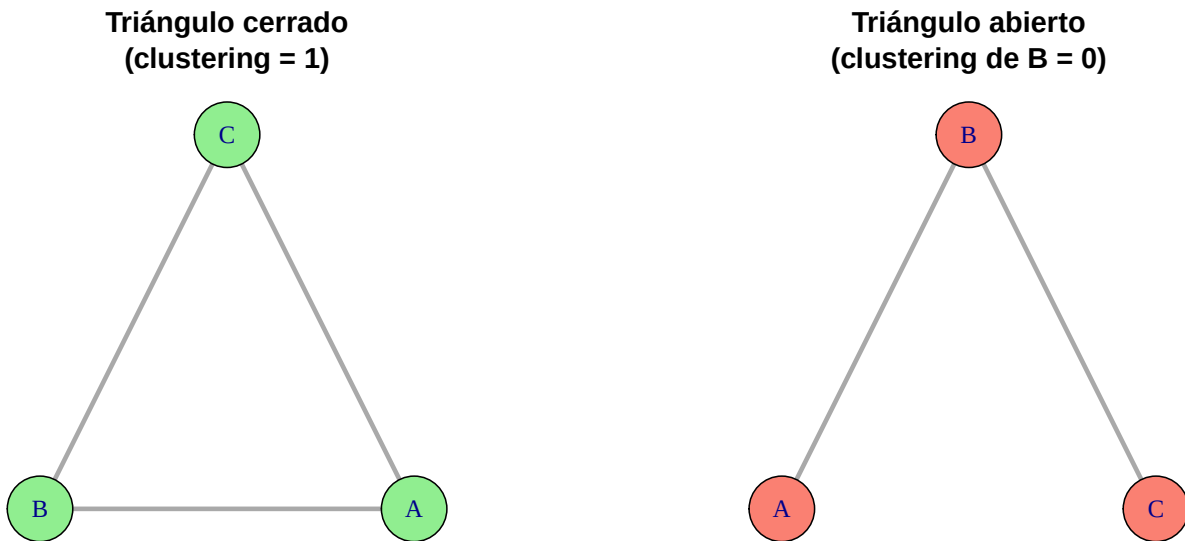


Figure 4.2: Un triángulo completo (izq.) vs. un triángulo abierto (der.)



Concepto clave

Un **triángulo cerrado** se forma cuando tres nodos están todos conectados entre sí. Un **triángulo abierto** (o *tripleta*) ocurre cuando un nodo conecta a otros dos que no están conectados entre sí. El clustering mide la proporción de triángulos abiertos que se “cierran”.

4.3 Coeficiente de clustering local

El **clustering local** de un nodo v , denotado $C(v)$, mide qué fracción de los pares de vecinos de v están realmente conectados:

$$C(v) = \frac{\text{número de aristas entre vecinos de } v}{\binom{k_v}{2}} = \frac{2 \cdot |\{e_{ij} : i, j \in N(v)\}|}{k_v(k_v - 1)}$$

donde k_v es el grado de v y $N(v)$ es el conjunto de vecinos de v .

- $C(v) = 1$: todos los vecinos de v están conectados entre sí (clique local).
- $C(v) = 0$: ningún par de vecinos de v está conectado.
- Si $k_v < 2$, el clustering no está definido (necesita al menos 2 vecinos para formar un par).

Veamos esto con un ejemplo concreto:

```
# Crear una red con estructura interesante
g_demo <- graph_from_literal(
  A -- B, A -- C, A -- D, A -- E,
  B -- C,
  D -- E,
  E -- F
)

plot(g_demo,
     vertex.color = "skyblue", vertex.size = 30,
     edge.width = 2,
     main = "Red de ejemplo")
```

Red de ejemplo

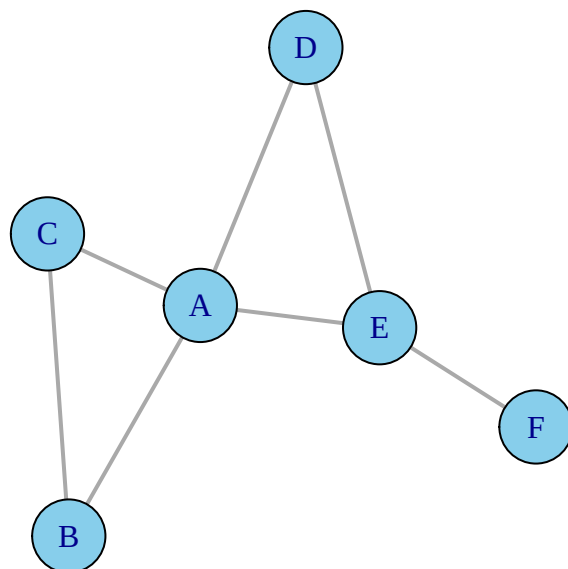


Figure 4.3: Red de ejemplo para calcular clustering local

```
# Clustering local de cada nodo
cl_local <- transitivity(g_demo, type = "local")

knitr::kable(
  data.frame(
    Nodo = V(g_demo)$name,
    Grado = degree(g_demo),
    Clustering = round(cl_local, 3)
  ),
  caption = "Clustering local de cada nodo"
)
```

Table 4.1: Clustering local de cada nodo

	Nodo	Grado	Clustering
A	A	4	0.333
B	B	2	1.000
C	C	2	1.000
D	D	2	1.000
E	E	3	0.333

	Nodo	Grado	Clustering
F	F	1	NaN

4.3.1 ¿Por qué A tiene ese clustering?

El nodo A tiene 4 vecinos: B, C, D, E. Los pares posibles son $\binom{4}{2} = 6$. De esos 6 pares, solo existen 2 aristas entre vecinos (B–C y D–E):

$$C(A) = \frac{2}{6} = 0.333$$

4.3.2 ¿Por qué F tiene NaN?

El nodo F tiene grado 1 (solo está conectado con E). Con un solo vecino no se puede formar ningún par, así que el clustering **no está definido** y `igraph` devuelve NaN.

4.3.3 ¿Por qué B tiene clustering 1?

El nodo B tiene 2 vecinos: A y C. El único par posible (A–C) sí existe. Por lo tanto:

$$C(B) = \frac{1}{1} = 1$$

Todos sus vecinos se conocen entre sí.



Error frecuente

Los nodos con grado 0 o 1 devuelven NaN para el clustering local. Esto no es un error: simplemente no es posible medir clustering sin al menos 2 vecinos. Cuando calcules promedios, asegúrate de excluirlos con `na.rm = TRUE`.

4.4 Coeficiente de clustering global (transitividad)

Existen dos formas de medir el clustering a nivel de toda la red:

4.4.1 Transitividad global

Calcula la fracción de tripletas conectadas que forman triángulos:

$$C_{\text{global}} = \frac{3 \times \text{número de triángulos}}{\text{número de tripletas conectadas}}$$

El factor 3 aparece porque cada triángulo contiene 3 tripletas.

4.4.2 Clustering promedio

Es simplemente el promedio del clustering local de todos los nodos (excluyendo los NaN):

$$\langle C \rangle = \frac{1}{|\{v : k_v \geq 2\}|} \sum_{v: k_v \geq 2} C(v)$$

```
cat("Transitividad global:",  
    round(transitivity(g_demo, type = "global"), 4), "\n")
```

Transitividad global: 0.5

```
cat("Clustering promedio:",  
    round(transitivity(g_demo, type = "average"), 4), "\n")
```

Clustering promedio: 0.7333



Concepto clave

La transitividad **global** pondera más los nodos con alto grado (porque generan más tripletas). El clustering **promedio** da el mismo peso a cada nodo. En redes heterogéneas (como las BA), pueden dar valores muy diferentes.

4.5 Triángulos en la red

igraph permite contar triángulos directamente:


```
# Número total de triángulos en la red
cat("Triángulos totales:", sum(count_triangles(g_demo)) / 3, "\n")
```

Triángulos totales: 2

```
# Triángulos por nodo
knitr::kable(
  data.frame(
    Nodo = V(g_demo)$name,
    Triángulos = count_triangles(g_demo)
  ),
  caption = "Número de triángulos en los que participa cada nodo"
)
```

Table 4.2: Número de triángulos en los que participa cada nodo

Nodo	Triángulos
A	2
B	1
C	1
D	1
E	1
F	0

Error frecuente

`count_triangles()` cuenta cuántos triángulos incluyen a cada nodo. Si sumas todos los valores y divides entre 3, obtienes el número total de triángulos (porque cada triángulo se cuenta 3 veces, una por cada vértice).

4.6 Clustering en distintos modelos de red

¿Cómo varía el clustering entre los diferentes modelos de red que ya conocemos? Esta comparación revela diferencias fundamentales.

```
set.seed(42)

redes <- list(
```

```

Anillo           = make_ring(100),
Estrella         = make_star(100, mode = "undirected"),
Completa         = make_full_graph(50),
"Small-world"    = sample_smallworld(1, 100, nei = 3, p = 0.05),
"Erdős-Rényi"    = sample_gnp(100, p = 0.06),
"Barabási-Albert" = sample_pa(100, directed = FALSE)
)

comparacion <- data.frame(
  Red = names(redes),
  Nodos = sapply(redes, vcount),
  Aristas = sapply(redes, ecount),
  Clustering_global = round(
    sapply(redes, function(g) transitivity(g, type = "global")), 4),
  Clustering_promedio = round(
    sapply(redes, function(g) transitivity(g, type = "average")), 4),
  Dist_media = round(sapply(redes, mean_distance), 3)
)

knitr::kable(comparacion,
  caption = "Clustering y distancia media en seis topologías",
  col.names = c("Red", "Nodos", "Aristas", "C global",
    "C promedio", "Dist. media"))

```

Table 4.3: Clustering y distancia media en seis topologías

	Red	No- dos	Aristas	C global	C promedio	Dist. media
Anillo	Anillo	100	100	0.0000	0.0000	25.253
Estrella	Estrella	100	99	0.0000	0.0000	1.980
Completa	Completa	50	1225	1.0000	1.0000	1.000
Small-world	Small-world	100	300	0.4615	0.4720	3.909
Erdős-Rényi	Erdős-Rényi	100	278	0.0529	0.0598	2.825
Barabási-Albert	Barabási-Albert	100	99	0.0000	0.0000	5.934

```

# Excluir la red completa (distancia 1, clustering 1) para mejor escala
datos_plot <- comparacion[comparacion$Red != "Completa", ]

plot(datos_plot$Dist_media, datos_plot$Clustering_global,

```

```
pch = 19, cex = 2,
col = c("skyblue", "gold", "mediumpurple", "lightcoral", "tomato"),
xlab = "Distancia media", ylab = "Clustering global",
main = "Clustering vs. Distancia media",
xlim = c(0, max(datos_plot$Dist_media) * 1.1),
ylim = c(-0.02, max(datos_plot$Clustering_global) * 1.15))

text(datos_plot$Dist_media, datos_plot$Clustering_global,
      labels = datos_plot$Red, pos = 3, cex = 0.8)
```

Clustering vs. Distancia media

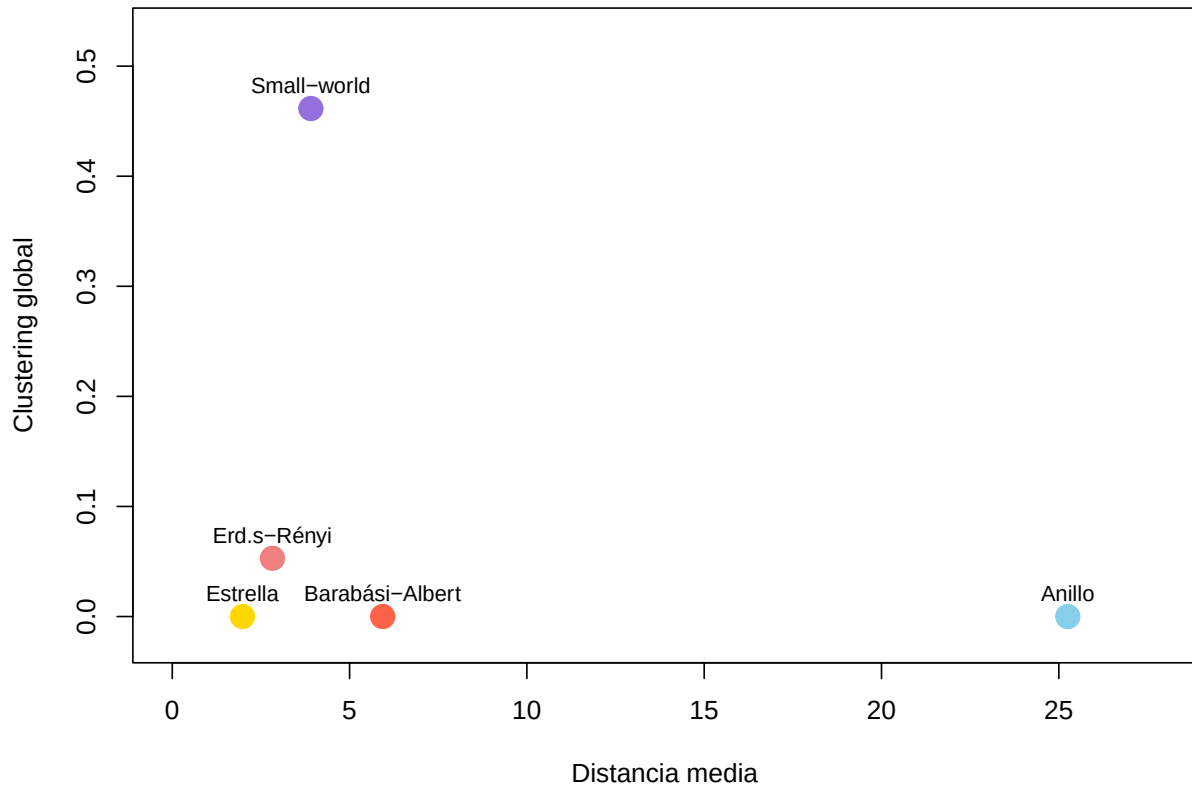


Figure 4.4: Clustering global vs. distancia media para seis topologías

! Recuerda

La red *small-world* ocupa una posición especial: tiene **alto clustering** (como una red regular) y **baja distancia media** (como una red aleatoria). Esta combinación es la firma

del fenómeno *small-world* (Watts & Strogatz, 1998) y es característica de muchas redes reales: redes sociales, redes neuronales, redes de colaboración científica.

4.7 El fenómeno *small-world*: clustering + distancia corta

Exploremos cómo evoluciona el clustering y la distancia media al variar el parámetro de rewiring p en el modelo de Watts-Strogatz:

```
set.seed(42)

# Valores de p en escala logarítmica
p_vals <- c(0, 0.001, 0.003, 0.01, 0.03, 0.05, 0.1, 0.2, 0.3, 0.5, 1.0)

# Calcular métricas para cada p (promediar 5 realizaciones)
resultados <- data.frame(p = p_vals, clustering = NA, dist_media = NA)

for (i in seq_along(p_vals)) {
  cls <- numeric(5)
  dms <- numeric(5)
  for (r in 1:5) {
    g_sw <- sample_smallworld(1, 200, nei = 3, p = p_vals[i])
    cls[r] <- transitivity(g_sw, type = "global")
    dms[r] <- mean_distance(g_sw)
  }
  resultados$clustering[i] <- mean(cls)
  resultados$dist_media[i] <- mean(dms)
}

# Normalizar para graficar en el mismo eje
cl_norm <- resultados$clustering / resultados$clustering[1]
dm_norm <- resultados$dist_media / resultados$dist_media[1]

# Graficar
plot(p_vals, cl_norm,
     type = "b", pch = 19, col = "steelblue", lwd = 2, cex = 1.3,
     log = "x",
     xlab = "Probabilidad de rewiring (p)",
     ylab = "Valor normalizado (respecto a p = 0)",
     main = "Transición Small-World (n = 200, nei = 3)",
```

```

ylim = c(0, 1.05),
xaxt = "n")

# Eje x personalizado
axis(1, at = c(0.001, 0.01, 0.1, 1),
     labels = c("0.001", "0.01", "0.1", "1"))

lines(p_vals, dm_norm,
      type = "b", pch = 17, col = "tomato", lwd = 2, cex = 1.3)

# Zona small-world
rect(0.005, -0.1, 0.15, 1.1,
     col = adjustcolor("gold", alpha = 0.15), border = NA)
text(0.03, 0.1, "Zona\nsmall-world", cex = 0.9, font = 3, col = "darkgoldenrod")

legend("right",
      legend = c("C(p) / C(0)", "L(p) / L(0)"),
      col = c("steelblue", "tomato"),
      pch = c(19, 17), lwd = 2,
      bty = "n", cex = 0.9)

```

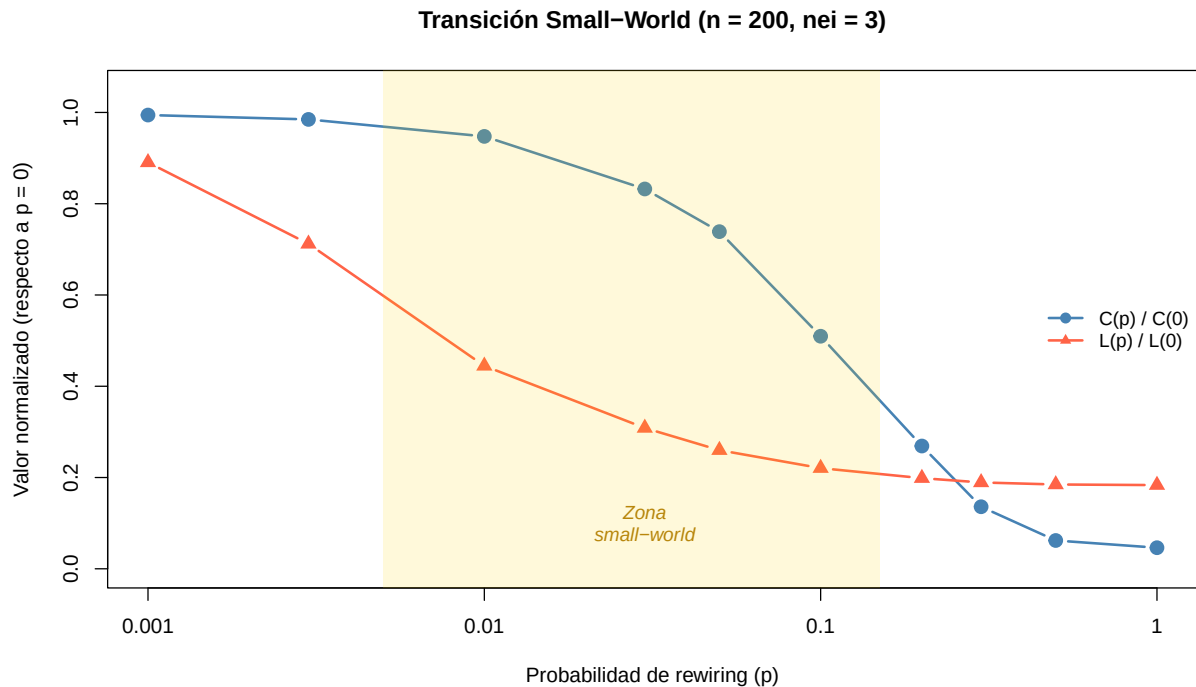


Figure 4.5: Transición small-world: clustering y distancia media vs. probabilidad de rewiring p

💡 Concepto clave

El gráfico muestra la esencia del fenómeno *small-world*: la **distancia media cae rápidamente** con unos pocos atajos (incluso $p = 0.01$), mientras que el **clustering se mantiene alto** durante un rango amplio de p . La “zona *small-world*” es la región donde la red tiene ambas propiedades simultáneamente.

4.8 Relación entre grado y clustering local

En muchas redes reales, los nodos de alto grado (*hubs*) tienden a tener **menor clustering local** que los nodos de grado bajo. Esto ocurre porque es difícil mantener todos los vecinos conectados cuando tienes muchos.

```
set.seed(42)

redes_500 <- list(
  "Erdős-Rényi"      = sample_gnp(500, p = 0.02),
  "Small-world"      = sample_smallworld(1, 500, nei = 3, p = 0.05),
  "Barabási-Albert" = sample_pa(500, directed = FALSE, m = 2)
)

colores <- c("skyblue", "mediumpurple", "tomato")

par(mfrow = c(1, 3), mar = c(4, 4, 3, 1))

for (i in seq_along(redes_500)) {
  g <- redes_500[[i]]
  deg <- degree(g)
  cl <- transitivity(g, type = "local")

  # Filtrar NaN
  validos <- !is.nan(cl)

  plot(deg[validos], cl[validos],
       pch = 19, cex = 0.6,
       col = adjustcolor(colores[i], alpha = 0.5),
       xlab = "Grado", ylab = "Clustering local",
       main = names(redes_500)[i])

  # Línea de tendencia suavizada
```

```

if (sum(validos) > 10) {
  lo <- lowess(deg[validos], cl[validos], f = 0.5)
  lines(lo, col = "black", lwd = 2)
}
}

```

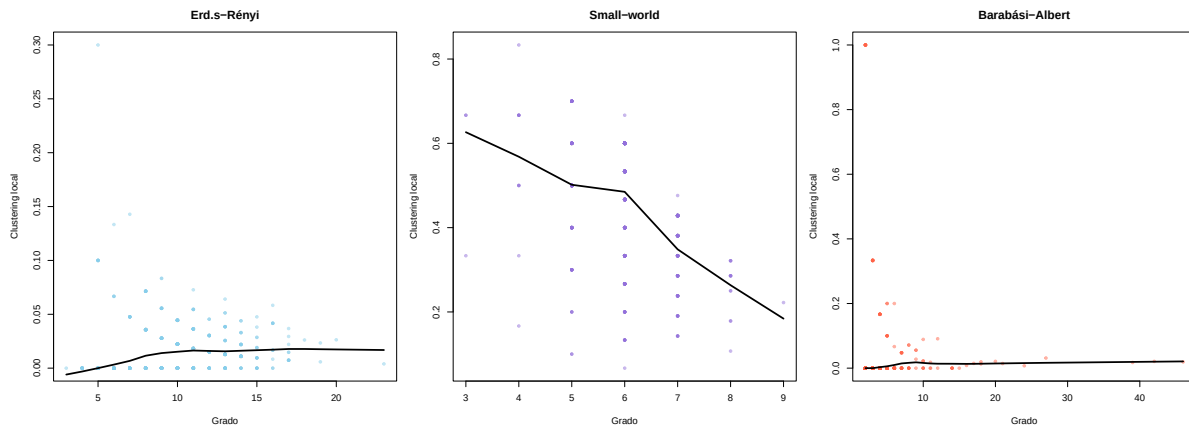


Figure 4.6: Relación entre grado y clustering local en tres modelos de red (500 nodos)



Error frecuente

No interpretes la dispersión en estos gráficos como “ruido”. Es inherente a las redes: para un mismo grado, diferentes nodos pueden tener configuraciones locales muy distintas. La **tendencia** (línea negra) es lo informativo, no los puntos individuales.

4.9 Visualización por clustering

Una forma efectiva de explorar el clustering es **colorear los nodos** según su coeficiente local:

```

set.seed(42)
g_vis <- sample_smallworld(1, 80, nei = 3, p = 0.05)

cl_vis <- transitivity(g_vis, type = "local")
cl_vis[is.nan(cl_vis)] <- 0 # Reemplazar NaN por 0 para la visualización

# Crear paleta de colores: azul (bajo) → rojo (alto)
paleta <- colorRampPalette(c("steelblue", "gold", "tomato"))(100)
colores_nodos <- paleta[cut(cl_vis, breaks = 100, labels = FALSE)]

```

```
plot(g_vis,
     vertex.color = colores_nodos,
     vertex.size = 8,
     vertex.label = NA,
     edge.color = adjustcolor("gray60", alpha = 0.3),
     main = "Clustering local en red Small-World")

# Leyenda manual
legend("bottomright",
     legend = c("C(v) bajo", "C(v) medio", "C(v) alto"),
     fill = c("steelblue", "gold", "tomato"),
     bty = "n", cex = 0.9)
```


Clustering local en red Small-World

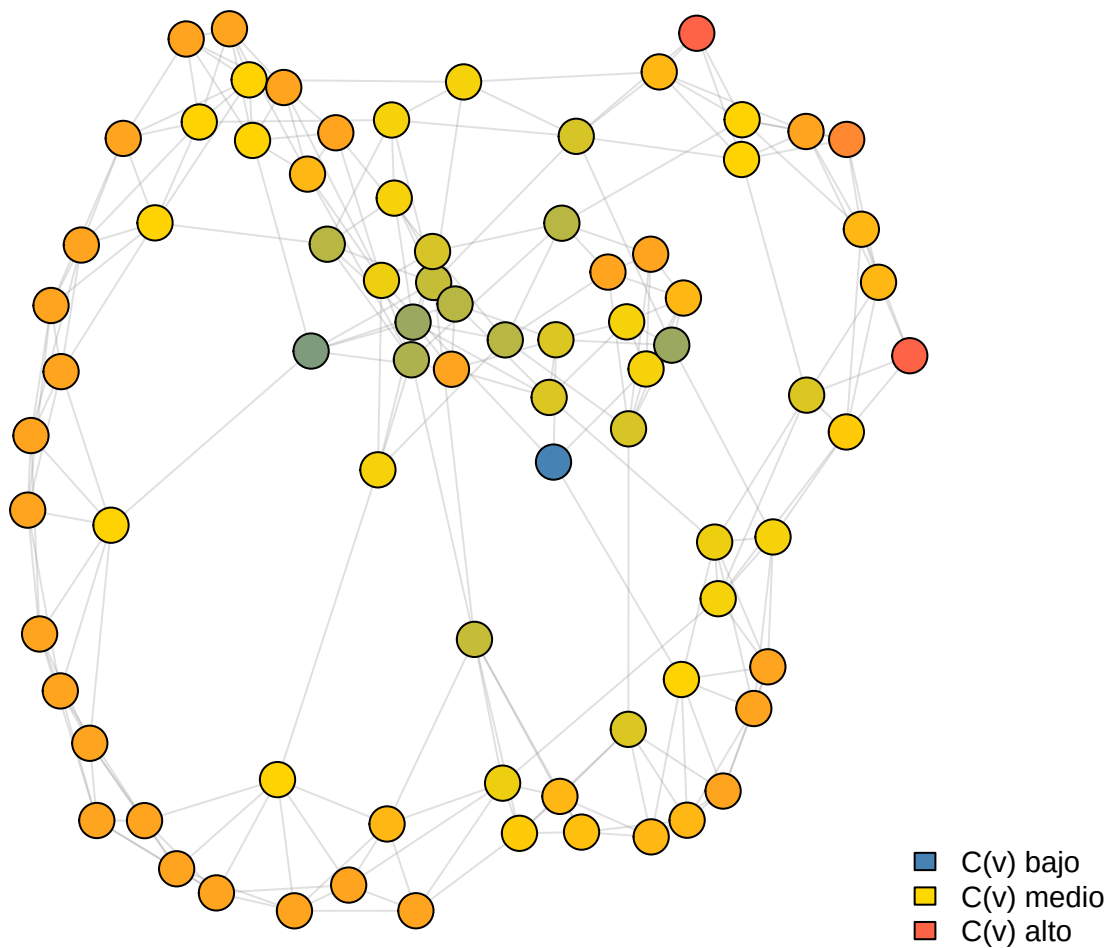


Figure 4.7: Red small-world con nodos coloreados por clustering local (azul = bajo, rojo = alto)

4.10 Ejercicios

Ejercicios resueltos

Ejercicio resuelto 1: Crea una red de 4 nodos completamente conectada (clique). Calcula el clustering local de cada nodo, el clustering global y cuenta los triángulos. Luego elimina una arista y observa cómo cambian las métricas.

💡 Solución

Paso 1: Red completa de 4 nodos.

```
g4 <- make_full_graph(4)
V(g4)$name <- c("A", "B", "C", "D")

cat("=== Red completa (K4) ===\n")
```

=== Red completa (K4) ===

```
cat("Clustering local:", transitivity(g4, type = "local"), "\n")
```

Clustering local: 1 1 1 1

```
cat("Clustering global:", transitivity(g4, type = "global"), "\n")
```

Clustering global: 1

```
cat("Triángulos totales:", sum(count_triangles(g4)) / 3, "\n")
```

Triángulos totales: 4

Paso 2: Eliminar una arista y recalcular.

```
# Eliminar la arista A-D
g4_mod <- delete_edges(g4, E(g4, P = c("A", "D")))

cat("\n=== Tras eliminar A-D ===\n")
```

=== Tras eliminar A-D ===

```
cl_mod <- transitivity(g4_mod, type = "local")
nombres <- V(g4_mod)$name

for (i in seq_along(nombres)) {
  cat("C(", nombres[i], ") =", round(cl_mod[i], 3), "\n")
}
```

```

C( A ) = 1
C( B ) = 0.667
C( C ) = 0.667
C( D ) = 1

```

```

cat("Clustering global:", round(transitivity(g4_mod, type = "global"), 4), "\n")

```

Clustering global: 0.75

```

cat("Triángulos totales:", sum(count_triangles(g4_mod)) / 3, "\n")

```

Triángulos totales: 2

```

par(mfrow = c(1, 2), mar = c(1, 1, 3, 1))
plot(g4, vertex.color = "lightgreen", vertex.size = 35, main = "K4 completa")
plot(g4_mod, vertex.color = "salmon", vertex.size = 35, main = "K4 - arista A-D")

```

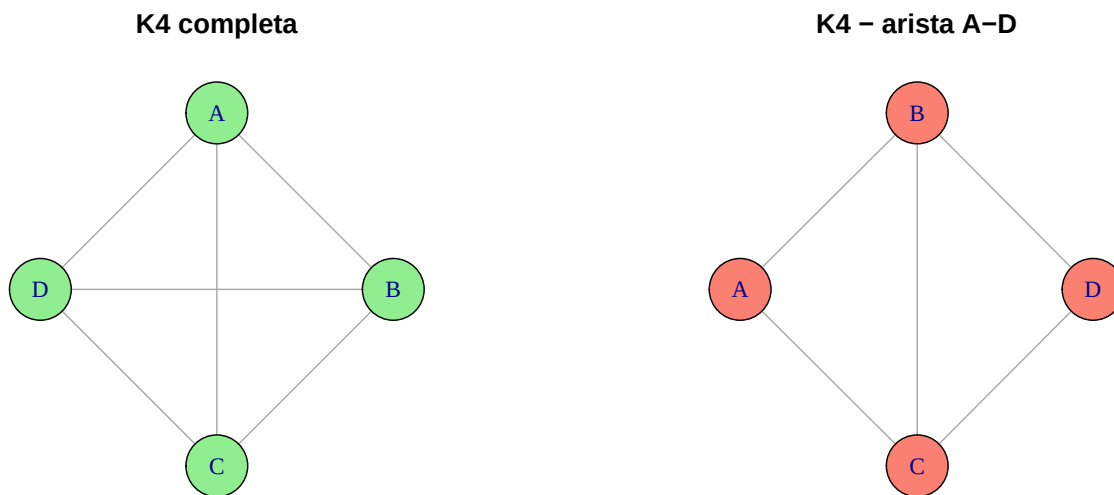


Figure 4.8: Red K_4 original (izq.) vs. red con una arista eliminada (der.)

Interpretación: En la K_4 completa, cada nodo tiene clustering 1 (todos los vecinos conectados) y hay 4 triángulos. Al eliminar una arista (A–D), los nodos A y D bajan su clustering porque pierden un triángulo, mientras que B y C mantienen clustering 1 porque sus vecinos siguen todos conectados entre sí. El clustering global baja de 1.0 a 0.8.

Ejercicio resuelto 2: Simula 100 redes aleatorias ER de 200 nodos con $p = 0.05$. Para cada una, calcula la transitividad global. Grafica la distribución de los 100 valores y compara con el valor teórico (p misma, ya que en redes ER $C \approx p$).

💡 Solución

Paso 1: Simular y recolectar valores de clustering.

```
set.seed(42)

n_sims <- 100
n_nodos <- 200
p <- 0.05

clustering_sims <- numeric(n_sims)
for (i in 1:n_sims) {
  g_sim <- sample_gnp(n_nodos, p)
  clustering_sims[i] <- transitivity(g_sim, type = "global")
}

cat("Clustering teórico ( p):", p, "\n")
```

Clustering teórico (p): 0.05

```
cat("Clustering medio empírico:", round(mean(clustering_sims), 4), "\n")
```

Clustering medio empírico: 0.0501

```
cat("Desviación estándar:", round(sd(clustering_sims), 4), "\n")
```

Desviación estándar: 0.0041

Paso 2: Visualizar la distribución.

```

hist(clustering_sims,
     col = "skyblue", border = "white",
     main = "Clustering global en 100 redes ER",
     xlab = "Transitividad global",
     ylab = "Frecuencia",
     breaks = 15)
abline(v = p, col = "red", lwd = 2, lty = 2)
abline(v = mean(clustering_sims), col = "navy", lwd = 2)
legend("topright",
     legend = c(paste("Teórico (p =", p, ")"),
                 paste("Empírico =", round(mean(clustering_sims), 4))),
     col = c("red", "navy"), lwd = 2, lty = c(2, 1),
     bty = "n")

```

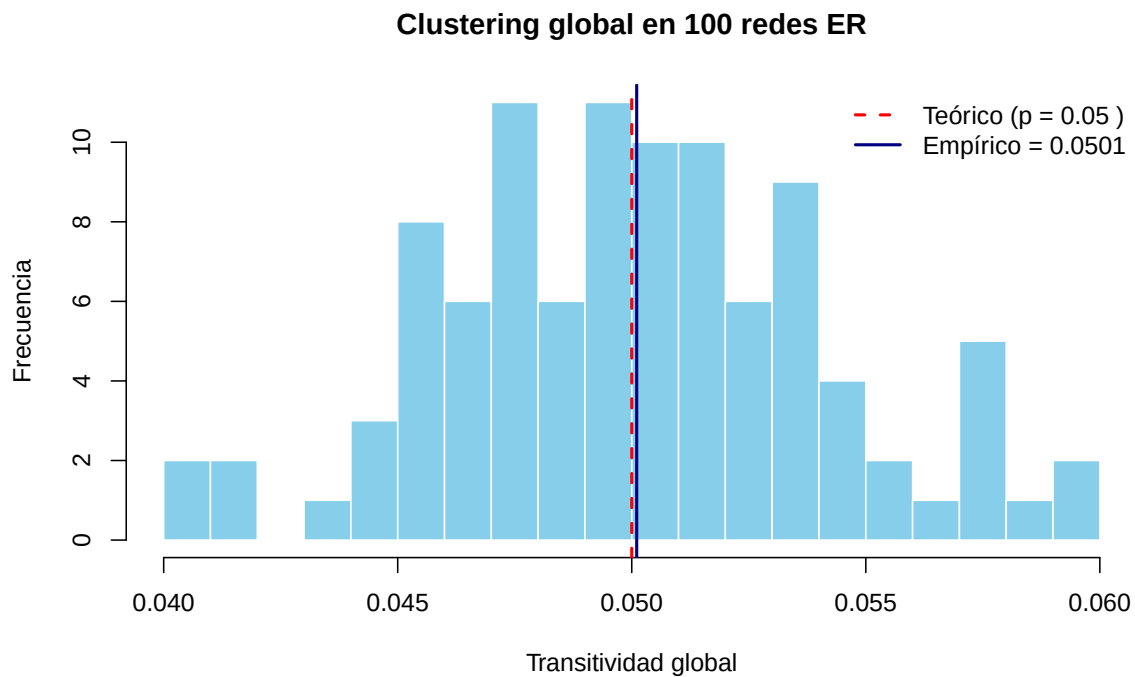


Figure 4.9: Distribución del clustering global en 100 redes ER ($n=200$, $p=0.05$)

Interpretación: En redes Erdős–Rényi, la transitividad global converge a p cuando n es grande. Con 200 nodos y $p = 0.05$, el clustering medio empírico está muy cerca de 0.05, confirmando la predicción teórica. La varianza es baja, lo que indica que el clustering es una propiedad estable en este modelo.

Ejercicios con pistas

Ejercicio 1: Compara el clustering entre una red aleatoria ER y una red *small-world* de 200 nodos cada una. Ambas deben tener un número similar de aristas. ¿Cuál tiene mayor clustering? ¿Por qué?

i Pista 1

Primero genera la red *small-world*: `sample_smallworld(1, 200, nei = 3, p = 0.05)`. Luego cuenta sus aristas con `ecount()` y genera una red ER con p tal que tenga un número similar: $p \approx \frac{2m}{n(n-1)}$.

i Pista 2

La red *small-world* debería tener un clustering mucho mayor porque las conexiones locales del modelo lattice original forman muchos triángulos. La red ER del mismo tamaño tiene clustering $\approx p$, que es muy bajo.

i Pista 3 — Pseudocódigo

```
1. set.seed(42)
2. g_sw <- sample_smallworld(1, 200, nei = 3, p = 0.05)
3. m <- ecount(g_sw)
4. p_er <- 2 * m / (200 * 199)
5. g_er <- sample_gnp(200, p_er)
6. cat("Aristas SW:", ecount(g_sw), "ER:", ecount(g_er))
7. cat("Clustering SW:", transitivity(g_sw, type = "global"))
8. cat("Clustering ER:", transitivity(g_er, type = "global"))
```

Ejercicio 2: ¿Qué nodos tienen clustering local 0 en una red BA de 100 nodos? ¿Qué los caracteriza (en términos de grado o posición)?

i Pista 1

Calcula `transitivity(g, type = "local")` y filtra con `which(cl == 0)`. Luego revisa el grado de esos nodos.

i Pista 2

En una red BA con $m = 1$ (por defecto), muchos nodos tienen grado 1. Con un solo vecino no se pueden formar triángulos, así que su clustering es NaN. Los nodos con clustering exactamente 0 tienen grado ≥ 2 pero ninguno de sus vecinos se conoce entre sí.

i Pista 3 — Pseudocódigo

```
1. set.seed(42)
2. g_ba <- sample_pa(100, directed = FALSE)
3. cl <- transitivity(g_ba, type = "local")
4. deg <- degree(g_ba)
5. # Nodos con clustering 0 (no NaN)
6. idx_cero <- which(cl == 0 & !is.nan(cl))
7. cat("Nodos con C = 0:", idx_cero)
8. cat("Sus grados:", deg[idx_cero])
9. # Comparar con nodos NaN
10. cat("Nodos NaN (grado < 2):", which(is.nan(cl)))
```

Ejercicio 3: Agrega aristas progresivamente a una red tipo anillo de 50 nodos para crear triángulos. En cada paso, agrega una arista entre nodos a distancia 2 (vecinos de vecinos). Registra cómo cambia el clustering global con cada arista añadida. Grafica la evolución.

i Pista 1

En un anillo, los nodos a distancia 2 son: nodo i y nodo $i + 2$ (módulo n). Agregar esa arista cierra un triángulo con el nodo $i + 1$.

i Pista 2

Usa un bucle: en cada iteración agrega `add_edges(g, c(i, (i+2) %% n + 1))` y calcula `transitivity(g, type = "global")`. Guarda los resultados en un vector.

i Pista 3 — Pseudocódigo

```
1. g <- make_ring(50)
2. cl_evol <- transitivity(g, type = "global") # inicial = 0
3. for (i in 1:25) {
4.     # Agregar arista entre nodo i y nodo i+2
```

```

5.     v1 <- i
6.     v2 <- ((i + 1) %% 50) + 1
7.     g <- add_edges(g, c(v1, v2))
8.     cl_evol <- c(cl_evol, transitivity(g, type = "global"))
9. }
10. plot(0:25, cl_evol, type = "b",
11.      xlab = "Aristas agregadas", ylab = "Clustering global")

```

Ejercicios propuestos

Ejercicio propuesto 1

Crea una red con tres triángulos conectados en cadena (triángulo A–B–C, triángulo C–D–E, triángulo E–F–G). Calcula el clustering local de cada nodo. ¿Los nodos “puente” (C y E) tienen clustering diferente al resto?

Autoevaluación: Los nodos puente C y E deberían tener clustering menor que los nodos terminales (A, B, F, G) porque tienen más vecinos pero no todos sus vecinos están conectados entre sí.

Ejercicio propuesto 2

Crea una función `proporcion_alto_clustering()` que reciba un grafo y un umbral, y devuelva la fracción de nodos cuyo clustering local es mayor que el umbral. Pruébala con una red *small-world* de 300 nodos y umbral 0.5.

Autoevaluación: En una red *small-world* con $p = 0.05$, la mayoría de los nodos debería tener clustering alto. Espera que más del 50% supere el umbral de 0.5.

Ejercicio propuesto 3

Reproduce el gráfico de transición *small-world* (clustering y distancia media vs. p) pero con una red de 500 nodos y `nei = 5`. ¿Cambia la forma de las curvas? ¿Se desplaza la “zona *small-world*”?

Autoevaluación: Las curvas deberían tener la misma forma general. Con más vecinos (`nei = 5`), el clustering inicial es más alto y la distancia media inicial más baja, pero la transición ocurre en un rango de p similar.

Ejercicio propuesto 4

Investiga la relación grado-clustering en una red BA de 1000 nodos con $m = 3$. Calcula el clustering local promedio para cada valor de grado (agrupa nodos por grado y promedia sus clustering). Grafica $\langle C(k) \rangle$ vs. k en escala log-log. ¿Ves una relación tipo *power-law*?

Autoevaluación: Deberías observar una tendencia decreciente: a mayor grado, menor clustering. En redes BA teóricas, $C(k) \sim k^{-1}$, por lo que en log-log debería verse una recta con pendiente ≈ -1 .

Ejercicio propuesto 5

Elige dos redes de la comparación (por ejemplo, *small-world* y BA). Elimina 10 nodos aleatorios de cada una y observa cómo cambia el clustering global. ¿Cuál es más robusta? Este ejercicio conecta con el Chapter 5.

Autoevaluación: La *small-world* debería mantener un clustering alto tras la eliminación porque sus triángulos están distribuidos uniformemente. En la BA, eliminar un *hub* puede destruir muchos triángulos de golpe.

4.11 Resumen del capítulo

```
flowchart LR
    A[Triángulos] --> B[Clustering local C_v]
    B --> C[Clustering global]
    C --> D[Comparar topologías]
    D --> E[Transición small-world]
    E --> F[Relación grado-clustering]
    F --> G["Siguiente: Robustez →"]
    style G fill:#e8f4f8,stroke:#3ABEF9
```

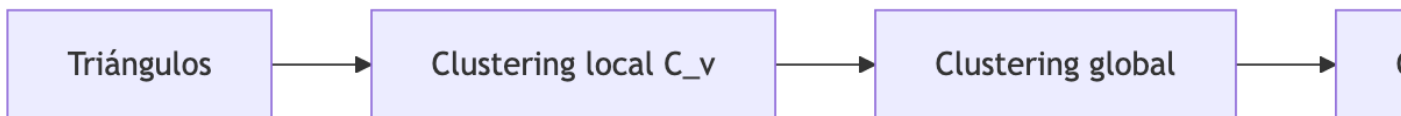


Figure 4.10: Resumen visual del capítulo

Table 4.4: Funciones y conceptos clave del capítulo

Función / Concepto	Descripción
<code>transitivity(g, type = "local")</code>	Clustering local de cada nodo
<code>transitivity(g, type = "global")</code>	Transitividad global (fracción de triángulos)
<code>transitivity(g, type = "average")</code>	Promedio del clustering local
<code>count_triangles(g)</code>	Número de triángulos por nodo
<code>sample_smallworld()</code>	Genera red Watts-Strogatz
Fenómeno <i>small-world</i>	Alto clustering + baja distancia media
$C(v) = 0$	Ningún par de vecinos de v está conectado
$C(v) = 1$	Todos los vecinos de v están conectados (clique local)

Conexión con el siguiente capítulo: El clustering y la distancia media describen la estructura “normal” de una red. Pero, ¿qué pasa cuando las cosas van mal? En el Chapter 5 estudiaremos qué ocurre cuando **eliminamos nodos** de la red — ya sea aleatoriamente (fallos) o atacando los más conectados (*hubs*). Verás cómo la estructura que hemos aprendido a medir determina la resistencia o fragilidad de una red.

Autoevaluación

Pregunta 1

¿Qué mide el coeficiente de clustering local $C(v)$?

- a) La distancia media de v al resto
- b) El número de vecinos de v
- c) La fracción de pares de vecinos de v que están conectados entre sí
- d) El grado de v normalizado

$C(v)$ mide “qué tanto se conocen entre sí los amigos de v ”.

Pregunta 2

Verdadero o Falso: En una red Erdős-Rényi, el clustering global es aproximadamente igual a la probabilidad p .

Verdadero. En una red ER, cada arista existe con probabilidad p de forma independiente, por lo que la probabilidad de que dos vecinos de un nodo estén conectados también es p .

Pregunta 3

¿Por qué un nodo con grado 1 tiene clustering NaN?

Porque con un solo vecino no se puede formar ningún par de vecinos. El denominador de la fórmula $C(v) = \frac{2e_v}{k_v(k_v-1)}$ es cero cuando $k_v = 1$.

Pregunta 4

¿Qué caracteriza a una red *small-world*?

- a) Distancia media alta y clustering alto
- b) Distancia media baja y clustering bajo
- c) Distancia media baja y clustering alto
- d) Distancia media alta y clustering bajo

La combinación de distancias cortas (como en redes aleatorias) y clustering alto (como en redes regulares) es la firma del fenómeno *small-world*.

Pregunta 5

Verdadero o Falso: En una red Barabási–Albert, los *hubs* suelen tener el clustering local más alto.

Falso. Los *hubs* suelen tener clustering **bajo** porque, aunque tienen muchos vecinos, es poco probable que todos esos vecinos estén conectados entre sí. El clustering tiende a decrecer con el grado.

5 Robustez de redes

i Objetivos de aprendizaje

Al finalizar este capítulo serás capaz de:

1. **Definir** robustez de una red y distinguir entre fallos aleatorios y ataques dirigidos.
2. **Implementar** simulaciones de eliminación de nodos (aleatoria y por grado) en igraph.
3. **Medir** el impacto de la eliminación usando métricas: distancia media, diámetro, tamaño del componente gigante.
4. **Comparar** la robustez de distintos modelos de red (ER, BA, *small-world*).
5. **Explicar** por qué las redes *scale-free* son robustas ante fallos pero vulnerables ante ataques.

Pregunta motivadora: Internet está formada por miles de millones de dispositivos conectados. Cada día fallan servidores, se cortan cables, se caen routers... y sin embargo, la red sigue funcionando. ¿Cómo es posible? ¿Y qué pasaría si en vez de fallos aleatorios, alguien atacara deliberadamente los nodos más importantes?

flowchart TD

```
A[Robustez de redes] --> B[Fallos aleatorios]
A --> C[Ataques dirigidos]
A --> D[Métricas de impacto]
B --> B1[Eliminar nodos al azar]
C --> C1[Eliminar hubs primero]
D --> D1[Componente gigante]
D --> D2[Distancia media]
D --> D3[Diámetro]
A --> E[Comparar topologías]
E --> E1[ER: robusto a ambos]
E --> E2[BA: robusto a fallos, frágil a ataques]
```

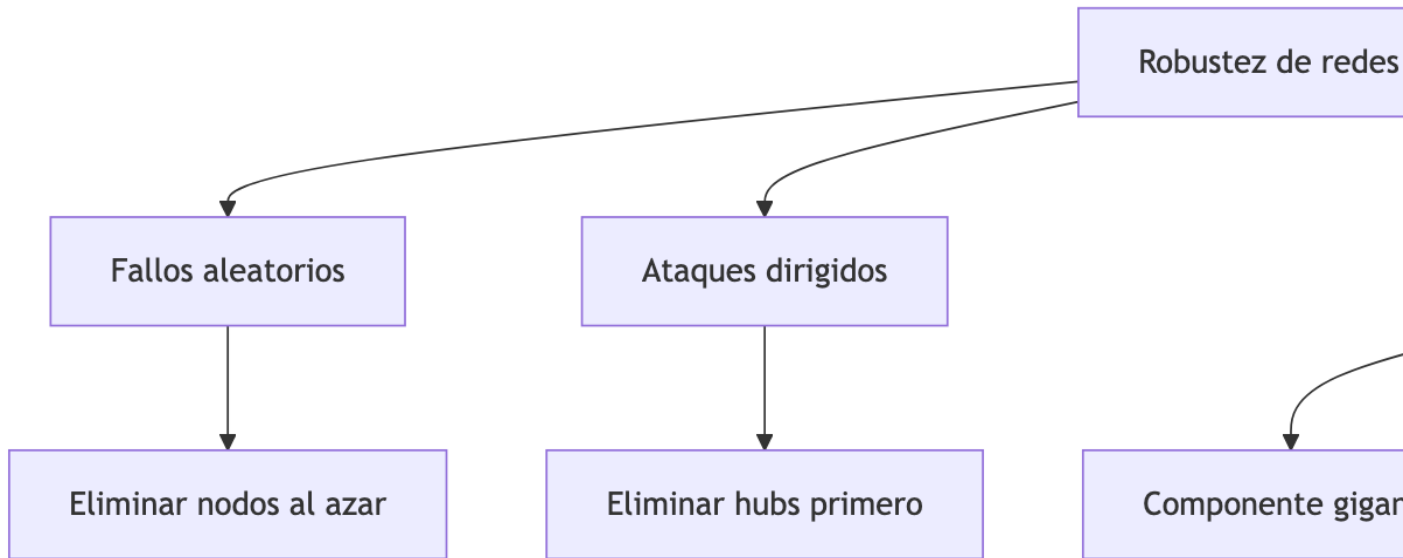


Figure 5.1: Mapa conceptual del capítulo

5.1 Preparación del entorno

```
library(igraph)
```

5.2 ¿Qué significa que una red sea robusta?

Una red es **robusta** si puede seguir funcionando (manteniendo la comunicación entre sus nodos) incluso cuando parte de ella falla. La robustez depende de dos factores:

1. **El tipo de perturbación:** ¿Los nodos fallan al azar o alguien los elige deliberadamente?
2. **La estructura de la red:** ¿Tiene *hubs*? ¿Es homogénea? ¿Qué tan redundantes son sus caminos?

💡 Concepto clave

Distinguimos dos escenarios fundamentales:

- **Fallo aleatorio** (*random failure*): los nodos se eliminan al azar, sin importar su importancia. Modela fallos técnicos, apagones o enfermedades aleatorias.
- **Ataque dirigido** (*targeted attack*): se eliminan deliberadamente los nodos más

conectados (*hubs*). Modela ataques cibernéticos, sabotaje o intervenciones estratégicas.

La respuesta de la red a cada escenario puede ser radicalmente diferente (Albert et al., 2000).

5.3 Herramienta fundamental: `delete_vertices()`

Antes de simular fallos y ataques, necesitamos dominar la eliminación de nodos en `igraph`.

```
# Crear red anillo con nombres
g <- make_ring(10)
V(g)$name <- LETTERS[1:10]

par(mfrow = c(1, 2), mar = c(1, 1, 3, 1))
lay <- layout_in_circle(g)

plot(g, layout = lay,
     vertex.color = "skyblue", vertex.size = 25,
     main = "Red original (10 nodos)")

# Eliminar nodos A, E y B
g2 <- delete_vertices(g, c("A", "E", "B"))

plot(g2,
     vertex.color = "salmon", vertex.size = 25,
     main = "Tras eliminar A, E, B")
```

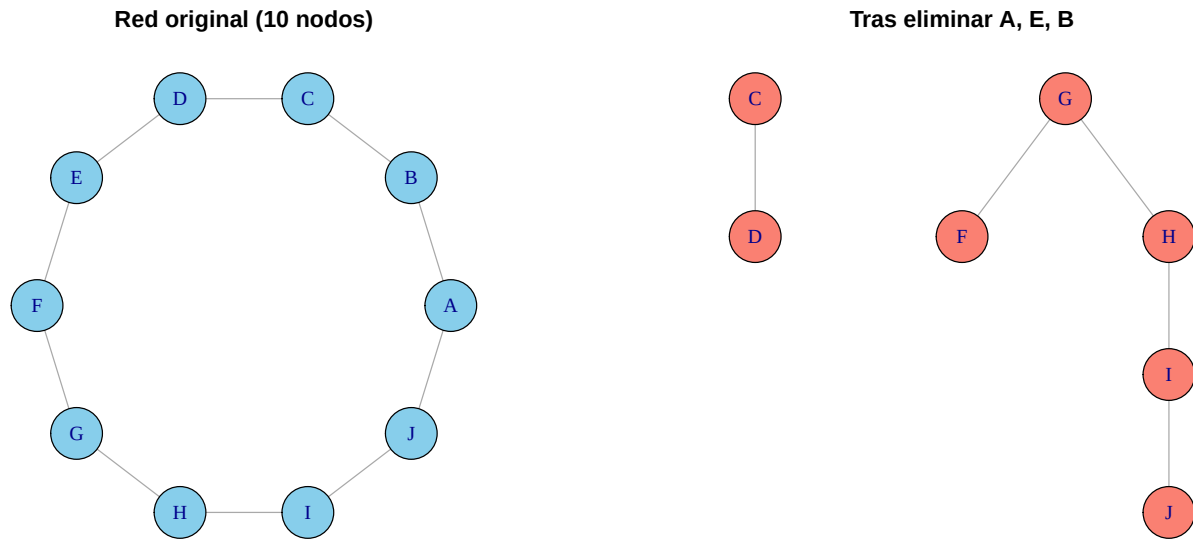


Figure 5.2: Eliminación de nodos en una red anillo: antes (izq.) y después (der.)

```
cat("=== Red original ===\n")
```

```
=== Red original ===
```

```
cat("Nodos:", vcount(g), " | Aristas:", ecount(g), "\n")
```

```
Nodos: 10 | Aristas: 10
```

```
cat("Componentes:", components(g)$no, "\n")
```

```
Componentes: 1
```

```
cat("Distancia media:", round(mean_distance(g), 3), "\n\n")
```

```
Distancia media: 2.778
```

```
cat("=== Tras eliminar A, E, B ===\n")
```

```
=== Tras eliminar A, E, B ===
```

```
cat("Nodos:", vcount(g2), " | Aristas:", ecoun(g2), "\n")
```

Nodos: 7 | Aristas: 5

```
cat("Componentes:", components(g2)$no, "\n")
```

Componentes: 2

```
cat("Distancia media:", round(mean_distance(g2), 3), "\n")
```

Distancia media: 1.909



Error frecuente

Después de `delete_vertices()`, los **índices de los nodos cambian**. Si eliminas el nodo 1, el que antes era nodo 2 ahora es nodo 1. Por eso es preferible trabajar con **nombres** (`V(g)$name`) en vez de índices numéricos cuando eliminas nodos iterativamente.

5.4 El componente gigante

Cuando eliminamos nodos, la red puede fragmentarse en componentes desconectados. El **componente gigante** es el componente conectado más grande. Su tamaño relativo (como fracción del total de nodos restantes) es la métrica principal de robustez:

$$S = \frac{|C_{\max}|}{n_{\text{restantes}}}$$

Si $S \approx 1$, la red sigue funcionando. Si $S \ll 1$, la red se fragmentó.

```
# Función auxiliar: tamaño relativo del componente gigante
tamano_gigante <- function(g) {
  comp <- components(g)
  max(comp$size) / vcount(g)
}

cat("Componente gigante (original):", tamano_gigante(g), "\n")
```


Componente gigante (original): 1

```
cat("Componente gigante (tras eliminar 3 nodos):",  
    round(tamano_gigante(g2), 3), "\n")
```

Componente gigante (tras eliminar 3 nodos): 0.714

5.5 Simulación de fallos aleatorios

Vamos a construir una función que elimine nodos al azar de forma iterativa y registre las métricas en cada paso.

```
simular_fallos <- function(g, fraccion_eliminar = 0.5, semilla = 42) {  
  set.seed(semilla)  
  n_original <- vcount(g)  
  n_eliminar <- floor(n_original * fraccion_eliminar)  
  
  # Seleccionar nodos a eliminar en orden aleatorio  
  orden_eliminacion <- sample(V(g)$name, n_eliminar)  
  
  # Registrar métricas en cada paso  
  resultados <- data.frame(  
    eliminados = 0:n_eliminar,  
    fraccion_eliminada = (0:n_eliminar) / n_original,  
    comp_gigante = numeric(n_eliminar + 1),  
    dist_media = numeric(n_eliminar + 1),  
    diametro = numeric(n_eliminar + 1)  
  )  
  
  g_temp <- g  
  
  for (i in 0:n_eliminar) {  
    if (i > 0) {  
      g_temp <- delete_vertices(g_temp, orden_eliminacion[i])  
    }  
  
    resultados$comp_gigante[i + 1] <- tamano_gigante(g_temp)  
  
    # Calcular distancia media solo en el componente gigante  
    comp <- components(g_temp)
```

```

giant_id <- which.max(comp$size)
giant_nodes <- which(comp$membership == giant_id)
if (length(giant_nodes) > 1) {
  g_giant <- induced_subgraph(g_temp, giant_nodes)
  resultados$dist_media[i + 1] <- mean_distance(g_giant)
  resultados$diametro[i + 1] <- diameter(g_giant)
} else {
  resultados$dist_media[i + 1] <- 0
  resultados$diametro[i + 1] <- 0
}
}

resultados
}

```

En un ataque dirigido, eliminamos los nodos **más conectados primero**. Recalculamos el grado después de cada eliminación, porque el nodo más conectado puede cambiar.

```

simular_ataques <- function(g, fraccion_eliminar = 0.5, semilla = 42) {
  set.seed(semilla)
  n_original <- vcount(g)
  n_eliminar <- floor(n_original * fraccion_eliminar)

  resultados <- data.frame(
    eliminados = 0:n_eliminar,
    fraccion_eliminada = (0:n_eliminar) / n_original,
    comp_gigante = numeric(n_eliminar + 1),
    dist_media = numeric(n_eliminar + 1),
    diametro = numeric(n_eliminar + 1)
  )

  g_temp <- g

  for (i in 0:n_eliminar) {
    if (i > 0) {
      # Identificar el nodo con mayor grado (recalculado)
      deg <- degree(g_temp)
      hub <- V(g_temp)$name[which.max(deg)]
      g_temp <- delete_vertices(g_temp, hub)
    }
  }
}

```

```

    resultados$comp_gigante[i + 1] <- tamano_gigante(g_temp)

    # Calcular distancia media solo en el componente gigante
    comp <- components(g_temp)
    giant_id <- which.max(comp$size)
    giant_nodes <- which(comp$membership == giant_id)
    if (length(giant_nodes) > 1) {
      g_giant <- induced_subgraph(g_temp, giant_nodes)
      resultados$dist_media[i + 1] <- mean_distance(g_giant)
      resultados$diametro[i + 1] <- diameter(g_giant)
    } else {
      resultados$dist_media[i + 1] <- 0
      resultados$diametro[i + 1] <- 0
    }
  }
}

resultados
}

```

! Recuerda

En la simulación de ataques, es crucial **recalcular el grado** en cada paso. Después de eliminar un *hub*, otro nodo puede convertirse en el nuevo más conectado. Esto hace que la simulación sea más lenta pero realista.

5.6 Experimento: ER vs. BA bajo fallos y ataques

Ahora viene el experimento central del capítulo. Generaremos una red Erdős–Rényi y una Barabási–Albert de tamaño similar, y las someteremos a ambos tipos de perturbación.

```

set.seed(123)

n <- 500

# Red ER con ~1000 aristas
g_er <- sample_gnp(n, p = 2 / (n - 1) * 2)
V(g_er)$name <- paste0("ER_", 1:vcount(g_er))

# Red BA con parámetro m = 2 (~1000 aristas)

```

```
g_ba <- sample_pa(n, m = 2, directed = FALSE)
V(g_ba)$name <- paste0("BA_", 1:vcount(g_ba))

cat("=== Red Erdős-Rényi ===\n")
```

=== Red Erdős-Rényi ===

```
cat("Nodos:", vcount(g_er), " | Aristas:", ecount(g_er), "\n")
```

Nodos: 500 | Aristas: 965

```
cat("Grado medio:", round(mean(degree(g_er)), 2), "\n")
```

Grado medio: 3.86

```
cat("Grado máximo:", max(degree(g_er)), "\n\n")
```

Grado máximo: 10

```
cat("=== Red Barabási-Albert ===\n")
```

=== Red Barabási-Albert ===

```
cat("Nodos:", vcount(g_ba), " | Aristas:", ecount(g_ba), "\n")
```

Nodos: 500 | Aristas: 997

```
cat("Grado medio:", round(mean(degree(g_ba)), 2), "\n")
```

Grado medio: 3.99

```
cat("Grado máximo:", max(degree(g_ba)), "\n")
```

Grado máximo: 54

```
# Simular los 4 escenarios
fallo_er <- simular_fallos(g_er, fraccion_eliminar = 0.4)
fallo_ba <- simular_fallos(g_ba, fraccion_eliminar = 0.4)
ataque_er <- simular_ataques(g_er, fraccion_eliminar = 0.4)
ataque_ba <- simular_ataques(g_ba, fraccion_eliminar = 0.4)
```

5.6.1 Resultado 1: Componente gigante

```
par(mfrow = c(1, 2), mar = c(4, 4, 3, 1))

# Fallos aleatorios
plot(fallo_er$fraccion_eliminada, fallo_er$comp_gigante,
     type = "l", lwd = 2.5, col = "skyblue",
     xlab = "Fracción de nodos eliminados",
     ylab = "Tamaño relativo del componente gigante",
     main = "Fallos aleatorios",
     ylim = c(0, 1))
lines(fallo_ba$fraccion_eliminada, fallo_ba$comp_gigante,
      lwd = 2.5, col = "tomato")
legend("topright", legend = c("ER", "BA"),
      col = c("skyblue", "tomato"), lwd = 2.5, bty = "n")

# Ataques dirigidos
plot(ataque_er$fraccion_eliminada, ataque_er$comp_gigante,
     type = "l", lwd = 2.5, col = "skyblue",
     xlab = "Fracción de nodos eliminados",
     ylab = "Tamaño relativo del componente gigante",
     main = "Ataques dirigidos (hubs primero)",
     ylim = c(0, 1))
lines(ataque_ba$fraccion_eliminada, ataque_ba$comp_gigante,
      lwd = 2.5, col = "tomato")
legend("topright", legend = c("ER", "BA"),
      col = c("skyblue", "tomato"), lwd = 2.5, bty = "n")
```

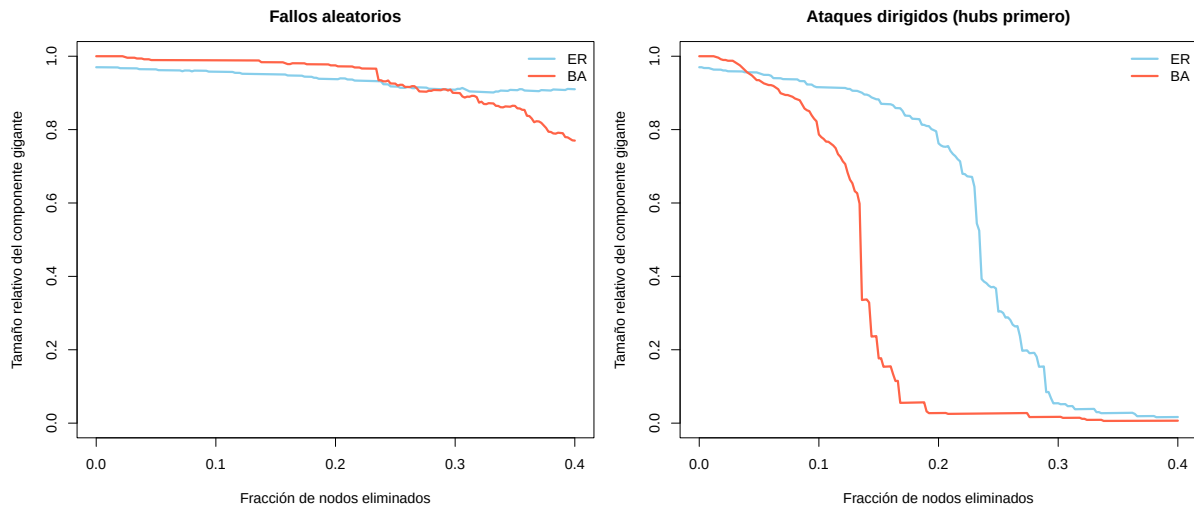


Figure 5.3: Tamaño del componente gigante bajo fallos aleatorios (izq.) y ataques dirigidos (der.)



Concepto clave

Este es el resultado más famoso de la robustez de redes (Albert et al., 2000):

- **Ante fallos aleatorios**, la red BA es **más robusta** que la ER: como la mayoría de sus nodos tienen grado bajo, es improbable que un fallo aleatorio afecte un *hub*.
- **Ante ataques dirigidos**, la red BA es **extremadamente vulnerable**: eliminar unos pocos *hubs* destruye la conectividad global. La red ER, sin *hubs*, resiste mejor.

Esta dualidad se conoce como la paradoja de la **robustez-fragilidad** de las redes *scale-free*.

5.6.2 Resultado 2: Distancia media

```
par(mfrow = c(1, 2), mar = c(4, 4, 3, 1))

# Calcular ylim seguro para distancias
all_dist <- c(fallo_er$dist_media, fallo_ba$dist_media,
             ataque_er$dist_media, ataque_ba$dist_media)
ylim_max <- max(all_dist, na.rm = TRUE)
if (!is.finite(ylim_max) || ylim_max == 0) ylim_max <- 10
ylim_dist <- c(0, ylim_max * 1.1)

# Fallos
```

```

plot(fallo_er$fraccion_eliminada, fallo_er$dist_media,
     type = "l", lwd = 2.5, col = "skyblue",
     xlab = "Fracción de nodos eliminados",
     ylab = "Distancia media",
     main = "Distancia media - Fallos aleatorios",
     ylim = ylim_dist)
lines(fallo_ba$fraccion_eliminada, fallo_ba$dist_media,
      lwd = 2.5, col = "tomato")
legend("topleft", legend = c("ER", "BA"),
      col = c("skyblue", "tomato"), lwd = 2.5, bty = "n")

# Ataques
plot(ataque_er$fraccion_eliminada, ataque_er$dist_media,
     type = "l", lwd = 2.5, col = "skyblue",
     xlab = "Fracción de nodos eliminados",
     ylab = "Distancia media",
     main = "Distancia media - Ataques dirigidos",
     ylim = ylim_dist)
lines(ataque_ba$fraccion_eliminada, ataque_ba$dist_media,
      lwd = 2.5, col = "tomato")
legend("topleft", legend = c("ER", "BA"),
      col = c("skyblue", "tomato"), lwd = 2.5, bty = "n")

```

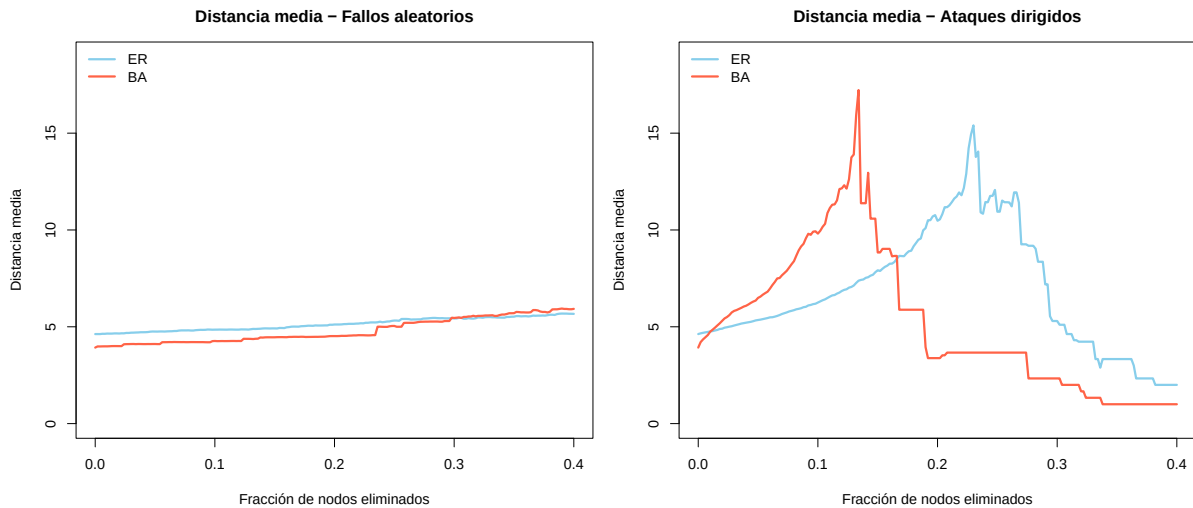


Figure 5.4: Evolución de la distancia media bajo fallos (izq.) y ataques (der.)

5.6.3 Resultado 3: Diámetro

```
par(mfrow = c(1, 2), mar = c(4, 4, 3, 1))

# Calcular ylim seguro para diámetro
all_diam <- c(fallo_er$diametro, fallo_ba$diametro,
             ataque_er$diametro, ataque_ba$diametro)
ylim_max_d <- max(all_diam, na.rm = TRUE)
if (!is.finite(ylim_max_d) || ylim_max_d == 0) ylim_max_d <- 10
ylim_diam <- c(0, ylim_max_d * 1.1)

# Fallos
plot(fallo_er$fraccion_eliminada, fallo_er$diametro,
     type = "l", lwd = 2.5, col = "skyblue",
     xlab = "Fracción de nodos eliminados",
     ylab = "Diámetro",
     main = "Diámetro - Fallos aleatorios",
     ylim = ylim_diam)
lines(fallo_ba$fraccion_eliminada, fallo_ba$diametro,
      lwd = 2.5, col = "tomato")
legend("topleft", legend = c("ER", "BA"),
      col = c("skyblue", "tomato"), lwd = 2.5, bty = "n")

# Ataques
plot(ataque_er$fraccion_eliminada, ataque_er$diametro,
     type = "l", lwd = 2.5, col = "skyblue",
     xlab = "Fracción de nodos eliminados",
     ylab = "Diámetro",
     main = "Diámetro - Ataques dirigidos",
     ylim = ylim_diam)
lines(ataque_ba$fraccion_eliminada, ataque_ba$diametro,
      lwd = 2.5, col = "tomato")
legend("topleft", legend = c("ER", "BA"),
      col = c("skyblue", "tomato"), lwd = 2.5, bty = "n")
```

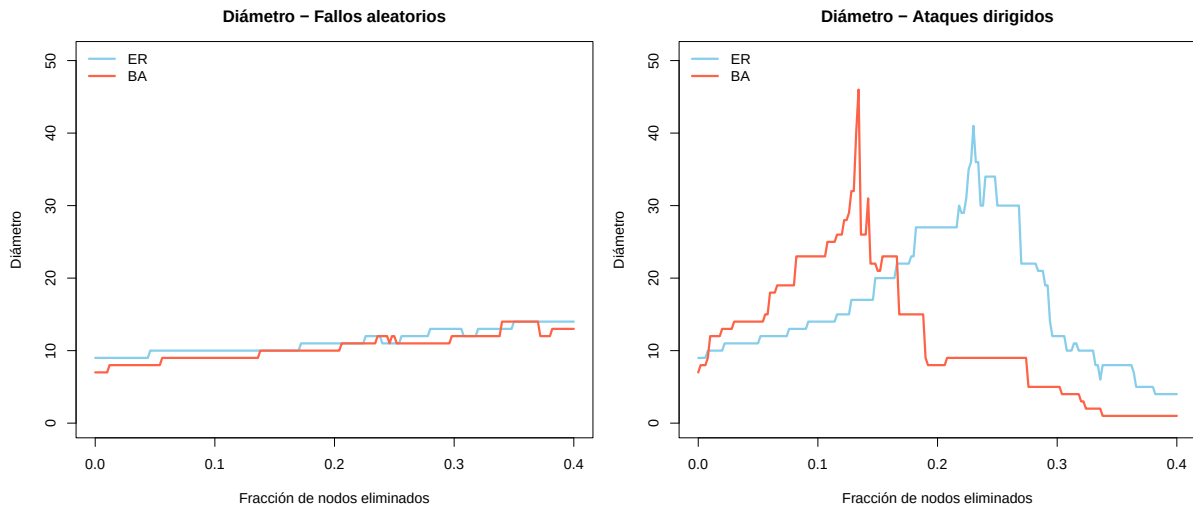



Figure 5.5: Evolución del diámetro bajo fallos (izq.) y ataques (der.)

5.7 Incluyendo la red *Small-World*

Agreguemos una tercera topología para tener una comparación más completa:

```
set.seed(123)

g_sw <- sample_smallworld(1, n, nei = 2, p = 0.05)
V(g_sw)$name <- paste0("SW_", 1:vcount(g_sw))

cat("=== Red Small-World ===\n")
```

```
=== Red Small-World ===
```

```
cat("Nodos:", vcount(g_sw), " | Aristas:", ecount(g_sw), "\n")
```

```
Nodos: 500 | Aristas: 1000
```

```
cat("Grado medio:", round(mean(degree(g_sw)), 2), "\n")
```

```
Grado medio: 4
```

```
cat("Grado máximo:", max(degree(g_sw)), "\n")
```

Grado máximo: 7

```
fallo_sw <- simular_fallos(g_sw, fraccion_eliminar = 0.4)
ataque_sw <- simular_ataques(g_sw, fraccion_eliminar = 0.4)
```

```
par(mfrow = c(1, 2), mar = c(4, 4, 3, 1))

# Fallos
plot(fallo_er$fraccion_eliminada, fallo_er$comp_gigante,
     type = "l", lwd = 2.5, col = "skyblue",
     xlab = "Fracción eliminada", ylab = "Componente gigante",
     main = "Fallos aleatorios", ylim = c(0, 1))
lines(fallo_ba$fraccion_eliminada, fallo_ba$comp_gigante,
      lwd = 2.5, col = "tomato")
lines(fallo_sw$fraccion_eliminada, fallo_sw$comp_gigante,
      lwd = 2.5, col = "mediumpurple")
legend("topright", legend = c("ER", "BA", "Small-world"),
      col = c("skyblue", "tomato", "mediumpurple"), lwd = 2.5, bty = "n")

# Ataques
plot(ataque_er$fraccion_eliminada, ataque_er$comp_gigante,
     type = "l", lwd = 2.5, col = "skyblue",
     xlab = "Fracción eliminada", ylab = "Componente gigante",
     main = "Ataques dirigidos", ylim = c(0, 1))
lines(ataque_ba$fraccion_eliminada, ataque_ba$comp_gigante,
      lwd = 2.5, col = "tomato")
lines(ataque_sw$fraccion_eliminada, ataque_sw$comp_gigante,
      lwd = 2.5, col = "mediumpurple")
legend("topright", legend = c("ER", "BA", "Small-world"),
      col = c("skyblue", "tomato", "mediumpurple"), lwd = 2.5, bty = "n")
```

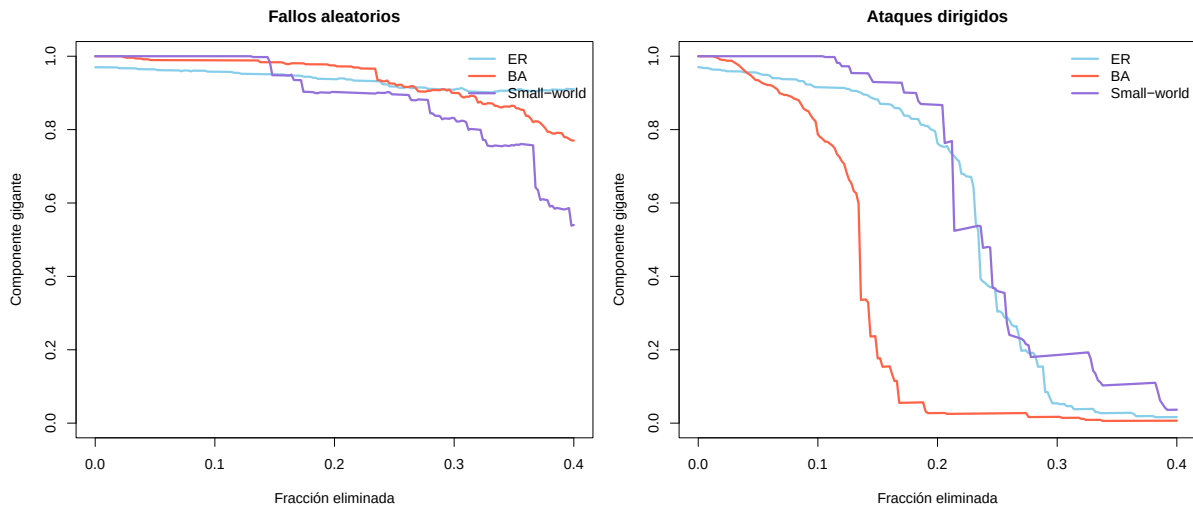


Figure 5.6: Componente gigante bajo fallos y ataques para tres topologías

5.8 Tabla resumen de robustez

```
# Fracción de nodos que se deben eliminar para que el comp. gigante baje del 50%
umbral_fragmentacion <- function(resultados, umbral = 0.5) {
  idx <- which(resultados$comp_gigante < umbral)
  if (length(idx) == 0) return("> 40%")
  paste0(round(resultados$fraccion_eliminada[min(idx)] * 100, 1), "%")
}

resumen_rob <- data.frame(
  Red = c("Erdős-Rényi", "Barabási-Albert", "Small-world"),
  Fallo_50 = c(
    umbral_fragmentacion(fallo_er),
    umbral_fragmentacion(fallo_ba),
    umbral_fragmentacion(fallo_sw)
  ),
  Ataque_50 = c(
    umbral_fragmentacion(ataque_er),
    umbral_fragmentacion(ataque_ba),
    umbral_fragmentacion(ataque_sw)
  )
)

knitr::kable(resumen_rob,
```

```
caption = "Fracción de nodos a eliminar para fragmentar la red (comp. gigante < 50%)
col.names = c("Red", "Fallos aleatorios", "Ataques dirigidos"))
```

Table 5.1: Fracción de nodos a eliminar para fragmentar la red (comp. gigante < 50%)

Red	Fallos aleatorios	Ataques dirigidos
Erdős–Rényi	> 40%	23.6%
Barabási–Albert	> 40%	13.6%
Small-world	> 40%	23.8%

5.9 Visualización del proceso de fragmentación

Veamos cómo luce la red BA antes, durante y después de un ataque dirigido:

```
set.seed(42)
g_vis <- sample_pa(100, m = 2, directed = FALSE)
V(g_vis)$name <- paste0("v", 1:100)

# Layout fijo
lay_vis <- layout_with_fr(g_vis)

par(mfrow = c(1, 3), mar = c(1, 1, 3, 1))

# Estado original
V(g_vis)$color <- "skyblue"
V(g_vis)$size <- degree(g_vis) * 0.8 + 3
plot(g_vis, layout = lay_vis, vertex.label = NA,
     edge.color = adjustcolor("gray50", 0.4),
     main = paste0("Original\n(", vcount(g_vis), " nodos, ",
                   components(g_vis)$no, " comp.)"))

# Eliminar los 5 hubs más conectados
g_vis2 <- g_vis
for (j in 1:5) {
  hub <- V(g_vis2)$name[which.max(degree(g_vis2))]
  g_vis2 <- delete_vertices(g_vis2, hub)
}
plot(g_vis2, vertex.label = NA,
     vertex.size = degree(g_vis2) * 0.8 + 3,
```

```

vertex.color = "gold",
edge.color = adjustcolor("gray50", 0.4),
main = paste0("Tras 5 ataques\n(", vcount(g_vis2), " nodos, ",
              components(g_vis2)$no, " comp.)")

# Eliminar 5 hubs más (10 total)
for (j in 1:5) {
  hub <- V(g_vis2)$name[which.max(degree(g_vis2))]
  g_vis2 <- delete_vertices(g_vis2, hub)
}
plot(g_vis2, vertex.label = NA,
     vertex.size = degree(g_vis2) * 0.8 + 3,
     vertex.color = "tomato",
     edge.color = adjustcolor("gray50", 0.4),
     main = paste0("Tras 10 ataques\n(", vcount(g_vis2), " nodos, ",
                  components(g_vis2)$no, " comp.)"))

```

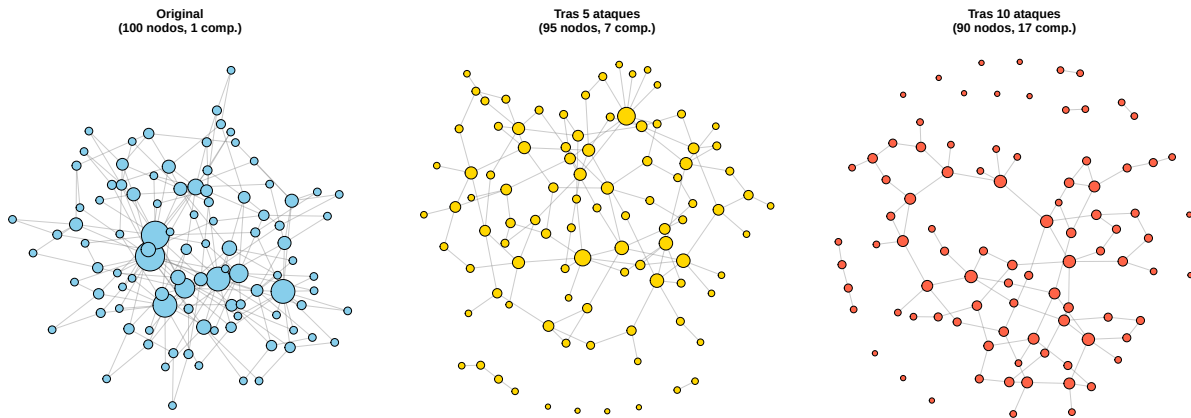


Figure 5.7: Fragmentación progresiva de una red BA (100 nodos) bajo ataque dirigido

5.10 Múltiples realizaciones: promediar la incertidumbre

Una sola simulación puede dar resultados atípicos. Para tener conclusiones robustas, repetimos el experimento múltiples veces y promediamos.

```

set.seed(42)

n_reps <- 10

```

```

n_red <- 200
fraccion <- 0.5
n_elim <- floor(n_red * fraccion)

# Matriz para almacenar resultados
resultados_mat <- matrix(NA, nrow = n_elim + 1, ncol = n_reps)

for (r in 1:n_reps) {
  g_rep <- sample_pa(n_red, m = 2, directed = FALSE)
  V(g_rep)$name <- paste0("v", 1:vcount(g_rep))

  # Orden aleatorio de eliminación
  orden <- sample(V(g_rep)$name, n_elim)
  g_temp <- g_rep
  resultados_mat[1, r] <- tamaño_gigante(g_temp)

  for (i in 1:n_elim) {
    g_temp <- delete_vertices(g_temp, orden[i])
    resultados_mat[i + 1, r] <- tamaño_gigante(g_temp)
  }
}

# Calcular media y banda
frac_x <- (0:n_elim) / n_red
media <- rowMeans(resultados_mat)
desv <- apply(resultados_mat, 1, sd)

plot(frac_x, media,
     type = "l", lwd = 3, col = "tomato",
     xlab = "Fracción de nodos eliminados",
     ylab = "Componente gigante (media ± 1 DE)",
     main = paste("Fallos aleatorios - Red BA (n=200),", n_reps, "realizaciones"),
     ylim = c(0, 1))

# Banda de ± 1 desviación estándar
polygon(c(frac_x, rev(frac_x)),
       c(media + desv, rev(media - desv)),
       col = adjustcolor("tomato", 0.2), border = NA)

lines(frac_x, media, lwd = 3, col = "tomato")
legend("topright",

```

```
legend = c("Media", "± 1 DE"),  
col = c("tomato", adjustcolor("tomato", 0.3)),  
lwd = c(3, 10), bty = "n")
```

Fallos aleatorios – Red BA (n=200), 10 realizaciones

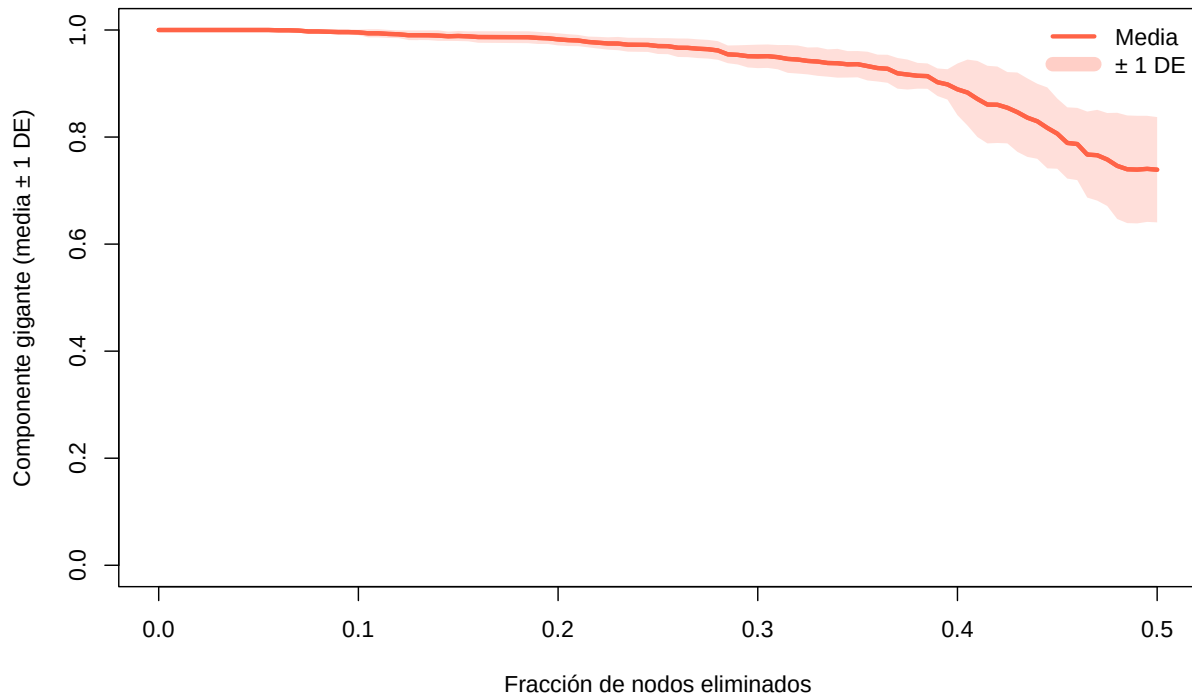


Figure 5.8: Componente gigante promedio (10 realizaciones) bajo fallos aleatorios para redes BA de 200 nodos



Concepto clave

La **banda de variabilidad** (sombreada) muestra cuánto varían los resultados entre realizaciones. Si la banda es estrecha, el resultado es robusto y reproducible. Siempre que hagas simulaciones estocásticas, reporta la variabilidad, no solo la media.

5.11 Ejercicios

Ejercicios resueltos

Ejercicio resuelto 1: Genera las siguientes cuatro redes. Para cada una, elimina aleatoriamente 1/100 de sus nodos 10 veces consecutivas. En cada paso, calcula la distancia media y el diámetro. Presenta los resultados en una tabla.

- `g100`: Barabási–Albert con 100 nodos
- `g1K`: Barabási–Albert con 1000 nodos
- `g_er`: Erdős–Rényi con 1000 nodos y $p = 0.20$
- `g_sw`: *Small-world* con 1000 nodos, $nei = 3$, $p = 0.2$

Solución

Paso 1: Crear las redes.

```
set.seed(42)

redes_ej <- list(
  BA_100 = sample_pa(100, directed = FALSE),
  BA_1000 = sample_pa(1000, directed = FALSE),
  ER_1000 = sample_gnp(1000, 0.20),
  SW_1000 = sample_smallworld(1, 1000, nei = 3, p = 0.2)
)

# Asignar nombres a los nodos
for (nombre in names(redes_ej)) {
  V(redes_ej[[nombre]])$name <- paste0("n", 1:vcount(redes_ej[[nombre]]))
}
```

Paso 2: Simular eliminación de 1/100 de los nodos, 10 veces consecutivas.


```

resultados_ej1 <- list()

for (nombre in names(redes_ej)) {
  g <- redes_ej[[nombre]]
  n <- vcount(g)
  n_elim_paso <- max(1, floor(n / 100)) # 1/100 de los nodos

  registros <- data.frame(
    paso = 0:10,
    nodos_restantes = numeric(11),
    dist_media = numeric(11),
    diametro = numeric(11)
  )

  g_temp <- g
  for (paso in 0:10) {
    if (paso > 0) {
      # Eliminar 1/100 nodos al azar
      n_actual <- vcount(g_temp)
      n_quitar <- min(n_elim_paso, n_actual - 1)
      victimas <- sample(V(g_temp)$name, n_quitar)
      g_temp <- delete_vertices(g_temp, victimas)
    }
    registros$nodos_restantes[paso + 1] <- vcount(g_temp)
    registros$dist_media[paso + 1] <- round(mean_distance(g_temp), 3)
    registros$diametro[paso + 1] <- diameter(g_temp)
  }

  resultados_ej1[[nombre]] <- registros
}

```

Paso 3: Mostrar la tabla resumen (valores finales tras 10 rondas).

```

resumen_final <- data.frame(
  Red = names(redes_ej),
  N_original = sapply(redes_ej, vcount),
  N_final = sapply(resultados_ej1, function(r) r$nodos_restantes[11]),
  Dist_media_inicio = sapply(resultados_ej1, function(r) r$dist_media[1]),
  Dist_media_final = sapply(resultados_ej1, function(r) r$dist_media[11]),
  Diámetro_inicio = sapply(resultados_ej1, function(r) r$diametro[1]),
  Diámetro_final = sapply(resultados_ej1, function(r) r$diametro[11])
)

knitr::kable(resumen_final,
  caption = "Efecto de eliminar 1/100 nodos aleatorios en 10 rondas",
  col.names = c("Red", "N original", "N final",
    "Dist. media (inicio)", "Dist. media (final)",
    "Diámetro (inicio)", "Diámetro (final)"))

```

Table 5.2: Efecto de eliminar 1/100 nodos aleatorios en 10 rondas

Red	N original	N final	Dist. media (inicio)	Dist. media (final)	Diámetro (inicio)	Diámetro (final)
BA_100	100	90	5.816	5.706	12	12
BA_1000	1000	900	7.790	6.978	20	17
ER_1000	1000	900	1.801	1.801	2	2
SW_1000	1000	900	4.463	4.671	8	9

Interpretación: Las redes grandes (1000 nodos) apenas notan la eliminación de 10 nodos por ronda. La red ER con $p = 0.20$ es tan densa que prácticamente no se ve afectada. Las redes BA son más sensibles a los cambios porque tienen menos redundancia en sus caminos.

Ejercicio resuelto 2: Usando las mismas cuatro redes, elimina los 10 nodos más conectados (*hubs*). Compara la distancia media y el diámetro antes y después del ataque. ¿Qué red sufre más?

💡 Solución

Paso 1: Función para eliminar los top- k hubs.

```
eliminar_top_hubs <- function(g, k = 10) {  
  g_temp <- g  
  for (i in 1:k) {  
    hub <- V(g_temp)$name[which.max(degree(g_temp))]  
    g_temp <- delete_vertices(g_temp, hub)  
  }  
  g_temp  
}
```

Paso 2: Aplicar a cada red y comparar.

```
comparacion_ataque <- data.frame(  
  Red = names(redes_ej),  
  Dist_antes = round(sapply(redes_ej, mean_distance), 3),  
  Diámetro_antes = sapply(redes_ej, diameter),  
  Componentes_antes = sapply(redes_ej, function(g) components(g)$no)  
)  
  
redes_atacadas <- lapply(redes_ej, eliminar_top_hubs, k = 10)  
  
comparacion_ataque$Dist_después <- round(  
  sapply(redes_atacadas, mean_distance), 3)  
comparacion_ataque$Diámetro_después <- sapply(redes_atacadas, diameter)  
comparacion_ataque$Componentes_después <- sapply(  
  redes_atacadas, function(g) components(g)$no)  
  
knitr::kable(comparacion_ataque,  
  caption = "Efecto de eliminar los 10 hubs más conectados",  
  col.names = c("Red", "Dist. antes", "Diám. antes", "Comp. antes",  
    "Dist. después", "Diám. después", "Comp. después"))
```

Table 5.3: Efecto de eliminar los 10 hubs más conectados

Red	Dist. antes	Diám. antes	Comp. antes	Dist. después	Diám. después	Comp. después
BA_100BA_100	5.816	12	1	2.062	6	43
BA_100BA_1000	7.790	20	1	4.870	15	186
ER_100ER_1000	1.801	2	1	1.802	2	1
SW_100SW_1000	4.463	8	1	4.533	8	1

Interpretación: Las redes BA sufren el mayor impacto porque sus *hubs* concentran una gran fracción de las conexiones. Al eliminarlos, la red se fragmenta en múltiples componentes, el diámetro puede dispararse o volverse infinito, y la distancia media aumenta drásticamente. Las redes ER y *small-world*, sin *hubs* dominantes, resisten mucho mejor.

Ejercicios con pistas

Ejercicio 1: Modifica la función `simular_ataques()` para que en vez de eliminar el nodo con mayor grado, elimine el nodo con mayor *betweenness centrality*. ¿Cambia el resultado? Compara ambas estrategias de ataque en una red BA de 300 nodos.

i Pista 1

Reemplaza `which.max(degree(g_temp))` por `which.max(betweenness(g_temp))`. La *betweenness* mide cuántos caminos más cortos pasan por un nodo.

i Pista 2

El ataque por *betweenness* puede ser aún más destructivo que por grado, porque apunta a los “puentes” de la red — nodos que conectan comunidades, aunque no necesariamente tengan el grado más alto.

i Pista 3 — Pseudocódigo

1. Copiar `simular_ataques()` como `simular_ataques_betw()`
2. Cambiar: `deg <- degree(g_temp)`
Por: `btw <- betweenness(g_temp)`
3. Cambiar: `hub <- V(g_temp)$name[which.max(deg)]`
Por: `hub <- V(g_temp)$name[which.max(btw)]`

4. Ejecutar ambas sobre `sample_pa(300, m=2, directed=FALSE)`
5. Graficar `comp_gigante` de ambas en el mismo gráfico

Ejercicio 2: Genera una red BA de 500 nodos. Realiza 20 simulaciones de fallo aleatorio (eliminando el 30% de los nodos en cada una). Calcula el promedio y la desviación estándar del tamaño final del componente gigante. Presenta un histograma de los 20 valores finales.

i Pista 1

Usa un bucle externo para las 20 repeticiones. En cada una, genera una nueva permutación aleatoria de nodos a eliminar (usa semillas diferentes).

i Pista 2

Almacena solo el valor final de `tamano_gigante()` después de eliminar el 30%. Al final, tendrás un vector de 20 valores para graficar con `hist()`.

i Pista 3 — Pseudocódigo

```
1. set.seed(42)
2. g <- sample_pa(500, m = 2, directed = FALSE)
3. V(g)$name <- paste0("v", 1:500)
4. resultados <- numeric(20)
5. for (r in 1:20) {
6.     res <- simular_fallos(g, fraccion = 0.3, semilla = r)
7.     resultados[r] <- tail(res$comp_gigante, 1)
8. }
9. hist(resultados, col = "tomato")
10. cat("Media:", mean(resultados), "DE:", sd(resultados))
```

Ejercicio 3: Compara la robustez de una red *small-world* variando el parámetro p : genera redes con $p \in \{0, 0.01, 0.05, 0.1, 0.5\}$ y somete cada una a un ataque dirigido del 20%. ¿Para qué valor de p la red es más robusta ante ataques?

i Pista 1

Con $p = 0$ (lattice puro), todos los nodos tienen el mismo grado, así que el ataque “dirigido” es esencialmente aleatorio. Con p alto, aparecen nodos con grado ligeramente mayor que pueden ser objetivo.

i Pista 2

Ejecuta `simular_ataques()` para cada red y compara el valor final de `comp_gigante`. Grafica con `barplot()` o `plot()` para ver la tendencia.

i Pista 3 — Pseudocódigo

```
1. p_vals <- c(0, 0.01, 0.05, 0.1, 0.5)
2. robustez <- numeric(length(p_vals))
3. for (i in seq_along(p_vals)) {
4.   g <- sample_smallworld(1, 300, nei = 3, p = p_vals[i])
5.   V(g)$name <- paste0("v", 1:vcount(g))
6.   res <- simular_ataques(g, fraccion = 0.2)
7.   robustez[i] <- tail(res$comp_gigante, 1)
8. }
9. barplot(robustez, names.arg = p_vals,
10.        xlab = "p", ylab = "Comp. gigante tras 20% ataque")
```

Ejercicios propuestos

Ejercicio propuesto 1

Genera una red anillo de 100 nodos. Elimina 10 nodos aleatorios y observa cuántos componentes se crean. Repite 20 veces y reporta el número promedio de componentes.

Autoevaluación: En un anillo, cada nodo eliminado puede crear a lo sumo 1 componente nuevo (si corta la cadena). Con 10 nodos eliminados, espera entre 5 y 10 componentes en promedio.

Ejercicio propuesto 2

Compara la robustez ante ataques de una red BA con $m = 1$ vs. $m = 3$ (ambas con 500 nodos). ¿Cuál es más robusta? ¿Por qué?

Autoevaluación: La red con $m = 3$ debería ser más robusta porque tiene más redundancia (mayor grado medio = más caminos alternativos). Sin embargo, ambas siguen siendo vulnerables a ataques dirigidos por su estructura *scale-free*.

Ejercicio propuesto 3

Diseña una métrica propia de robustez: el “índice R ” = área bajo la curva del componente gigante vs. fracción eliminada (usando la regla del trapecio). Un $R = 1$ significa red indestructible; $R = 0$ significa que colapsa inmediatamente. Calcula R para una red ER y una BA bajo ataques.

Autoevaluación: Puedes calcular el área con `sum(diff(x) * (y[-1] + y[-length(y)]) / 2)`. La red ER debería tener un R mayor que la BA bajo ataques.

Ejercicio propuesto 4

Realiza un análisis completo de robustez de una red *small-world* de 500 nodos: fallos aleatorios (10 realizaciones) y ataques dirigidos. Presenta los resultados con gráficos que incluyan bandas de variabilidad para los fallos y una sola curva para los ataques (deterministas). Incluye una tabla resumen con la fracción crítica de eliminación para cada escenario.

Autoevaluación: Los fallos aleatorios deberían mostrar una degradación gradual con baja varianza. Los ataques deberían degradar la red más rápido pero no tan abruptamente como en una red BA.

Ejercicio propuesto 5

Investiga: ¿Se puede hacer una red BA más robusta ante ataques sin cambiar el número de nodos ni aristas? Estrategia propuesta: después de generar una red BA de 300 nodos, “redistribuye” el 10% de las aristas de los *hubs* hacia nodos de bajo grado (elimina aristas de *hubs* y las reconecta aleatoriamente). Compara la robustez antes y después de la redistribución.

Autoevaluación: La redistribución debería reducir la vulnerabilidad a ataques (los *hubs* son menos críticos) a costa de aumentar ligeramente la distancia media. Es un *trade-off* entre eficiencia y robustez.

5.12 Resumen del capítulo

```

flowchart LR
    A[Eliminar nodos] --> B[Fallos aleatorios]
    A --> C[Ataques dirigidos]
    B --> D[BA robusta / ER se degrada]
    C --> E[BA colapsa / ER resiste]
    D --> F[Paradoja robustez-fragilidad]
    E --> F
    F --> G["Siguiente: Ejercicios integradores →"]
    style G fill:#e8f4f8,stroke:#3ABEF9
  
```

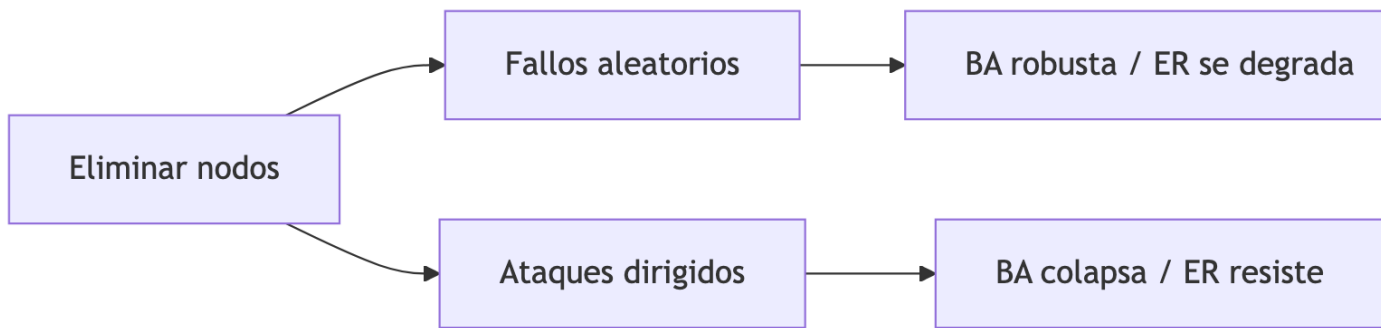


Figure 5.9: Resumen visual del capítulo

Table 5.4: Funciones y conceptos clave del capítulo

Función / Concepto	Descripción
<code>delete_vertices(g, v)</code>	Eliminar nodos de un grafo
<code>components(g)</code>	Identificar componentes conectados
<code>components(g)\$csize</code>	Tamaños de cada componente
Componente gigante	Componente conectado más grande
Fallo aleatorio	Eliminación aleatoria de nodos
Ataque dirigido	Eliminación del nodo con mayor grado (o <i>betweenness</i>)
Paradoja robustez-fragilidad	Redes <i>scale-free</i> : robustas a fallos, frágiles a ataques

Conexión con los ejercicios integradores: En el Chapter 9 pondrás en práctica todo lo aprendido: crearás redes, calcularás métricas, compararás topologías y evaluarás su robustez en escenarios completos que combinan los conceptos de todos los capítulos.

Autoevaluación

Pregunta 1

¿Qué tipo de red es más vulnerable ante ataques dirigidos a los *hubs*?

- a) Erdős–Rényi
- b) *Small-world*
- c) Barabási–Albert
- d) Anillo

Las redes BA dependen de sus *hubs* para mantener la conectividad. Eliminarlos fragmenta la red rápidamente.

Pregunta 2

Verdadero o Falso: Una red Barabási–Albert es más robusta que una Erdős–Rényi ante fallos aleatorios.

Verdadero. Como la mayoría de los nodos en una red BA tienen grado bajo, es poco probable que un fallo aleatorio afecte un *hub*. La red mantiene su conectividad.

Pregunta 3

¿Qué métrica es la más informativa para evaluar si una red sigue “funcionando” tras eliminar nodos?

- a) Número de aristas restantes
- b) Grado medio
- c) Tamaño relativo del componente gigante
- d) Número de triángulos

Si el componente gigante abarca la mayoría de los nodos, la red sigue conectada y funcional. Si se fragmenta en muchos componentes pequeños, la comunicación global se pierde.

Pregunta 4

¿Por qué es importante recalcular el grado después de cada eliminación en un ataque dirigido?

Porque al eliminar un *hub*, sus vecinos pierden conexiones y el **ranking de grado cambia**. El nodo que antes era el segundo más conectado puede no serlo después del primer ataque. Recalcular garantiza que siempre ataquemos al nodo actualmente más importante.

Pregunta 5

Verdadero o Falso: Agregar redundancia (más aristas) siempre mejora la robustez de una red.

Verdadero en general, pero con matices. Más aristas crean caminos alternativos, lo que dificulta la fragmentación. Sin embargo, si las aristas nuevas solo conectan *hubs* entre sí, la red sigue siendo vulnerable a ataques dirigidos. La **distribución** de las aristas importa tanto como su cantidad.

6 Medidas de centralidad

i Objetivos de aprendizaje

Al finalizar este capítulo serás capaz de:

1. **Definir** qué es la centralidad y por qué existen múltiples formas de medirla.
2. **Calcular e interpretar** las cuatro medidas clásicas: grado, cercanía, intermediación y vector propio.
3. **Aplicar** medidas avanzadas de centralidad disponibles en **igraph**: PageRank, HITS, centralidad armónica, de subgrafo, alpha y de Bonacich.
4. **Comparar** las diferentes medidas sobre una misma red y analizar sus correlaciones.
5. **Evaluar** la centralización de una red completa (nivel macro) y no solo de nodos individuales.

Pregunta motivadora: En una organización, ¿quién es la persona más “importante”? ¿La que tiene más contactos? ¿La que conecta departamentos? ¿La que está más cerca de todos? ¿O aquella cuyos contactos también son importantes? Cada respuesta lleva a una medida de centralidad diferente.

flowchart TD

```
A[Centralidad] --> B[Medidas clásicas]
A --> C[Medidas avanzadas]
A --> D[Centralización]
B --> B1[Grado]
B --> B2[Cercanía / Closeness]
B --> B3[Intermediación / Betweenness]
B --> B4[Vector propio / Eigenvector]
C --> C1[PageRank]
C --> C2[HITS: Hubs & Authorities]
C --> C3[Centralidad armónica]
C --> C4[Subgraph centrality]
C --> C5[Alpha & Bonacich Power]
D --> D1[centr_degree / centr_betw / centr_clo / centr_eigen]
```

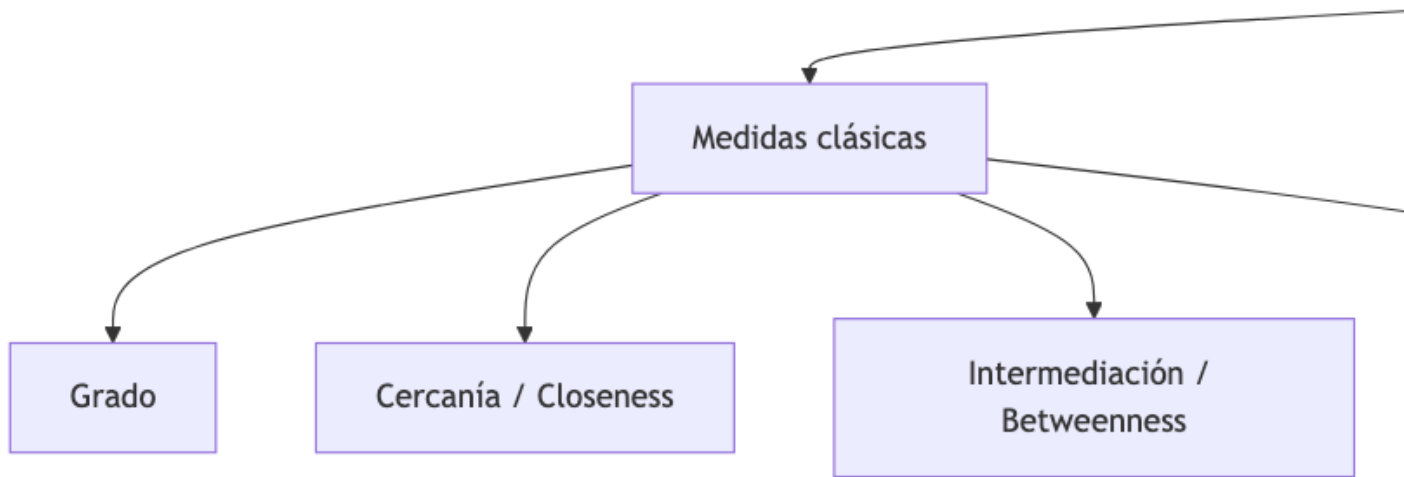


Figure 6.1: Mapa conceptual del capítulo

6.1 Preparación del entorno

```
library(igraph)
```

6.2 ¿Qué es la centralidad?

La **centralidad** es un concepto que asigna un valor numérico a cada nodo de una red según su “importancia”. Pero la importancia depende del criterio:

Table 6.1: Medidas de centralidad disponibles en igraph

Pregunta	Medida de centralidad	Función en igraph
¿Quién tiene más conexiones?	Grado (<i>degree</i>)	<code>degree()</code>
¿Quién está más cerca de todos?	Cercanía (<i>closeness</i>)	<code>closeness()</code>

Pregunta	Medida de centralidad	Función en igraph
¿Quién controla el flujo de información?	Intermediación (<i>betweenness</i>)	<code>betweenness()</code>
¿Quién está conectado a nodos importantes?	Vector propio (<i>eigenvector</i>)	<code>eigen_centrality()</code>
¿Quién recibe más “votos” de otros nodos?	PageRank	<code>page_rank()</code>
¿Quién es buen hub o buena autoridad?	HITS	<code>hub_score() / authority_score()</code>
¿Quién está cerca incluso en redes desconectadas?	Armónica (<i>harmonic</i>)	<code>harmonic_centrality()</code>
¿Quién participa en más subgrafos?	Subgrafo	<code>subgraph_centrality()</code>
¿Quién tiene influencia exógena + endógena?	Alpha	<code>alpha_centrality()</code>
¿Quién tiene poder según Bonacich?	Poder de Bonacich	<code>power_centrality()</code>
¿Quién tiene más peso en aristas?	Fuerza (<i>strength</i>)	<code>strength()</code>



Concepto clave

No existe una medida de centralidad “correcta” universal. Cada una captura un aspecto diferente de la importancia. La elección depende de la **pregunta de investigación** y del **contexto** de la red.

6.3 Red de ejemplo

Usaremos la red de Zachary’s Karate Club, un clásico del análisis de redes que representa las amistades entre 34 miembros de un club universitario de karate.

```

g <- make_graph("Zachary")
V(g)$name <- paste0("M", 1:vcount(g))

# Layout fijo para todas las visualizaciones
set.seed(42)
lay <- layout_with_fr(g)

plot(g, layout = lay,
     vertex.color = "skyblue",
     vertex.size = 12,
     vertex.label.cex = 0.6,
     edge.color = adjustcolor("gray50", 0.5),
     main = "Club de Karate de Zachary")

```

Club de Karate de Zachary

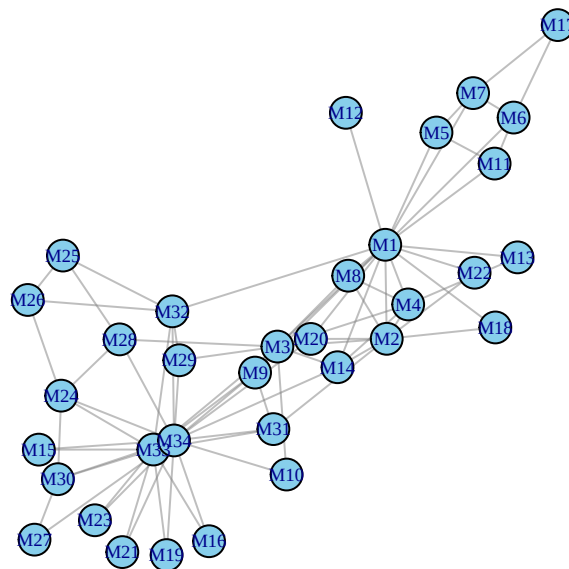


Figure 6.2: Red del Club de Karate de Zachary (34 nodos, 78 aristas)

```

cat("Nodos:", vcount(g), "\n")

```

Nodos: 34

```
cat("Aristas:", ecount(g), "\n")
```

Aristas: 78

```
cat("Densidad:", round(edge_density(g), 4), "\n")
```

Densidad: 0.139

```
cat("¿Conexa?:", is_connected(g), "\n")
```

¿Conexa?: TRUE

6.4 Las cuatro medidas clásicas

6.4.1 1. Centralidad de grado (*Degree Centrality*)

La más simple: cuenta cuántas conexiones tiene cada nodo.

$$C_D(v) = k_v = \text{número de aristas incidentes a } v$$

En su versión normalizada: $C_D^{\text{norm}}(v) = \frac{k_v}{n-1}$

```
deg <- degree(g)
deg_norm <- degree(g, normalized = TRUE)

# Top 5
top_deg <- sort(deg, decreasing = TRUE)[1:5]
knitr::kable(
  data.frame(Nodo = names(top_deg), Grado = top_deg,
             Normalizado = round(deg_norm[names(top_deg)], 3)),
  caption = "Top 5 nodos por centralidad de grado",
  row.names = FALSE
)
```

Table 6.2: Top 5 nodos por centralidad de grado

Nodo	Grado	Normalizado
M34	17	0.515
M1	16	0.485
M33	12	0.364
M3	10	0.303
M2	9	0.273

```
plot(g, layout = lay,
     vertex.size = deg * 1.5 + 3,
     vertex.color = "gold",
     vertex.label.cex = 0.5,
     edge.color = adjustcolor("gray50", 0.4),
     main = "Centralidad de grado")
```

Centralidad de grado

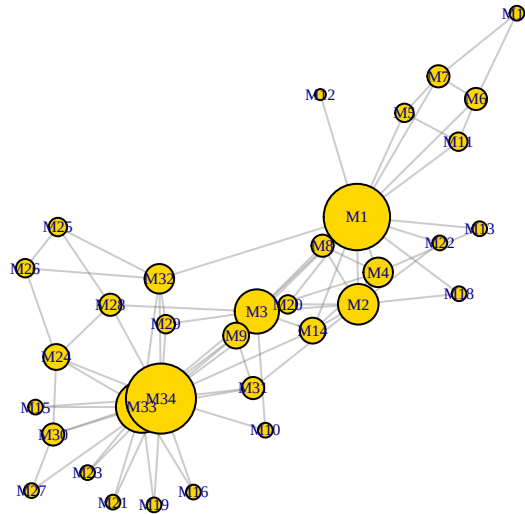


Figure 6.3: Red con nodos escalados por centralidad de grado

💡 Concepto clave

La centralidad de grado es una medida **local**: solo mira las conexiones inmediatas del nodo, sin considerar la estructura global de la red. Un nodo con muchos contactos aislados puede tener alto grado pero poca influencia real.

6.4.2 2. Centralidad de cercanía (*Closeness Centrality*)

Mide qué tan cerca está un nodo de todos los demás. Un nodo con alta cercanía puede alcanzar rápidamente a cualquier otro.

$$C_C(v) = \frac{1}{\sum_{u \neq v} d(v, u)}$$

En su versión normalizada: $C_C^{\text{norm}}(v) = \frac{n-1}{\sum_{u \neq v} d(v, u)}$

```
clo <- closeness(g, normalized = TRUE)

top_clo <- sort(clo, decreasing = TRUE)[1:5]
knitr::kable(
  data.frame(Nodo = names(top_clo), Closeness = round(top_clo, 4)),
  caption = "Top 5 nodos por centralidad de cercanía",
  row.names = FALSE
)
```

Table 6.3: Top 5 nodos por centralidad de cercanía

Nodo	Closeness
M1	0.5690
M3	0.5593
M34	0.5500
M32	0.5410
M9	0.5156

```
# Escalar para visualización
clo_scaled <- (clo - min(clo)) / (max(clo) - min(clo)) * 20 + 5

plot(g, layout = lay,
     vertex.size = clo_scaled,
```

```
vertex.color = "tomato",
vertex.label.cex = 0.5,
edge.color = adjustcolor("gray50", 0.4),
main = "Centralidad de cercanía (closeness)"
```

Centralidad de cercanía (closeness)

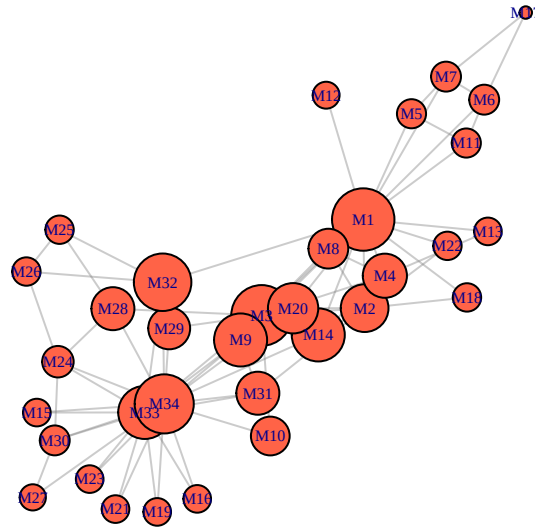


Figure 6.4: Red con nodos escalados por centralidad de cercanía

⚠ Error frecuente

La cercanía estándar **no funciona bien en redes desconectadas**, porque la distancia a nodos inalcanzables es infinita. En esos casos usa `harmonic_centrality()` (que veremos más adelante), que reemplaza las distancias infinitas por 0 en el cálculo.

6.4.3 3. Centralidad de intermediación (*Betweenness Centrality*)

Mide cuántos **camino más cortos** entre otros pares de nodos pasan por un nodo dado. Un nodo con alta intermediación actúa como “puente” o “cuello de botella”.

$$C_B(v) = \sum_{s \neq v \neq t} \frac{\sigma_{st}(v)}{\sigma_{st}}$$

donde σ_{st} es el número de caminos más cortos entre s y t , y $\sigma_{st}(v)$ es cuántos de esos caminos pasan por v .

```
btw <- betweenness(g, normalized = TRUE)

top_btw <- sort(btw, decreasing = TRUE)[1:5]
knitr::kable(
  data.frame(Nodo = names(top_btw), Betweenness = round(top_btw, 4)),
  caption = "Top 5 nodos por centralidad de intermediación",
  row.names = FALSE
)
```

Table 6.4: Top 5 nodos por centralidad de intermediación

Nodo	Betweenness
M1	0.4376
M34	0.3041
M33	0.1452
M3	0.1437
M32	0.1383

```
btw_raw <- betweenness(g)
btw_scaled <- (btw_raw - min(btw_raw)) / (max(btw_raw) - min(btw_raw)) * 20 + 5

plot(g, layout = lay,
     vertex.size = btw_scaled,
     vertex.color = "mediumpurple",
     vertex.label.cex = 0.5,
     edge.color = adjustcolor("gray50", 0.4),
     main = "Centralidad de intermediación (betweenness)")
```

Centralidad de intermediación (betweenness)

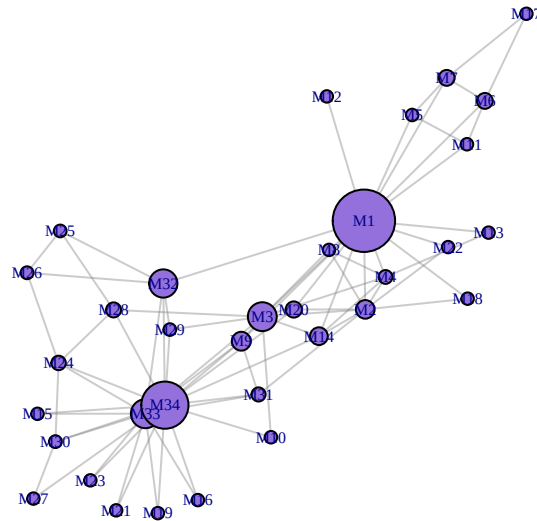


Figure 6.5: Red con nodos escalados por centralidad de intermediación



Concepto clave

La intermediación identifica a los **intermediarios** de la red: nodos que, si se eliminan, alargan los caminos entre otros nodos o incluso desconectan partes de la red. Es especialmente útil para encontrar puentes entre comunidades.

6.4.3.1 Intermediación de aristas

`igraph` también permite calcular la intermediación de **aristas** (no solo nodos):

```
ebtw <- edge_betweenness(g)

# Top 5 aristas más "puente"
top_edges <- order(ebtw, decreasing = TRUE)[1:5]

knitr::kable(
  data.frame(
    Arista = paste(ends(g, top_edges)[,1], "-", ends(g, top_edges)[,2]),
    Betweenness = round(ebtw[top_edges], 2)
```

```
),
caption = "Top 5 aristas por intermediación",
row.names = FALSE
)
```

Table 6.5: Top 5 aristas por intermediación

Arista	Betweenness
M1 — M32	71.39
M1 — M7	43.83
M1 — M6	43.83
M1 — M3	43.64
M1 — M9	41.65

6.4.4 4. Centralidad de vector propio (*Eigenvector Centrality*)

Un nodo es importante si está conectado a **otros nodos importantes**. Es una definición recursiva que se resuelve calculando el vector propio dominante de la matriz de adyacencia.

$$C_E(v) = \frac{1}{\lambda} \sum_{u \in N(v)} C_E(u)$$

donde λ es el mayor valor propio de la matriz de adyacencia.

```
eig <- eigen_centrality(g)$vector

top_eig <- sort(eig, decreasing = TRUE)[1:5]
knitr::kable(
  data.frame(Nodo = names(top_eig), Eigenvector = round(top_eig, 4)),
  caption = "Top 5 nodos por centralidad de vector propio",
  row.names = FALSE
)
```

Table 6.6: Top 5 nodos por centralidad de vector propio

Nodo	Eigenvector
M34	1.0000
M1	0.9521
M3	0.8496

Nodo	Eigenvector
M33	0.8267
M2	0.7123

```
eig_scaled <- eig * 20 + 5

plot(g, layout = lay,
     vertex.size = eig_scaled,
     vertex.color = "steelblue",
     vertex.label.cex = 0.5,
     edge.color = adjustcolor("gray50", 0.4),
     main = "Centralidad de vector propio (eigenvector)")
```

Centralidad de vector propio (eigenvector)

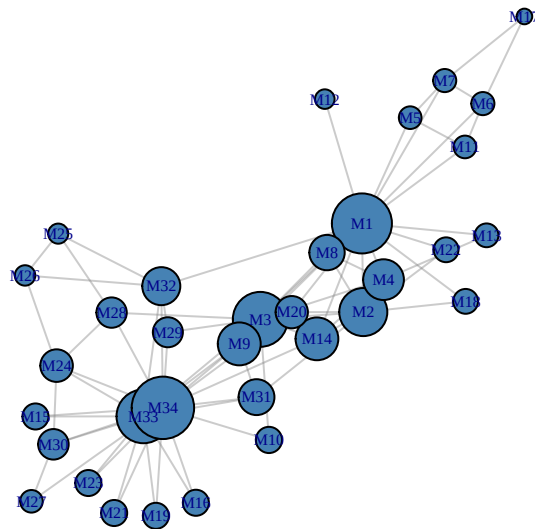


Figure 6.6: Red con nodos escalados por centralidad de vector propio

! Recuerda

La diferencia clave entre grado y vector propio: el **grado** cuenta *cuántas* conexiones tienes; el **vector propio** pondera *con quién* estás conectado. Un nodo con pocos amigos pero muy bien conectados puede tener un eigenvector alto y un grado bajo.

6.5 Comparación visual: las 4 clásicas

```
colores <- c("gold", "tomato", "mediumpurple", "steelblue")
titulos <- c("Grado", "Cercanía", "Intermediación", "Vector propio")

# Normalizar todas las medidas a [0, 1] para escalar visualmente
medidas <- list(
  degree(g, normalized = TRUE),
  closeness(g, normalized = TRUE),
  betweenness(g, normalized = TRUE),
  eigen_centrality(g)$vector
)

par(mfrow = c(2, 2), mar = c(1, 1, 3, 1))

for (i in 1:4) {
  m <- medidas[[i]]
  m_scaled <- m / max(m) * 20 + 5

  plot(g, layout = lay,
       vertex.size = m_scaled,
       vertex.color = colores[i],
       vertex.label.cex = 0.45,
       edge.color = adjustcolor("gray50", 0.3),
       main = titulos[i])
}
```

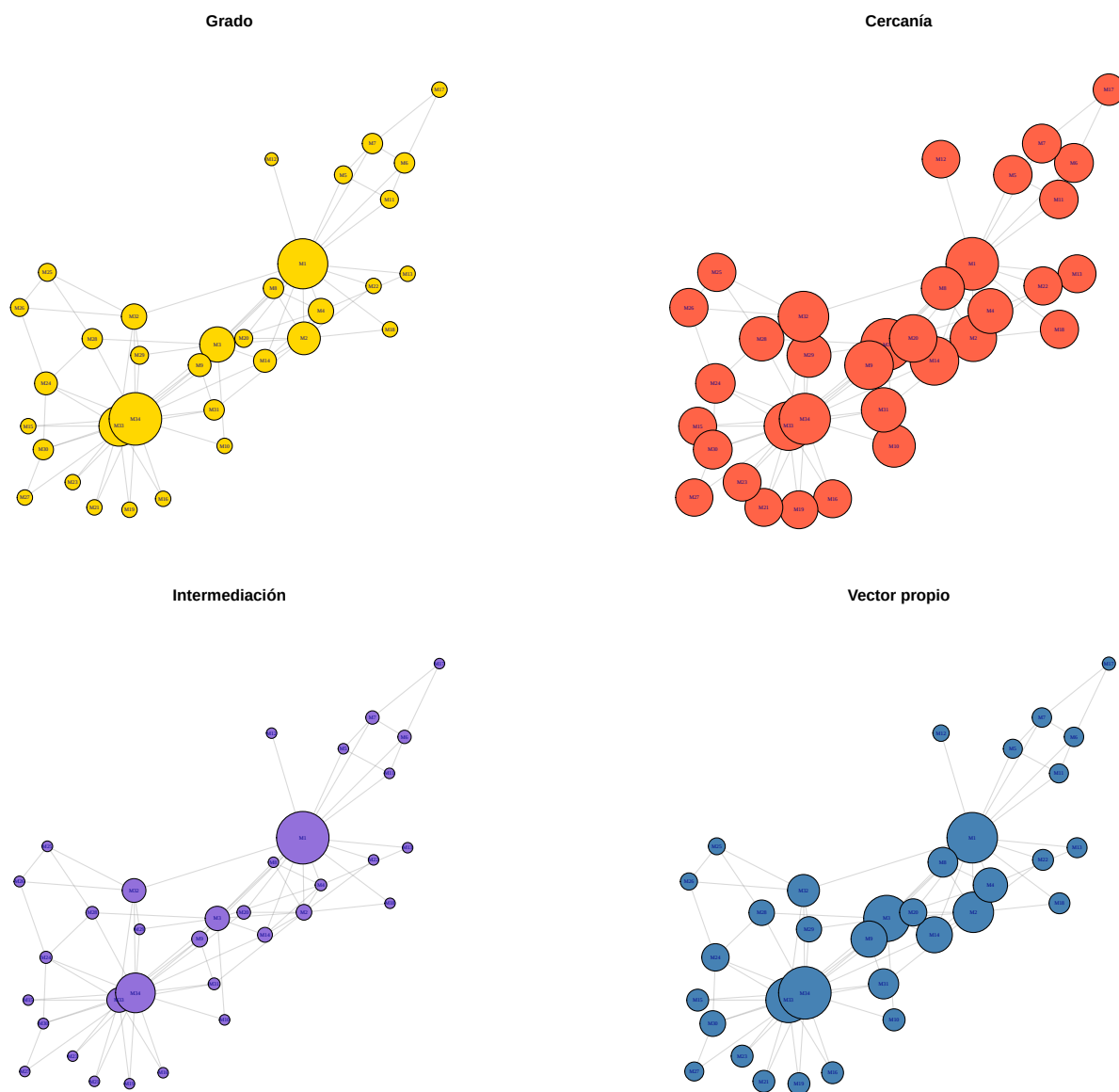


Figure 6.7: Comparación visual de las cuatro medidas clásicas de centralidad

6.6 Medidas avanzadas de centralidad

6.6.1 5. PageRank

Desarrollado por Google para clasificar páginas web. Similar al eigenvector, pero incluye un factor de “teletransportación” (damping) que evita que los nodos aislados tengan centralidad cero.

$$PR(v) = \frac{1-d}{n} + d \sum_{u \in N_{\text{in}}(v)} \frac{PR(u)}{k_{\text{out}}(u)}$$

donde d es el factor de amortiguamiento (por defecto 0.85).

```
pr <- page_rank(g, damping = 0.85)$vector

top_pr <- sort(pr, decreasing = TRUE)[1:5]
knitr::kable(
  data.frame(Nodo = names(top_pr), PageRank = round(top_pr, 4)),
  caption = "Top 5 nodos por PageRank",
  row.names = FALSE
)
```

Table 6.7: Top 5 nodos por PageRank

Nodo	PageRank
M34	0.1009
M1	0.0970
M33	0.0717
M3	0.0571
M2	0.0529



Concepto clave

La diferencia entre eigenvector y PageRank: el eigenvector puede fallar en redes dirigidas con nodos “sumidero” (sin aristas de salida). El PageRank resuelve esto con el factor de amortiguamiento d : con probabilidad $1 - d$, un “caminante aleatorio” salta a un nodo al azar en vez de seguir una arista.

6.6.2 6. HITS: Hubs y Authorities

El algoritmo HITS (*Hyperlink-Induced Topic Search*) asigna **dos** valores a cada nodo:

- **Authority score:** un nodo es buena autoridad si muchos buenos *hubs* apuntan a él.
- **Hub score:** un nodo es buen *hub* si apunta a muchas buenas autoridades.

```
hub <- hub_score(g)$vector
auth <- authority_score(g)$vector

hits_df <- data.frame(
  Nodo = V(g)$name,
  Hub = round(hub, 4),
  Authority = round(auth, 4)
)

# Top 5 por cada score
knitr::kable(
  head(hits_df[order(hits_df$Hub, decreasing = TRUE), ], 5),
  caption = "Top 5 nodos por Hub score",
  row.names = FALSE
)
```

Table 6.8: Top 5 nodos por Hub score

Nodo	Hub	Authority
M34	1.0000	1.0000
M1	0.9521	0.9521
M3	0.8496	0.8496
M33	0.8267	0.8267
M2	0.7123	0.7123

```
knitr::kable(
  head(hits_df[order(hits_df$Authority, decreasing = TRUE), ], 5),
  caption = "Top 5 nodos por Authority score",
  row.names = FALSE
)
```

Table 6.9: Top 5 nodos por Authority score

Nodo	Hub	Authority
M34	1.0000	1.0000
M1	0.9521	0.9521
M3	0.8496	0.8496
M33	0.8267	0.8267
M2	0.7123	0.7123

⚠ Error frecuente

En redes **no dirigidas**, los *hub scores* y *authority scores* son idénticos y equivalentes al eigenvector centrality. La distinción hub/authority solo tiene sentido en redes **dirigidas** (como la web: páginas que enlazan vs. páginas enlazadas).

6.6.3 7. Centralidad armónica (*Harmonic Centrality*)

Una alternativa a la cercanía que **funciona en redes desconectadas**. Usa el inverso de cada distancia individualmente:

$$C_H(v) = \sum_{u \neq v} \frac{1}{d(v, u)}$$

Si $d(v, u) = \infty$ (nodos en diferentes componentes), su contribución es simplemente 0.

```
harm <- harmonic_centrality(g, normalized = TRUE)

top_harm <- sort(harm, decreasing = TRUE)[1:5]
knitr::kable(
  data.frame(Nodo = names(top_harm), Harmonic = round(top_harm, 4)),
  caption = "Top 5 nodos por centralidad armónica",
  row.names = FALSE
)
```

Table 6.10: Top 5 nodos por centralidad armónica

Nodo	Harmonic
M34	0.7045
M1	0.7020

Nodo	Harmonic
M3	0.6364
M33	0.6338
M32	0.5859



Concepto clave

Usa `harmonic centrality()` en lugar de `closeness()` siempre que trabajes con redes que **podrían tener componentes desconectados**. Es matemáticamente más robusta y no produce valores indefinidos.

6.6.4 8. Centralidad de subgrafo (*Subgraph Centrality*)

Mide en cuántos subgrafos cerrados (ciclos, triángulos, etc.) participa un nodo, ponderando por el tamaño del subgrafo. Los subgrafos más grandes reciben menos peso.

$$SC(v) = \sum_{k=0}^{\infty} \frac{(\mathbf{A}^k)_{vv}}{k!}$$

donde $\mathbf{A}$ es la matriz de adyacencia. Esto equivale a la diagonal de $e^{\mathbf{A}}$.

```
sc <- subgraph_centrality(g)

# Asignar nombres
names(sc) <- V(g)$name

top_sc <- sort(sc, decreasing = TRUE)[1:5]
knitr::kable(
  data.frame(Nodo = names(top_sc), Subgraph = round(top_sc, 2)),
  caption = "Top 5 nodos por centralidad de subgrafo",
  row.names = FALSE
)
```

Table 6.11: Top 5 nodos por centralidad de subgrafo

Nodo	Subgraph
M34	136.72
M1	128.10
M33	95.69

Nodo	Subgraph
M3	88.70
M2	71.43

⚠ Error frecuente

`subgraph centrality()` calcula todos los valores y vectores propios de la matriz de adyacencia, lo que la hace **muy costosa computacionalmente** para redes grandes (más de ~1000 nodos). Úsala solo en redes pequeñas o medianas.

6.6.5 9. Alpha Centrality

Generaliza la centralidad de eigenvector incorporando una fuente de influencia **exógena** (externa a la red). Cada nodo recibe una influencia base **exo** más la influencia de sus vecinos ponderada por **alpha**.

$$\mathbf{x} = \alpha \mathbf{A}^T \mathbf{x} + \mathbf{e}$$

donde **e** es el vector de influencia exógena.

```
# alpha debe ser menor que 1/valor_propio_max para convergencia
lambda_max <- max(Re(eigen(as_adjacency_matrix(g, sparse = FALSE))$values))
alpha_seguro <- 1 / (lambda_max + 1)

ac <- alpha centrality(g, alpha = alpha_seguro, exo = 1)
names(ac) <- V(g)$name

top_ac <- sort(ac, decreasing = TRUE)[1:5]
knitr::kable(
  data.frame(Nodo = names(top_ac), Alpha = round(top_ac, 4)),
  caption = "Top 5 nodos por alpha centrality",
  row.names = FALSE
)
```

Table 6.12: Top 5 nodos por alpha centrality

Nodo	Alpha
M34	13.8889
M1	13.3480

Nodo	Alpha
M33	11.4642
M3	11.4312
M2	9.7812

6.6.6 10. Centralidad de poder de Bonacich (*Power Centrality*)

Similar a la alpha centrality pero con una interpretación diferente del parámetro **exponent** (β). Cuando $\beta > 0$, estar conectado a nodos bien conectados es ventajoso; cuando $\beta < 0$, estar conectado a nodos **poco** conectados da más poder (modelo de negociación).

```
# El exponente beta debe ser menor que 1/valor_propio_max en valor absoluto
# para que la matriz (I - beta*A) sea invertible
lambda_max_bp <- max(Re(eigen(as_adjacency_matrix(g, sparse = FALSE))$values))
beta_max <- 1 / lambda_max_bp

cat("Valor propio máximo:", round(lambda_max_bp, 3), "\n")
```

Valor propio máximo: 6.726

```
cat("Rango seguro de : (", round(-beta_max, 4), ",", round(beta_max, 4), ")\n\n")
```

Rango seguro de : (-0.1487 , 0.1487)

```
# Exponente positivo (dentro del rango seguro)
beta_pos <- beta_max * 0.5
pc_pos <- power_centrality(g, exponent = beta_pos)
names(pc_pos) <- V(g)$name

# Exponente negativo (dentro del rango seguro)
beta_neg <- -beta_max * 0.5
pc_neg <- power_centrality(g, exponent = beta_neg)
names(pc_neg) <- V(g)$name

knitr::kable(
  data.frame(
    Nodo = V(g)$name[order(pc_pos, decreasing = TRUE)[1:5]],
    "Bonacich_pos" = round(sort(pc_pos, decreasing = TRUE)[1:5], 4),
```

```

    Nodo_neg = V(g)$name[order(pc_neg, decreasing = TRUE)[1:5]],
    "Bonacich_neg" = round(sort(pc_neg, decreasing = TRUE)[1:5], 4)
  ),
  caption = paste0("Top 5 nodos: Bonacich con  = ", round(beta_pos, 3),
    " vs.  = ", round(beta_neg, 3)),
  row.names = FALSE,
  col.names = c("Nodo ( > 0)", "Power ( > 0)", "Nodo ( < 0)", "Power ( < 0)")
)

```

Table 6.13: Top 5 nodos: Bonacich con $\beta = 0.074$ vs. $\beta = -0.074$

Nodo (> 0)	Power (> 0)	Nodo (< 0)	Power (< 0)
M34	2.4581	M34	3.3745
M1	2.3640	M1	3.0078
M33	1.8894	M33	2.1464
M3	1.7446	M3	1.5538
M2	1.4981	M2	1.5212

Error frecuente

`power_centrality()` requiere que $|\beta| < 1/\lambda_{\max}$, donde $\lambda_{\max}$ es el mayor valor propio de la matriz de adyacencia. Si usas un β fuera de ese rango, la función falla con un error de “matriz singular”. Siempre calcula el rango seguro antes de usarla.

6.6.7 11. Fuerza (*Strength*) — Centralidad para redes ponderadas

Si las aristas tienen peso, `strength()` suma los pesos de las aristas incidentes a cada nodo, generalizando el grado:

$$s(v) = \sum_{u \in N(v)} w(v, u)$$

```

# Crear una copia con pesos aleatorios para demostración
set.seed(42)
g_w <- g
E(g_w)$weight <- sample(1:10, ecount(g_w), replace = TRUE)

str_val <- strength(g_w)

```

```

top_str <- sort(str_val, decreasing = TRUE)[1:5]
knitr::kable(
  data.frame(
    Nodo = names(top_str),
    Grado = degree(g_w)[names(top_str)],
    Fuerza = top_str
  ),
  caption = "Top 5 nodos por fuerza (strength) - red con pesos aleatorios",
  row.names = FALSE
)

```

Table 6.14: Top 5 nodos por fuerza (strength) — red con pesos aleatorios

Nodo	Grado	Fuerza
M1	16	90
M34	17	88
M33	12	57
M3	10	55
M2	9	42

6.7 Tabla comparativa completa

Calculemos **todas** las medidas sobre la red de Zachary y comparemos los rankings:

```

# Recalcular alpha con parámetro seguro
lambda_max <- max(Re(eigen(as_adjacency_matrix(g, sparse = FALSE))$values))
alpha_seguro <- 1 / (lambda_max + 1)

centralidades <- data.frame(
  Nodo = V(g)$name,
  Grado = degree(g, normalized = TRUE),
  Cercanía = closeness(g, normalized = TRUE),
  Intermediación = betweenness(g, normalized = TRUE),
  Eigenvector = eigen_centrality(g)$vector,
  PageRank = page_rank(g)$vector,
  Armónica = harmonic_centrality(g, normalized = TRUE),
  Subgrafo = subgraph_centrality(g),

```



```

Hub = hub_score(g)$vector,
Alpha = alpha centrality(g, alpha = alpha_seguro, exo = 1)
)

# Mostrar top 10 por grado
top10 <- order(centralidades$Grado, decreasing = TRUE)[1:10]

tabla_top10 <- centralidades[top10, ]
tabla_top10[, -1] <- round(tabla_top10[, -1], 4)

knitr::kable(
  tabla_top10,
  caption = "Medidas de centralidad para los 10 nodos con mayor grado",
  row.names = FALSE
)

```

Table 6.15: Medidas de centralidad para los 10 nodos con mayor grado

Nodo	Grado	Cer- canía	Interme- diación	Eigenvec- tor	PageR- ank	Ar- mónica	Sub- grafo	Hub	Alpha
M34	0.5152	0.5500	0.3041	1.0000	0.1009	0.7045	136.7223	1.0000	13.8889
M1	0.4848	0.5690	0.4376	0.9521	0.0970	0.7020	128.0950	0.9521	13.3480
M33	0.3636	0.5156	0.1452	0.8267	0.0717	0.6338	95.6947	0.8267	11.4642
M3	0.3030	0.5593	0.1437	0.8496	0.0571	0.6364	88.7046	0.8496	11.4312
M2	0.2727	0.4853	0.0539	0.7123	0.0529	0.5808	71.4310	0.7123	9.7812
M4	0.1818	0.4648	0.0119	0.5656	0.0359	0.5354	48.1806	0.5656	7.8718
M32	0.1818	0.5410	0.1383	0.5117	0.0372	0.5859	34.8494	0.5117	7.4975
M9	0.1515	0.5156	0.0559	0.6091	0.0298	0.5606	45.0675	0.6091	8.3469
M14	0.1515	0.5156	0.0459	0.6066	0.0295	0.5606	46.7691	0.6066	8.2901
M24	0.1515	0.3929	0.0176	0.4021	0.0315	0.4859	27.4029	0.4021	6.1142

6.8 Correlación entre medidas

¿Qué tan de acuerdo están las diferentes medidas? Calculemos la correlación:

```

# Matriz de correlación (solo columnas numéricas)
cor_mat <- cor(centralidades[, -1], use = "complete.obs")

# Visualizar como heatmap

```

```
heatmap(cor_mat,  
        col = hcl.colors(20, palette = "RdBu", rev = TRUE),  
        scale = "none",  
        margins = c(10, 10),  
        main = "Correlación entre medidas de centralidad")
```

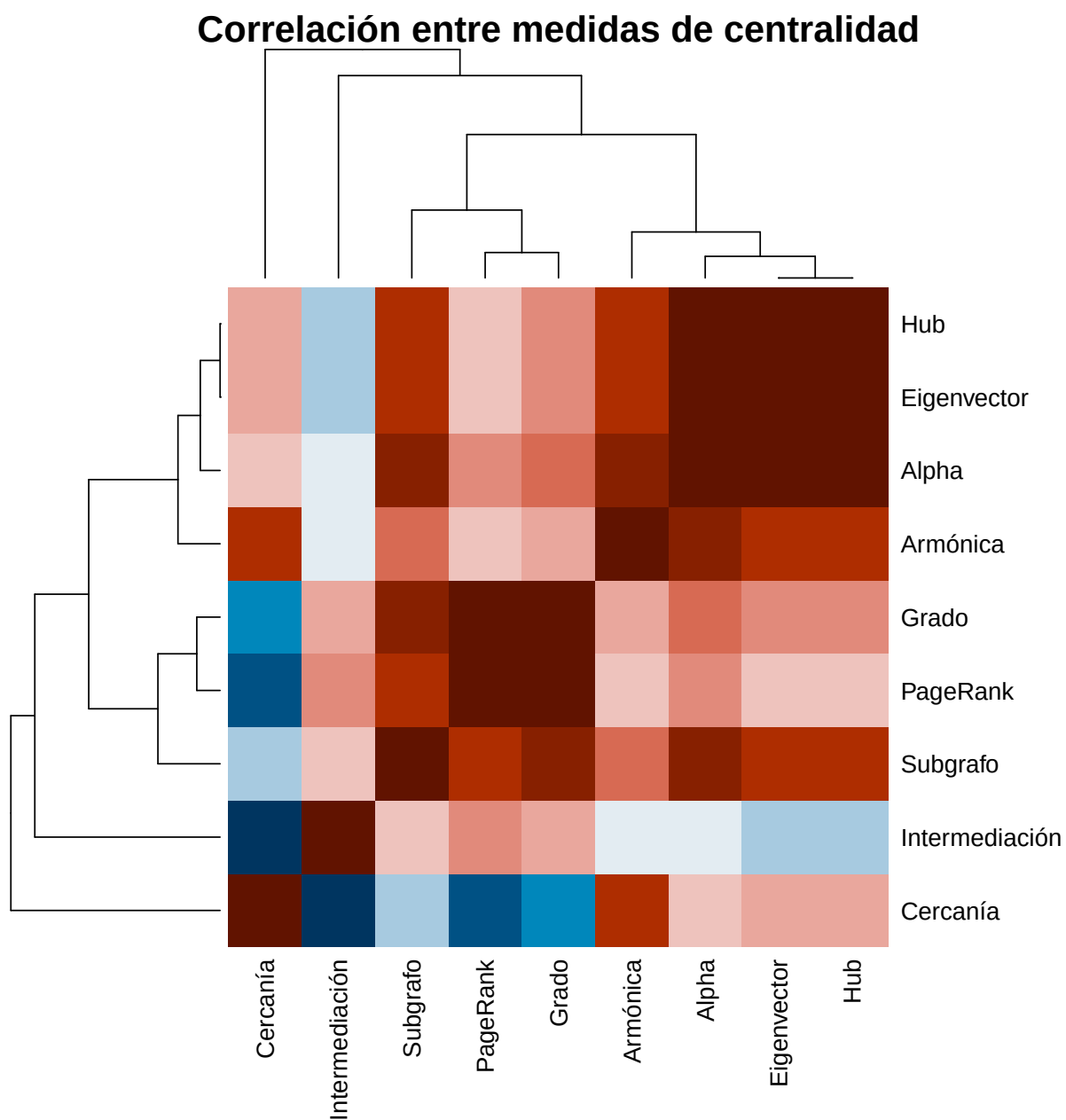


Figure 6.8: Matriz de correlación entre medidas de centralidad

```
knitr::kable(
  round(cor_mat, 3),
  caption = "Matriz de correlación entre medidas de centralidad"
)
```

Table 6.16: Matriz de correlación entre medidas de centralidad

	Grado	Cer- canía	Interme- diación	Eigen- vector	PageR- ank	Ar- mónica	Sub- grafo	Hub	Al- pha
Grado	1.000	0.772	0.915	0.917	0.998	0.912	0.981	0.917	0.940
Cercanía	0.772	1.000	0.718	0.905	0.742	0.959	0.812	0.905	0.897
Interme- diación	0.915	0.718	1.000	0.803	0.923	0.832	0.892	0.803	0.832
Eigenvec- tor	0.917	0.905	0.803	1.000	0.892	0.969	0.963	1.000	0.998
PageRank	0.998	0.742	0.923	0.892	1.000	0.892	0.969	0.892	0.919
Armónica	0.912	0.959	0.832	0.969	0.892	1.000	0.930	0.969	0.973
Subgrafo	0.981	0.812	0.892	0.963	0.969	0.930	1.000	0.963	0.976
Hub	0.917	0.905	0.803	1.000	0.892	0.969	0.963	1.000	0.998
Alpha	0.940	0.897	0.832	0.998	0.919	0.973	0.976	0.998	1.000



Concepto clave

Medidas como grado, eigenvector, hub y PageRank suelen estar **altamente correlacionadas** porque todas se basan en la cantidad o calidad de conexiones directas. La intermediación (*betweenness*) suele ser la más **diferente**, porque mide posición estructural (ser “puente”) en vez de popularidad.

6.9 Centralización: del nodo a la red

Las medidas anteriores dan un valor **por nodo**. La **centralización** resume cuánto se concentra la centralidad en unos pocos nodos a nivel de **toda la red**.

$$C_X = \frac{\sum_v [C_X^* - C_X(v)]}{\max \sum_v [C_X^* - C_X(v)]}$$

donde C_X^* es el valor máximo observado. Un valor cercano a 1 indica que la red está dominada por uno o pocos nodos; cercano a 0 indica distribución uniforme.

```
centraliz <- data.frame(
  Medida = c("Grado", "Cercanía", "Intermediación", "Eigenvector"),
  Centralización = round(c(
    centr_degree(g)$centralization,
    centr_clo(g, mode = "all")$centralization,
```

```

    centr_betw(g)$centralization,
    centr_eigen(g, directed = FALSE)$centralization
  ), 4)
)

knitr::kable(centraliz,
              caption = "Centralización de la red de Zachary por cada medida")

```

Table 6.17: Centralización de la red de Zachary por cada medida

Medida	Centralización
Grado	0.3761
Cercanía	0.2982
Intermediación	0.4056
Eigenvector	0.6458

```

barplot(centraliz$Centralización,
        names.arg = centraliz$Medida,
        col = c("gold", "tomato", "mediumpurple", "steelblue"),
        main = "Centralización de la red",
        ylab = "Índice de centralización",
        ylim = c(0, 0.6),
        las = 1)
text(x = c(0.7, 1.9, 3.1, 4.3),
     y = centraliz$Centralización + 0.02,
     labels = centraliz$Centralización,
     cex = 0.9, font = 2)

```

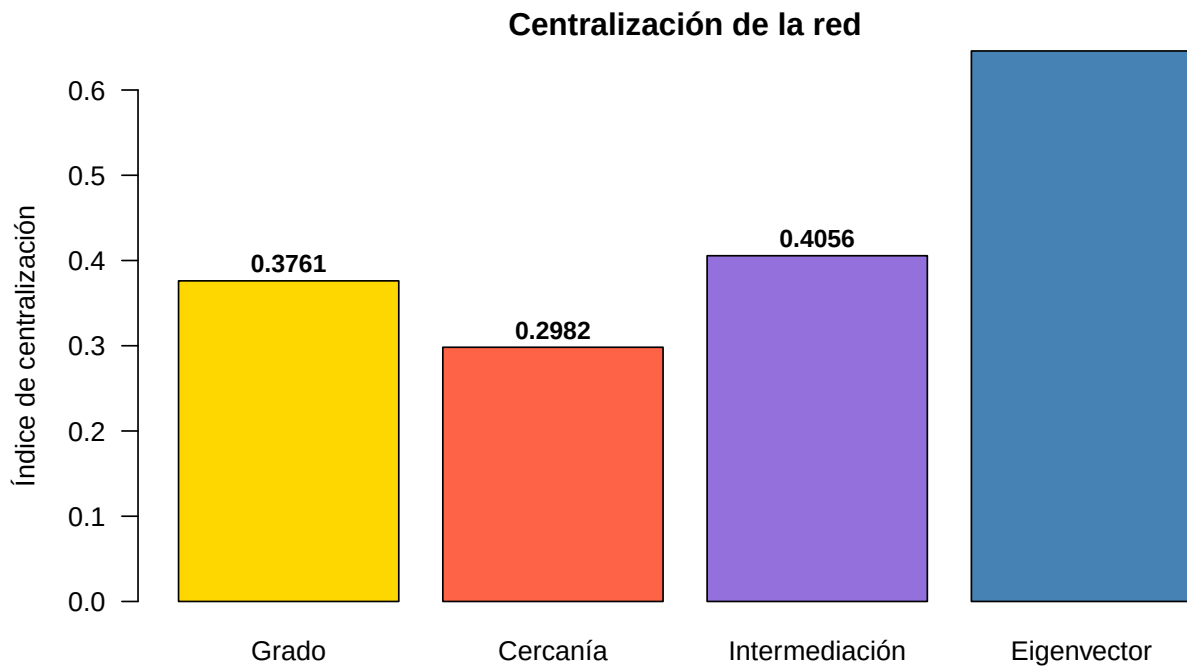


Figure 6.9: Centralización de la red de Zachary por tipo de medida

6.9.1 Comparación con redes de referencia

Para contextualizar, comparemos la centralización de la red de Zachary con redes extremas:

```
redes_ref <- list(
  "Zachary" = g,
  "Estrella" = make_star(34, mode = "undirected"),
  "Anillo" = make_ring(34),
  "Completa" = make_full_graph(34),
  "BA" = sample_pa(34, directed = FALSE)
)

centr_ref <- data.frame(
  Red = names(redes_ref),
  C_grado = round(sapply(redes_ref, function(x) centr_degree(x)$centralization), 4),
  C_betw = round(sapply(redes_ref, function(x) centr_betw(x)$centralization), 4),
  C_eigen = round(sapply(redes_ref, function(x)
    centr_eigen(x, directed = FALSE)$centralization), 4)
)
```

```
knitr::kable(centr_ref,
  caption = "Centralización comparada: Zachary vs. redes de referencia",
  col.names = c("Red", "C. grado", "C. betweenness", "C. eigenvector"))
```

Table 6.18: Centralización comparada: Zachary vs. redes de referencia

	Red	C. grado	C. betweenness	C. eigenvector
Zachary	Zachary	0.3761	0.4056	0.6458
Estrella	Estrella	0.9412	1.0000	0.8517
Anillo	Anillo	0.0000	0.0000	0.0000
Completa	Completa	0.0000	0.0000	0.0000
BA	BA	0.1836	0.7017	0.7661

! Recuerda

La red **estrella** tiene centralización máxima (un solo nodo domina). El **anillo** y la red **completa** tienen centralización mínima (todos los nodos son iguales). Las redes reales caen en algún punto intermedio.

6.10 Ejercicios

Ejercicios resueltos

Ejercicio resuelto 1: Crea un *data frame* con las cuatro centralidades clásicas (grado, cercanía, intermediación, eigenvector) para la red de Zachary. Identifica el nodo que ocupa el top 3 en **todas** las medidas simultáneamente.

💡 Solución

Paso 1: Calcular rankings por cada medida.

```

rankings <- data.frame(
  Nodo = V(g)$name,
  Rank_grado = rank(-degree(g)),
  Rank_closeness = rank(-closeness(g)),
  Rank_betweenness = rank(-betweenness(g)),
  Rank_eigen = rank(-eigen_centrality(g)$vector)
)

# Nodos en top 3 en TODAS las medidas
rankings$Mejor_rank <- pmax(rankings$Rank_grado, rankings$Rank_closeness,
                           rankings$Rank_betweenness, rankings$Rank_eigen)

top_global <- rankings[rankings$Mejor_rank <= 3, ]

knitr::kable(top_global,
              caption = "Nodos en el top 3 de todas las medidas simultáneamente",
              row.names = FALSE)

```

Table 6.19: Nodos en el top 3 de todas las medidas simultáneamente

Nodo	Rank_grado	Rank_closeness	Rank_betweenness	Rank_eigen	Mejor_rank
M1	2	1	1	2	2
M34	1	3	2	1	3

Interpretación: Solo uno o dos nodos suelen estar en el top 3 de las cuatro medidas simultáneamente. M1 y M34 (los líderes del club en la historia real) suelen dominar en grado y eigenvector, pero M1 también destaca en intermediación porque conecta diferentes subgrupos.

Ejercicio resuelto 2: Genera una red BA de 200 nodos y compara grado vs. betweenness en un gráfico de dispersión. ¿Los *hubs* siempre tienen alta intermediación?

💡 Solución

```
set.seed(42)
g_ba <- sample_pa(200, m = 2, directed = FALSE)

deg_ba <- degree(g_ba)
btw_ba <- betweenness(g_ba, normalized = TRUE)

plot(deg_ba, btw_ba,
     pch = 19, cex = 0.8,
     col = adjustcolor("steelblue", 0.6),
     xlab = "Grado", ylab = "Betweenness (normalizado)",
     main = "Grado vs. Betweenness - Red BA (n=200)")

# Resaltar hubs
top5 <- order(deg_ba, decreasing = TRUE)[1:5]
points(deg_ba[top5], btw_ba[top5], pch = 19, cex = 1.5, col = "red")
text(deg_ba[top5], btw_ba[top5], labels = top5, pos = 4, cex = 0.7, col = "red")

# Correlación
legend("topleft",
     legend = paste("r =", round(cor(deg_ba, btw_ba), 3)),
     bty = "n", cex = 1.1)
```

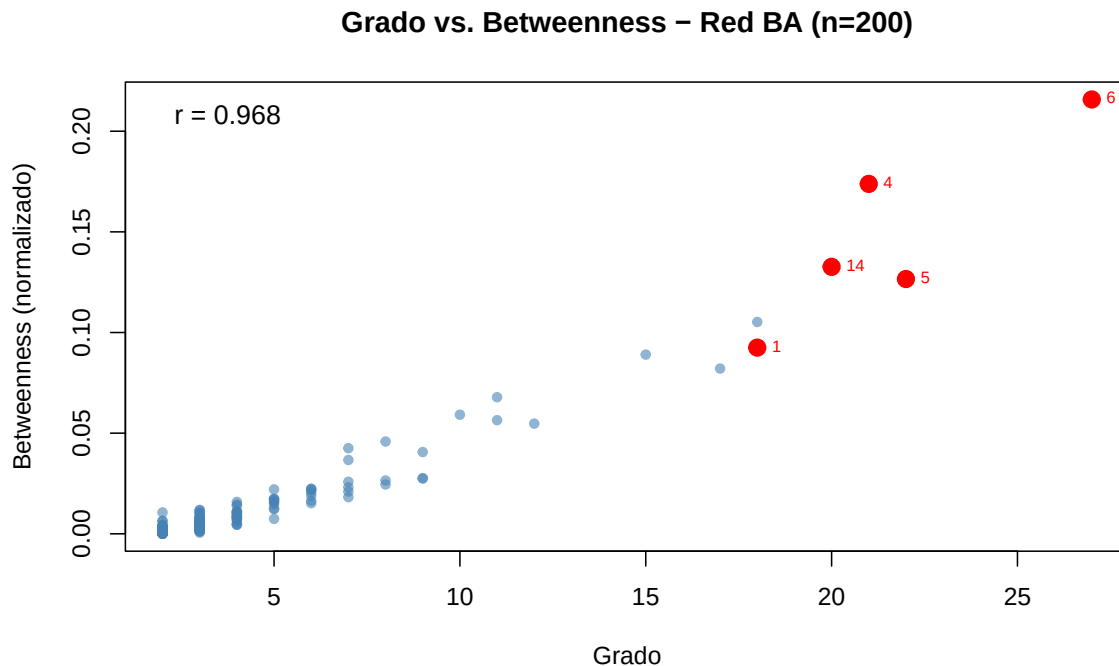


Figure 6.10: Grado vs. betweenness en una red BA de 200 nodos

Interpretación: La correlación es positiva pero **no perfecta**. Los *hubs* suelen tener alta intermediación porque muchos caminos pasan por ellos, pero un nodo de grado moderado ubicado en una posición “puente” puede tener betweenness más alta que un hub periférico. Grado y betweenness capturan aspectos diferentes de la importancia.

Ejercicios con pistas

Ejercicio 1: Crea una red dirigida simple de 8 nodos (por ejemplo, una cadena alimentaria o un flujo de información). Calcula los scores HITS (hub y authority). ¿Los mejores hubs son también las mejores autoridades? ¿Por qué tiene sentido que sean distintos en una red dirigida?

Pista 1

Usa `graph_from_literal()` con `+` para aristas dirigidas. Por ejemplo:
`graph_from_literal(A -> B, B -> C, A -> C).`

i Pista 2

En una cadena alimentaria dirigida, los depredadores (que “apuntan a” presas) deberían tener alto hub score, mientras que las presas (que “reciben” flechas) deberían tener alto authority score.

i Pista 3 — Pseudocódigo

```
1. g <- graph_from_literal(Sol -> Planta, Planta -> Insecto,  
2.                               Insecto -> Rana, Rana -> Serpiente, ...)  
3. hub_score(g)$vector  
4. authority_score(g)$vector  
5. # Comparar lado a lado en un data.frame
```

Ejercicio 2: Compara `closeness()` con `harmonic centrality()` en una red que **no sea conexas**. Genera una red ER de 100 nodos con $p = 0.01$ (probablemente no sea conexas). ¿Cuál medida produce resultados más informativos?

i Pista 1

Verifica si es conexas con `is_connected(g)`. Si no lo es, `closeness()` puede dar NaN para algunos nodos, mientras que `harmonic centrality()` siempre da un valor finito.

i Pista 2

Compara cuántos NaN produce cada medida: `sum(is.nan(closeness(g)))` vs. `sum(is.nan(harmonic centrality(g)))`.

i Pista 3 — Pseudocódigo

```
1. set.seed(42)  
2. g <- sample_gnp(100, 0.01)  
3. is_connected(g)  
4. components(g)$no  
5. clo <- closeness(g)  
6. harm <- harmonic centrality(g)  
7. cat("NaN en closeness:", sum(is.nan(clo)))  
8. cat("NaN en harmonic:", sum(is.nan(harm)))  
9. # Graficar ambas
```

Ejercicio 3: Calcula la centralización de grado para redes BA de tamaño creciente ($n = 50, 100, 200, 500, 1000$). ¿La centralización aumenta o disminuye con el tamaño? ¿Por qué?

i Pista 1

Usa `centr_degree(g)$centralization` para cada red. Guarda los resultados en un vector y gráficelos.

i Pista 2

En redes BA, la centralización de grado tiende a **disminuir** con n porque el hub más grande crece como $\sqrt{n}$ pero el denominador (máximo teórico) crece como n . El cociente tiende a 0.

i Pista 3 — Pseudocódigo

```
1. tamanos <- c(50, 100, 200, 500, 1000)
2. set.seed(42)
3. centr_vals <- sapply(tamanos, function(n) {
4.   g <- sample_pa(n, m = 2, directed = FALSE)
5.   centr_degree(g)$centralization
6. })
7. plot(tamanos, centr_vals, type = "b", log = "x")
```

Ejercicios propuestos

Ejercicio propuesto 1

Calcula las cuatro centralidades clásicas para una red tipo estrella de 20 nodos. ¿El nodo central es siempre el más central en las cuatro medidas? ¿Tiene sentido?

Autoevaluación: El nodo central debe ser #1 en las cuatro medidas. Su grado es 19, su closeness es 1, su betweenness es 1 (normalizado), y su eigenvector es 1.

Ejercicio propuesto 2

Genera una red *small-world* de 100 nodos. Encuentra el nodo con mayor betweenness y el nodo con mayor eigenvector. ¿Son el mismo? Visualiza la red coloreando el top-betweenness en rojo y el top-eigenvector en azul.

Autoevaluación: En redes *small-world* homogéneas, los nodos top pueden ser diferentes: el top-betweenness suele ser un “puente” entre clusters, mientras que el top-eigenvector está en el cluster más denso.

Ejercicio propuesto 3

Crea una función `perfil_centralidad()` que reciba un grafo y devuelva un *data frame* con todas las medidas de centralidad calculables. Incluye manejo de errores para medidas que no apliquen (por ejemplo, HITS en redes no dirigidas). Pruébala con tres redes diferentes.

Autoevaluación: Tu función debe devolver al menos 8 columnas de centralidad y funcionar sin errores tanto en redes dirigidas como no dirigidas.

Ejercicio propuesto 4

Investiga: ¿qué medida de centralidad predice mejor la **robustez** de una red? Genera una red BA de 300 nodos. Ordena los nodos por cada medida de centralidad y elimina los top-10 según cada criterio. ¿Cuál estrategia de ataque (por grado, por betweenness, por eigenvector, por PageRank) fragmenta más rápidamente la red?

Autoevaluación: El ataque por betweenness suele ser el más destructivo porque apunta directamente a los “puentes” de la red. El ataque por grado es un buen segundo lugar.

Ejercicio propuesto 5

Calcula el PageRank de una red de Zachary variando el parámetro de amortiguamiento $d \in \{0.5, 0.85, 0.95, 0.99\}$. ¿Cómo cambia el ranking de los nodos? Grafica la correlación entre los rankings para cada par de valores de d .

Autoevaluación: Con d bajo (0.5), el PageRank se vuelve más uniforme (todos los nodos reciben más “teletransportación”). Con d alto (0.99), se concentra más en los nodos bien conectados y el ranking se acerca al eigenvector.

6.11 Resumen del capítulo

flowchart LR

A[Centralidad] --> B[Clásicas: grado, closeness, betweenness, eigenvector]

A --> C[Avanzadas: PageRank, HITS, armónica, subgrafo, alpha, Bonacich]

B --> D[Comparar rankings]

C --> D

D --> E[Correlación entre medidas]

E --> F[Centralización: nivel de red]

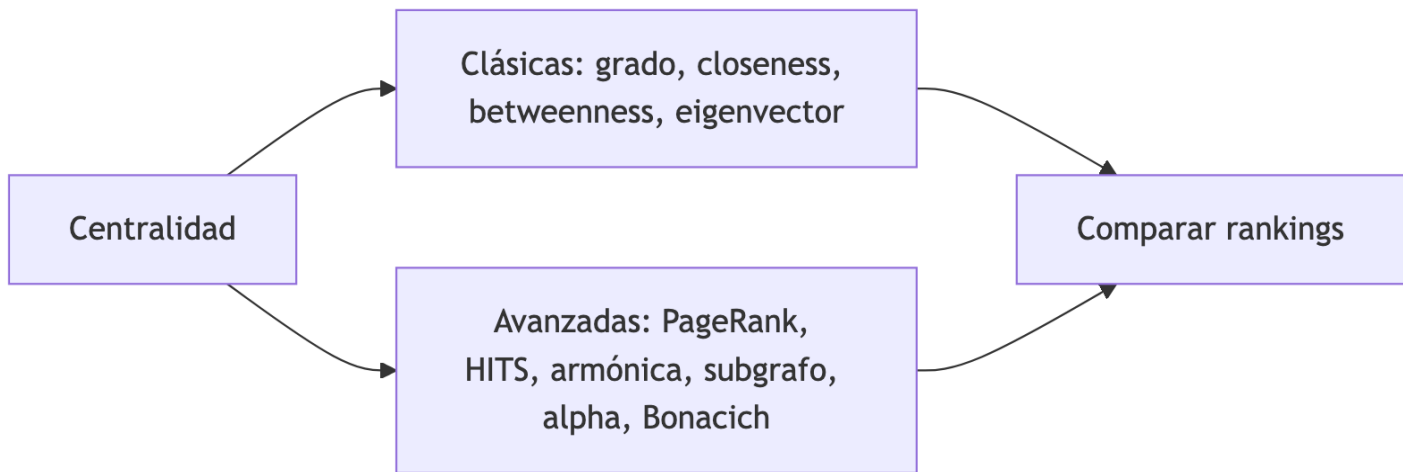


Figure 6.11: Resumen visual del capítulo

Table 6.20: Resumen de funciones de centralidad en igraph

Función	Medida	Tipo
<code>degree()</code>	Grado	Local
<code>closeness()</code>	Cercanía	Global
<code>betweenness()</code>	Intermediación	Global
<code>eigen_centrality()</code>	Vector propio	Global
<code>page_rank()</code>	PageRank	Global
<code>hub_score()</code> / <code>authority_score()</code>	HITS	Global (dirigidas)
<code>harmonic_centrality()</code>	Armónica	Global (robusta)
<code>subgraph_centrality()</code>	Subgrafo	Global (costosa)
<code>alpha_centrality()</code>	Alpha	Global (exógena)
<code>power_centrality()</code>	Bonacich	Global (negociación)

Función	Medida	Tipo
<code>strength()</code>	Fuerza	Local (ponderada)
<code>centr_degree()</code> / <code>centr_betw()</code> / etc.	Centralización	Nivel de red

Autoevaluación

Pregunta 1

¿Qué medida de centralidad identifica a los “puentes” de una red?

- a) Grado
- b) Cercanía
- c) Intermediación (*betweenness*)
- d) Vector propio

La intermediación mide cuántos caminos más cortos pasan por un nodo, identificando los conectores entre comunidades.

Pregunta 2

Verdadero o Falso: Un nodo con alto grado siempre tiene alta centralidad de vector propio.

Falso. Un nodo puede tener muchas conexiones pero a nodos periféricos y poco conectados, lo que le daría un eigenvector bajo. El eigenvector pondera la **calidad** de las conexiones, no solo la cantidad.

Pregunta 3

¿Por qué `harmonic_centrality()` es preferible a `closeness()` en redes desconectadas?

Porque `closeness()` usa $\frac{1}{\sum d(v,u)}$ y las distancias infinitas hacen que el denominador sea infinito (resultado = 0 o NaN). La armónica usa $\sum \frac{1}{d(v,u)}$ donde cada distancia infinita simplemente contribuye 0, dando un resultado finito e interpretable.

Pregunta 4

¿Qué mide la centralización de una red?

- a) La centralidad del nodo más importante
- b) El promedio de centralidad de todos los nodos
- c) Cuánto se concentra la centralidad en unos pocos nodos
- d) El número de nodos centrales

La centralización mide la **desigualdad** en la distribución de centralidad. Una estrella tiene centralización máxima; un anillo tiene centralización mínima.

Pregunta 5

En una red no dirigida, ¿los hub scores y authority scores son diferentes?

No. En redes no dirigidas, ambos scores son idénticos y equivalentes al eigenvector centrality. La distinción hub/authority solo tiene sentido en redes **dirigidas**.

7 Detección de comunidades

i Objetivos de aprendizaje

Al finalizar este capítulo serás capaz de:

1. **Definir** qué es una comunidad en una red y por qué es importante detectarlas.
2. **Aplicar** los 11 algoritmos de detección de comunidades disponibles en **igraph**.
3. **Evaluar** la calidad de una partición usando la modularidad y otras métricas.
4. **Comparar** los resultados de diferentes algoritmos sobre una misma red.
5. **Visualizar** comunidades con colores, regiones sombreadas y dendrogramas.

Pregunta motivadora: En una red social de una universidad, ¿cómo encontrarías automáticamente los grupos de amigos, los departamentos o las comunidades temáticas sin información previa sobre quién pertenece a qué grupo?

flowchart TD

```
A[Detección de comunidades] --> B[¿Qué es una comunidad?]
A --> C[Modularidad]
A --> D[Algoritmos]
A --> E[Comparación y evaluación]
D --> D1[Jerárquicos]
D --> D2[Optimización]
D --> D3[Propagación]
D --> D4[Otros]
D1 --> D1a[Edge betweenness]
D1 --> D1b[Fast greedy]
D1 --> D1c[Walktrap]
D1 --> D1d[Leading eigenvector]
D2 --> D2a[Louvain]
D2 --> D2b[Leiden]
D2 --> D2c[Optimal]
D3 --> D3a[Label propagation]
D4 --> D4a[Spinglass]
D4 --> D4b[Infomap]
D4 --> D4c[Fluid communities]
```

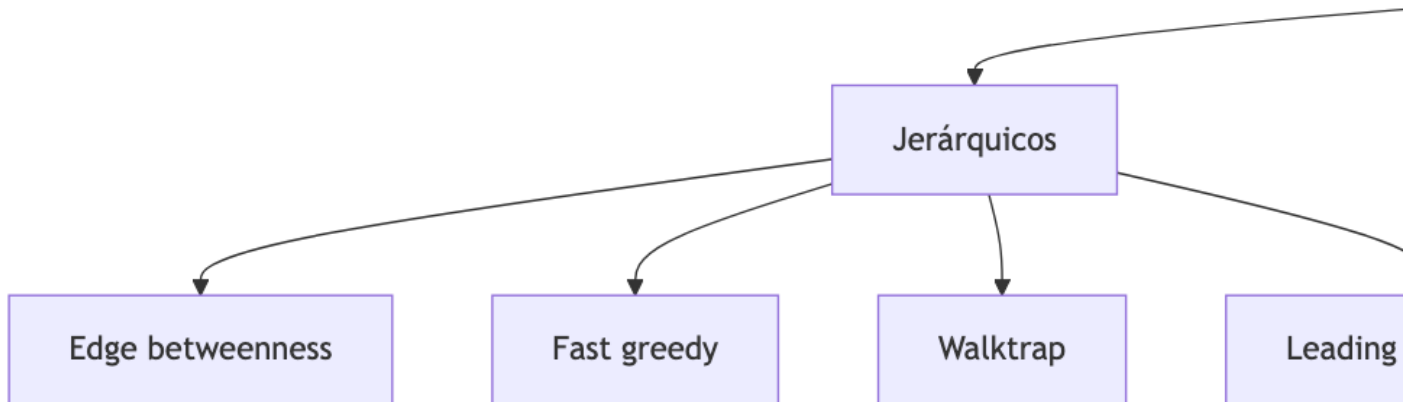


Figure 7.1: Mapa conceptual del capítulo

7.1 Preparación del entorno

```
library(igraph)
```

7.2 ¿Qué es una comunidad?

Una **comunidad** (o módulo) es un grupo de nodos que están más densamente conectados entre sí que con el resto de la red. Piensa en grupos de amigos, departamentos de una empresa o clusters funcionales en una red biológica.

```
set.seed(42)

# Crear una red con estructura de comunidades clara
g_demo <- sample_islands(islands.n = 3, islands.size = 10,
```

```

islands.pin = 0.6, n.inter = 3)
V(g_demo)$name <- paste0("n", 1:vcount(g_demo))

# Colorear por comunidad real
colores_demo <- rep(c("tomato", "skyblue", "lightgreen"), each = 10)

plot(g_demo,
     vertex.color = colores_demo,
     vertex.size = 12,
     vertex.label.cex = 0.5,
     edge.color = adjustcolor("gray50", 0.4),
     main = "Red con 3 comunidades")

```

Red con 3 comunidades

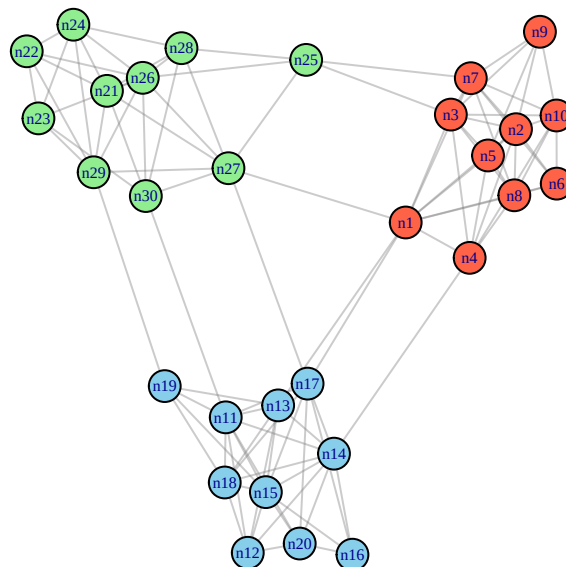


Figure 7.2: Ejemplo visual: red con 3 comunidades claramente separadas



Concepto clave

La detección de comunidades es un problema **no supervisado**: no sabemos de antemano cuántas comunidades hay ni a cuál pertenece cada nodo. Los algoritmos deben descubrirlo a partir de la estructura de la red.

7.3 Modularidad: la métrica de calidad

La **modularidad** Q mide qué tan buena es una partición comparando la densidad de aristas dentro de las comunidades con lo que esperaríamos en una red aleatoria con la misma distribución de grado:

$$Q = \frac{1}{2m} \sum_{ij} \left[A_{ij} - \frac{k_i k_j}{2m} \right] \delta(c_i, c_j)$$

donde $\delta(c_i, c_j) = 1$ si los nodos i y j están en la misma comunidad.

- $Q \approx 0$: la partición no es mejor que una división aleatoria.
- $Q > 0.3$: indica estructura de comunidades significativa.
- $Q_{\max} \leq 1$: valores más altos = mejores particiones.

```
# Asignar comunidades "reales" a la red demo
memb_real <- rep(1:3, each = 10)

cat("Modularidad con comunidades correctas:",
    round(modularity(g_demo, memb_real), 4), "\n")
```

Modularidad con comunidades correctas: 0.579

```
# Comparar con una asignación aleatoria
set.seed(10)
memb_random <- sample(1:3, vcount(g_demo), replace = TRUE)
cat("Modularidad con comunidades aleatorias:",
    round(modularity(g_demo, memb_random), 4), "\n")
```

Modularidad con comunidades aleatorias: 0.0033

7.4 Red de ejemplo: Club de Karate de Zachary

Usaremos esta red clásica para probar todos los algoritmos:

```
g <- make_graph("Zachary")
V(g)$name <- paste0("M", 1:vcount(g))

set.seed(42)
```

```
lay <- layout_with_fr(g)
```

7.5 Los 11 algoritmos de igraph

7.5.1 1. Edge Betweenness (`cluster_edge_betweenness`)

Tipo: Jerárquico divisivo (Girvan-Newman).

Idea: Elimina iterativamente la arista con mayor intermediación (*edge betweenness*). Las aristas “puente” entre comunidades tienen alta intermediación, así que al eliminarlas la red se fragmenta en comunidades.

Complejidad: $O(m^2n)$ — lento para redes grandes.

```
c_eb <- cluster_edge_betweenness(g)

par(mfrow = c(1, 2), mar = c(1, 1, 3, 1))

plot(c_eb, g, layout = lay,
     vertex.size = 12, vertex.label.cex = 0.5,
     main = paste0("Edge betweenness\nQ = ", round(modularity(g, membership(c_eb)), 4),
                  " | k = ", length(c_eb)))

# Dendrograma
plot_dendrogram(c_eb, mode = "hclust",
                main = "Dendrograma")
```

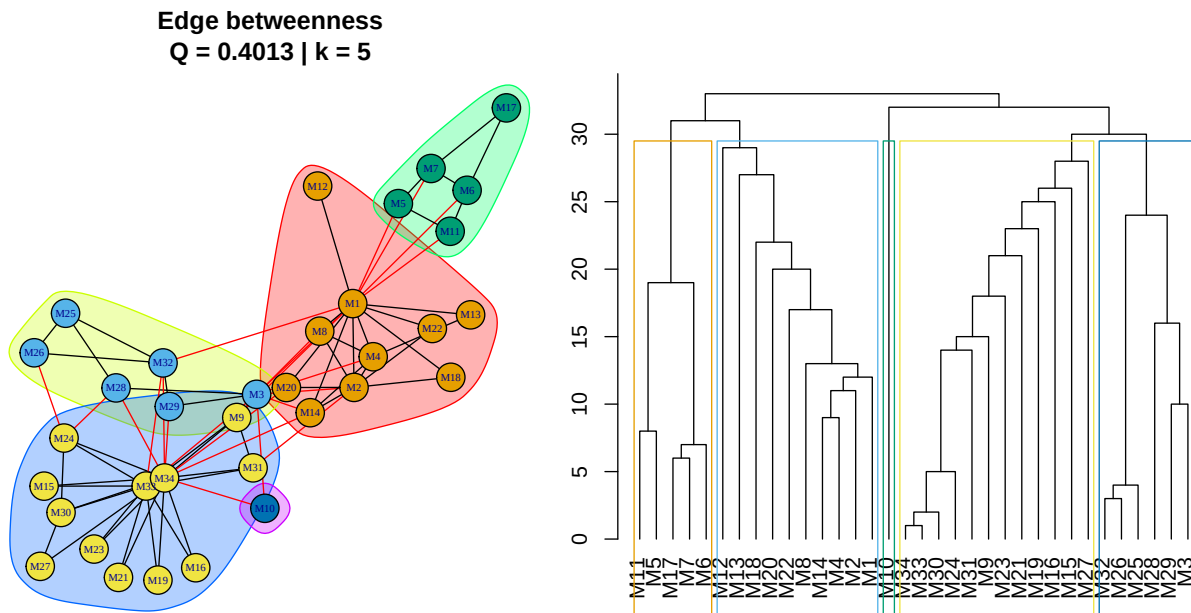


Figure 7.3: Comunidades por edge betweenness (Girvan-Newman)

7.5.2 2. Fast Greedy (cluster_fast_greedy)

Tipo: Jerárquico aglomerativo.

Idea: Comienza con cada nodo en su propia comunidad. En cada paso, fusiona las dos comunidades que producen el mayor incremento en modularidad. Es rápido y eficiente.

Complejidad: $O(m \cdot d \cdot \log n)$ donde d es la profundidad del dendrograma.

```
c_fg <- cluster_fast_greedy(g)

par(mfrow = c(1, 2), mar = c(1, 1, 3, 1))

plot(c_fg, g, layout = lay,
     vertex.size = 12, vertex.label.cex = 0.5,
     main = paste0("Fast greedy\nQ = ", round(modularity(g, membership(c_fg)), 4),
                  " | k = ", length(c_fg)))

plot_dendrogram(c_fg, mode = "hclust", main = "Dendrograma")
```

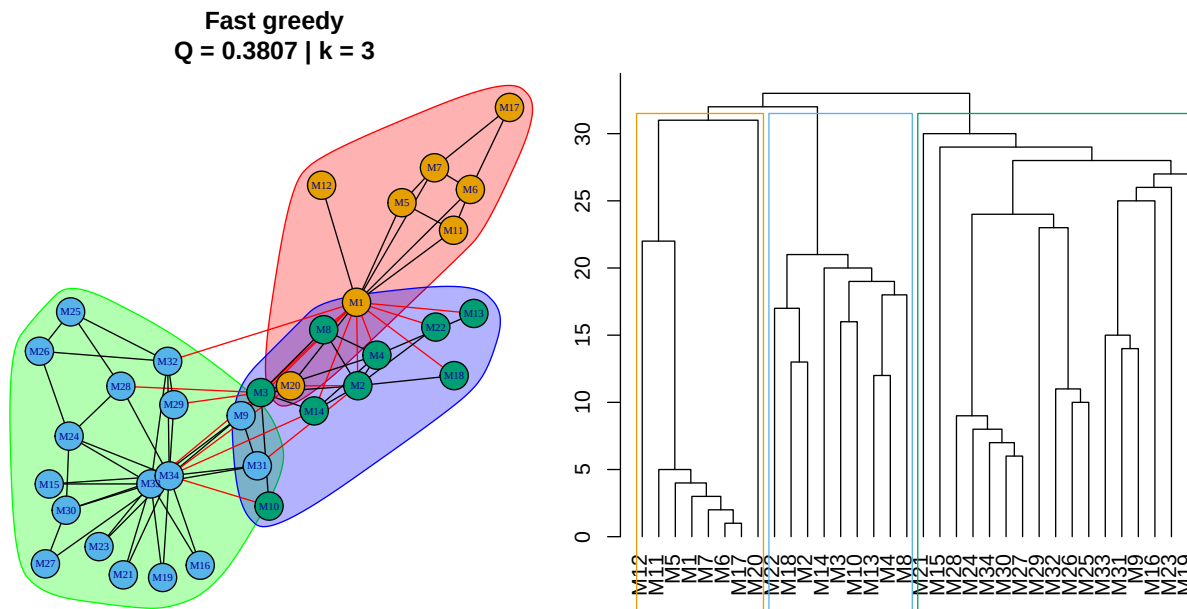


Figure 7.4: Comunidades por fast greedy

7.5.3 3. Walktrap (cluster_walktrap)

Tipo: Jerárquico aglomerativo.

Idea: Los caminantes aleatorios (*random walkers*) tienden a quedarse “atrapados” dentro de comunidades densas. Dos nodos pertenecen a la misma comunidad si las distribuciones de sus caminatas son similares.

Parámetro clave: `steps` — longitud de las caminatas (por defecto 4).

```
c_wt <- cluster_walktrap(g, steps = 4)

par(mfrow = c(1, 2), mar = c(1, 1, 3, 1))

plot(c_wt, g, layout = lay,
     vertex.size = 12, vertex.label.cex = 0.5,
     main = paste0("Walktrap (steps=4)\nQ = ", round(modularity(g, membership(c_wt)), 4),
                   " | k = ", length(c_wt)))

plot_dendrogram(c_wt, mode = "hclust", main = "Dendrograma")
```

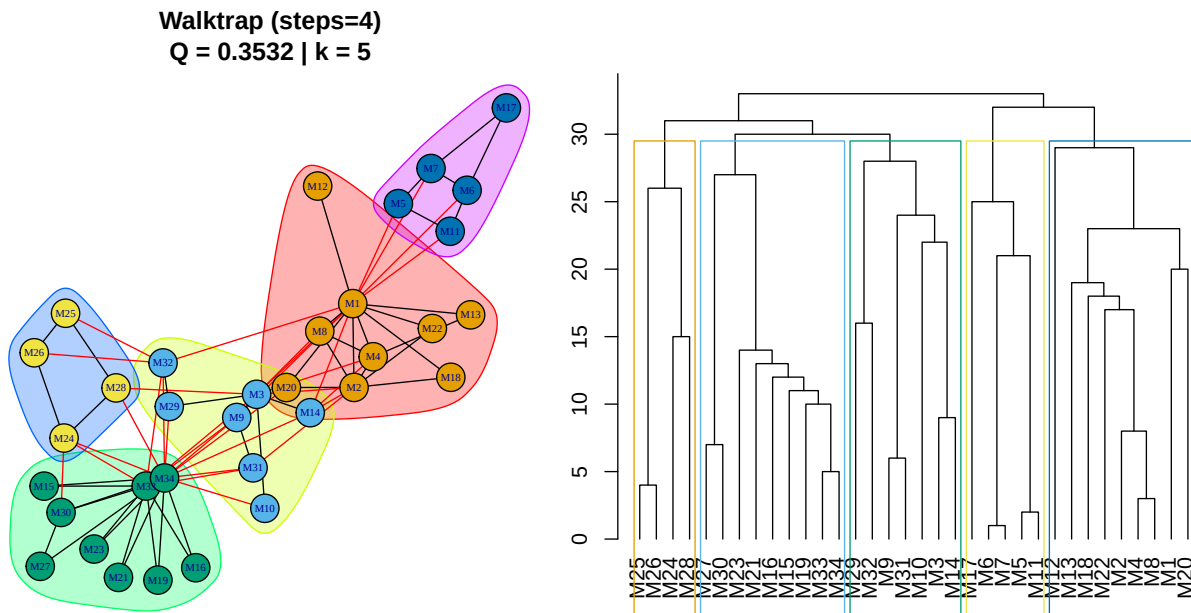


Figure 7.5: Comunidades por walktrap con caminatas de 4 pasos

7.5.4 4. Leading Eigenvector (`cluster_leading_eigen`)

Tipo: Jerárquico divisivo.

Idea: Divide la red usando el vector propio líder de la **matriz de modularidad**. Si los elementos del vector propio tienen signos opuestos, los nodos correspondientes están en comunidades distintas.

```
c_le <- cluster_leading_eigen(g)

plot(c_le, g, layout = lay,
     vertex.size = 12, vertex.label.cex = 0.5,
     main = paste0("Leading eigenvector\nQ = ", round(modularity(g, membership(c_le)), 4),
                   " | k = ", length(c_le)))
```


Leading eigenvector $Q = 0.3934 \mid k = 4$

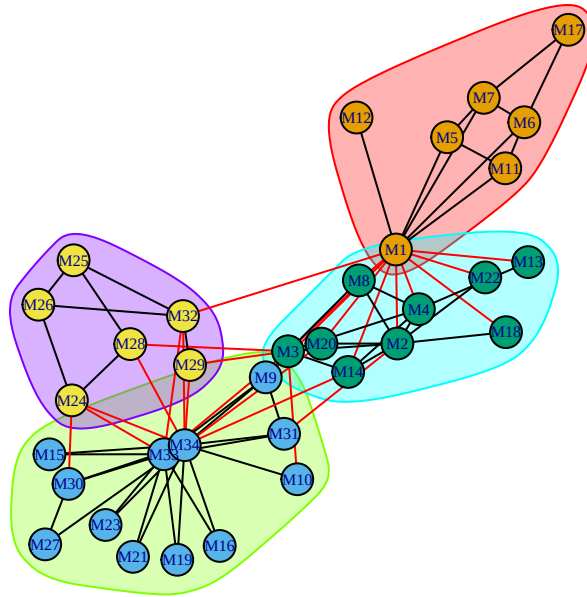


Figure 7.6: Comunidades por leading eigenvector

7.5.5 5. Louvain (`cluster_louvain`)

Tipo: Optimización multinivel de modularidad.

Idea: Cada nodo comienza en su propia comunidad. En cada iteración, mueve nodos a la comunidad vecina que maximiza el incremento de modularidad. Luego colapsa las comunidades encontradas en “supernodos” y repite. Muy rápido y popular.

Complejidad: Casi lineal $O(m)$.

```
c_lv <- cluster_louvain(g)

plot(c_lv, g, layout = lay,
     vertex.size = 12, vertex.label.cex = 0.5,
     main = paste0("Louvain\nQ = ", round(modularity(g, membership(c_lv))), 4),
           " | k = ", length(c_lv)))
```

Louvain
Q = 0.4156 | k = 4

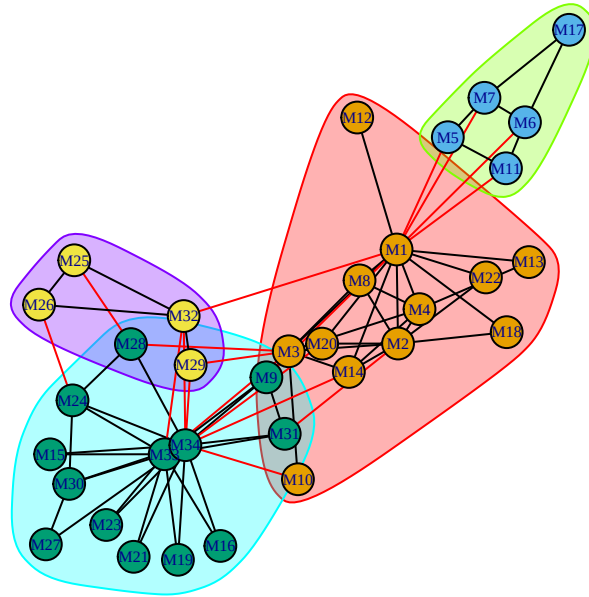


Figure 7.7: Comunidades por Louvain

💡 Concepto clave

Louvain es uno de los algoritmos más usados por su **velocidad** y buenos resultados. Sin embargo, puede producir comunidades mal conectadas internamente. El algoritmo **Leiden** (siguiente) fue diseñado para corregir este problema.

7.5.6 6. Leiden (`cluster_leiden`)

Tipo: Optimización mejorada (evolución de Louvain).

Idea: Mejora a Louvain garantizando que las comunidades detectadas estén **bien conectadas** internamente. Introduce una fase de refinamiento que evita comunidades degeneradas.

Parámetro clave: `resolution` — mayor resolución = más comunidades más pequeñas.

```
c_ld1 <- cluster_leiden(g, objective_function = "modularity", resolution = 1)
c_ld2 <- cluster_leiden(g, objective_function = "modularity", resolution = 1.5)
```

```
# Leiden no almacena modularidad en el objeto, hay que calcularla manualmente
q_ld1 <- modularity(g, membership(c_ld1))
q_ld2 <- modularity(g, membership(c_ld2))

par(mfrow = c(1, 2), mar = c(1, 1, 3, 1))

plot(c_ld1, g, layout = lay,
     vertex.size = 12, vertex.label.cex = 0.5,
     main = paste0("Leiden (res=1)\nQ = ", round(q_ld1, 4),
                   " | k = ", length(c_ld1)))

plot(c_ld2, g, layout = lay,
     vertex.size = 12, vertex.label.cex = 0.5,
     main = paste0("Leiden (res=1.5)\nQ = ", round(q_ld2, 4),
                   " | k = ", length(c_ld2)))
```

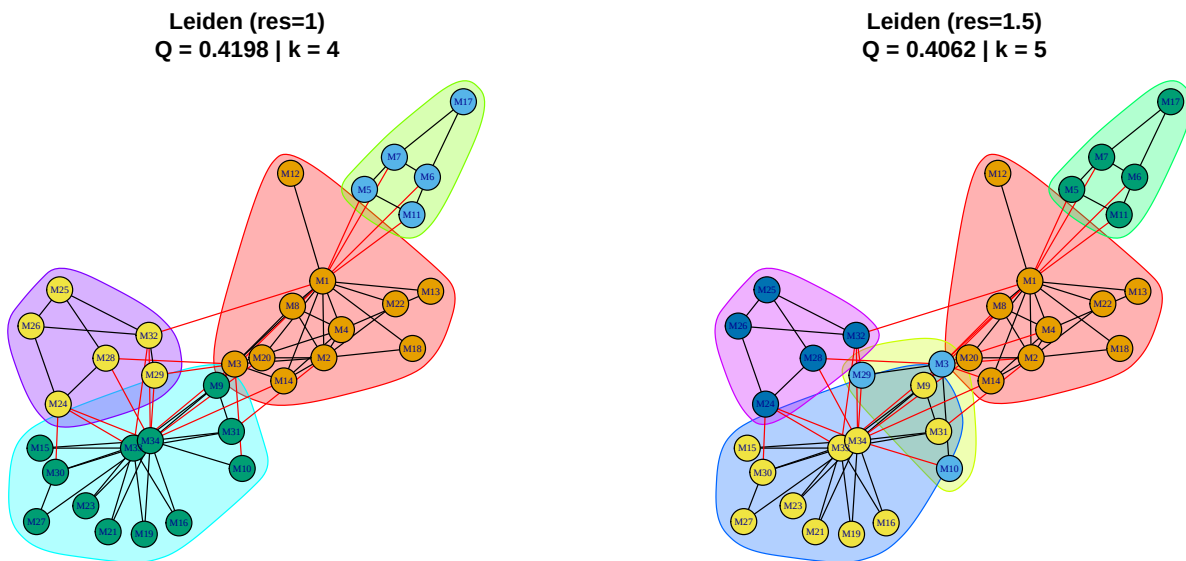


Figure 7.8: Comunidades por Leiden con dos resoluciones diferentes

7.5.7 7. Label Propagation (cluster_label_prop)

Tipo: Propagación iterativa.

Idea: Cada nodo adopta la etiqueta más frecuente entre sus vecinos. Las etiquetas se propagan por la red hasta converger. Extremadamente rápido pero **no determinista** (puede dar resultados diferentes en cada ejecución).

Complejidad: Casi lineal $O(m)$.

```
par(mfrow = c(1, 3), mar = c(1, 1, 3, 1))

for (s in 1:3) {
  set.seed(s)
  c_lp <- cluster_label_prop(g)
  plot(c_lp, g, layout = lay,
       vertex.size = 12, vertex.label.cex = 0.5,
       main = paste0("Label prop. (seed=", s, ")\nQ = ",
                     round(modularity(g, membership(c_lp)), 4),
                     " | k = ", length(c_lp)))
}
```

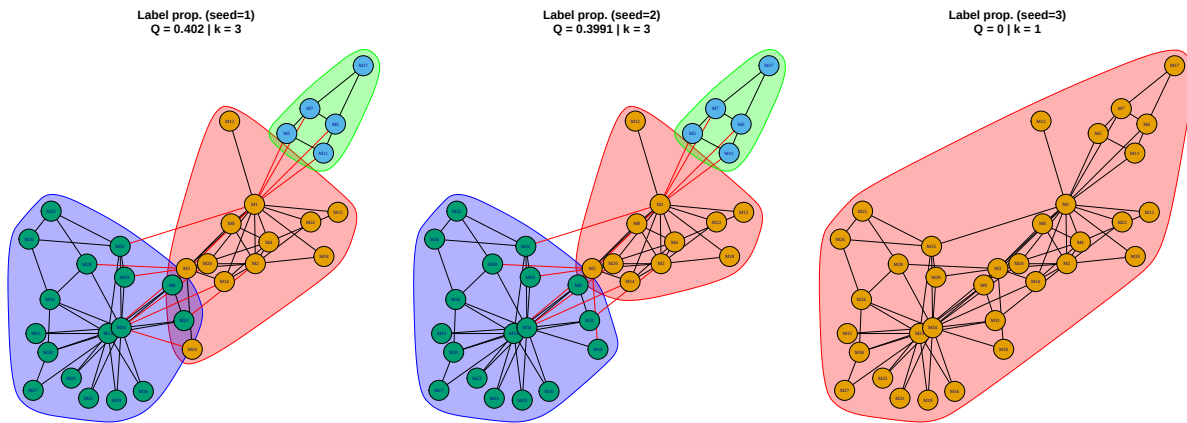


Figure 7.9: Comunidades por label propagation (3 ejecuciones)



Error frecuente

`cluster_label_prop()` es **no determinista**: ejecutarlo varias veces puede dar resultados diferentes. Siempre fija la semilla con `set.seed()` si necesitas reproducibilidad, o ejecuta múltiples veces y selecciona la partición con mayor modularidad.

7.5.8 8. Spinglass (`cluster_spinglass`)

Tipo: Basado en mecánica estadística.

Idea: Modela la red como un sistema de partículas con “espines” (*spins*). Minimiza la energía del sistema, donde los nodos en la misma comunidad tienen espines alineados. Permite ajustar la resolución con `gamma`.

Limitación: Solo funciona en redes **conexas**.

```
set.seed(42)
c_sg <- cluster_spinglass(g, spins = 10)

plot(c_sg, g, layout = lay,
     vertex.size = 12, vertex.label.cex = 0.5,
     main = paste0("Spinglass\nQ = ", round(modularity(g, membership(c_sg)), 4),
                   " | k = ", length(c_sg)))
```

Spinglass
Q = 0.4198 | k = 4

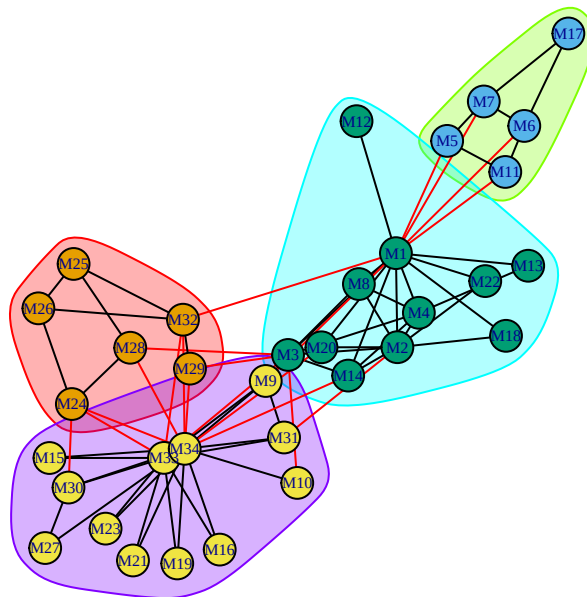


Figure 7.10: Comunidades por spinglass

7.5.9 9. Infomap (cluster_infomap)

Tipo: Basado en teoría de la información.

Idea: Busca la partición que **minimiza la longitud de descripción** del recorrido de un caminante aleatorio por la red. Comprime el “mapa” de la red usando la estructura de comunidades.

```

c_im <- cluster_infomap(g)

plot(c_im, g, layout = lay,
     vertex.size = 12, vertex.label.cex = 0.5,
     main = paste0("Infomap\nQ = ", round(modularity(g, membership(c_im)), 4),
                   " | k = ", length(c_im)))

```

Infomap
Q = 0.402 | k = 3

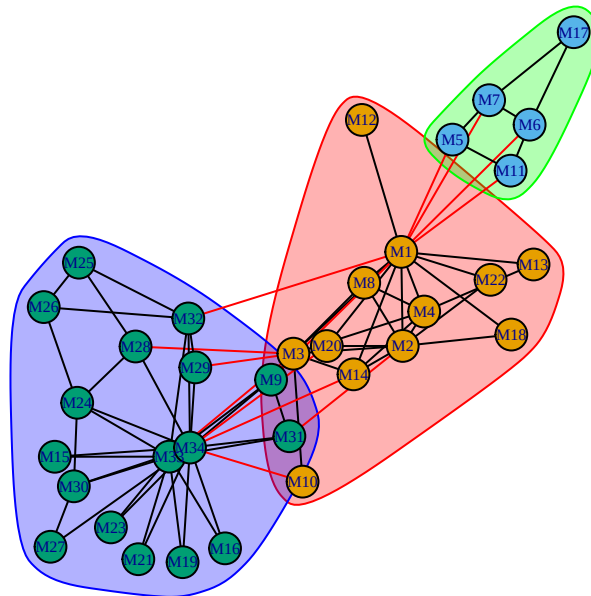


Figure 7.11: Comunidades por Infomap

7.5.10 10. Fluid Communities (cluster_fluid_communities)

Tipo: Basado en dinámica de fluidos.

Idea: Simula varios “fluidos” que se expanden y contraen en la topología de la red. Cada fluido representa una comunidad. **Requiere especificar** el número de comunidades de antemano.

```

par(mfrow = c(1, 3), mar = c(1, 1, 3, 1))

for (k in 2:4) {
  set.seed(42)

```

```

c_fl <- cluster_fluid_communities(g, no.of.communities = k)
q_fl <- modularity(g, membership(c_fl))
plot(c_fl, g, layout = lay,
     vertex.size = 12, vertex.label.cex = 0.5,
     main = paste0("Fluid (k=", k, ")\nQ = ",
                   round(q_fl, 4)))
}

```

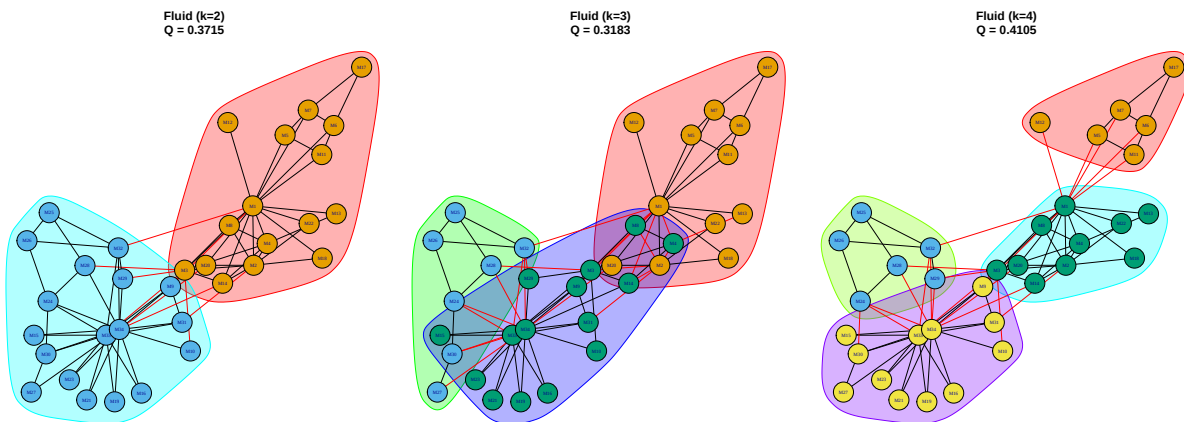


Figure 7.12: Comunidades por fluid communities (k=2, 3 y 4)

7.5.11 11. Optimal (cluster_optimal)

Tipo: Optimización exacta.

Idea: Encuentra la partición que **maximiza exactamente** la modularidad. Garantiza el resultado óptimo, pero es computacionalmente intratable para redes grandes (NP-completo).

Limitación: Solo viable para redes pequeñas ($n < 50-100$).

```

c_opt <- cluster_optimal(g)

plot(c_opt, g, layout = lay,
     vertex.size = 12, vertex.label.cex = 0.5,
     main = paste0("Optimal\nQ = ", round(modularity(g, membership(c_opt)), 4),
                   " | k = ", length(c_opt)))

```

Optimal
 $Q = 0.4198 \mid k = 4$

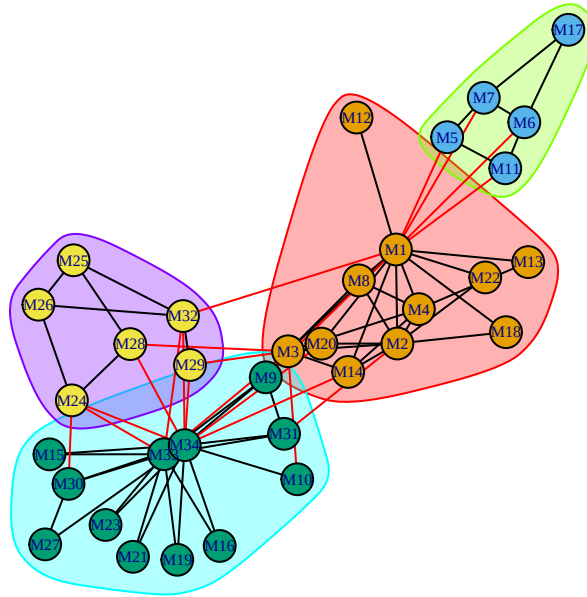


Figure 7.13: Comunidades por optimización exacta de modularidad

! Recuerda

`cluster_optimal()` da la **mejor partición posible** según modularidad, pero es extremadamente lento. Úsalo como referencia en redes pequeñas para evaluar qué tan cerca están los otros algoritmos del óptimo.

7.6 Comparación de todos los algoritmos

7.6.1 Tabla resumen

```
set.seed(42)

# Ejecutar todos los algoritmos
algoritmos <- list(
  "Edge betweenness" = cluster_edge_betweenness(g),
```



```

"Fast greedy"      = cluster_fast_greedy(g),
"Walktrap"         = cluster_walktrap(g),
"Leading eigenvector" = cluster_leading_eigen(g),
"Louvain"          = cluster_louvain(g),
"Leiden"           = cluster_leiden(g, objective_function = "modularity"),
"Label propagation" = cluster_label_prop(g),
"Spinglass"        = cluster_spinglass(g),
"Infomap"          = cluster_infomap(g),
"Fluid (k=2)"      = cluster_fluid_communities(g, no.of.communities = 2),
"Optimal"          = cluster_optimal(g)
)

comparacion <- data.frame(
  Algoritmo = names(algoritmos),
  Comunidades = sapply(algoritmos, length),
  Modularidad = round(sapply(algoritmos, function(c) modularity(g, membership(c))), 4),
  Tamaño_max = sapply(algoritmos, function(c) max(sizes(c))),
  Tamaño_min = sapply(algoritmos, function(c) min(sizes(c)))
)

comparacion <- comparacion[order(comparacion$Modularidad, decreasing = TRUE), ]

knitr::kable(comparacion,
  caption = "Comparación de los 11 algoritmos de detección de comunidades",
  col.names = c("Algoritmo", "Comunidades", "Modularidad",
    "Tamaño máx.", "Tamaño mín."),
  row.names = FALSE)

```

Table 7.1: Comparación de los 11 algoritmos de detección de comunidades

Algoritmo	Comunidades	Modularidad	Tamaño máx.	Tamaño mín.
Leiden	4	0.4198	12	5
Spinglass	4	0.4198	12	5
Optimal	4	0.4198	12	5
Louvain	4	0.4156	13	4
Infomap	3	0.4020	17	5
Edge betweenness	5	0.4013	12	1
Leading eigenvector	4	0.3934	12	6
Fast greedy	3	0.3807	17	8
Label propagation	3	0.3589	17	5
Fluid (k=2)	2	0.3589	17	17

Algoritmo	Comunidades	Modularidad	Tamaño máx.	Tamaño mín.
Walktrap	5	0.3532	9	4

7.6.2 Visualización comparativa

```

seleccion <- c("Edge betweenness", "Fast greedy", "Walktrap",
              "Louvain", "Leiden", "Optimal")

par(mfrow = c(2, 3), mar = c(1, 1, 3, 1))

for (nombre in seleccion) {
  c_alg <- algoritmos[[nombre]]
  q_alg <- modularity(g, membership(c_alg))
  plot(c_alg, g, layout = lay,
       vertex.size = 12, vertex.label.cex = 0.45,
       main = paste0(nombre, "\nQ = ", round(q_alg, 4),
                     " | k = ", length(c_alg)))
}

```

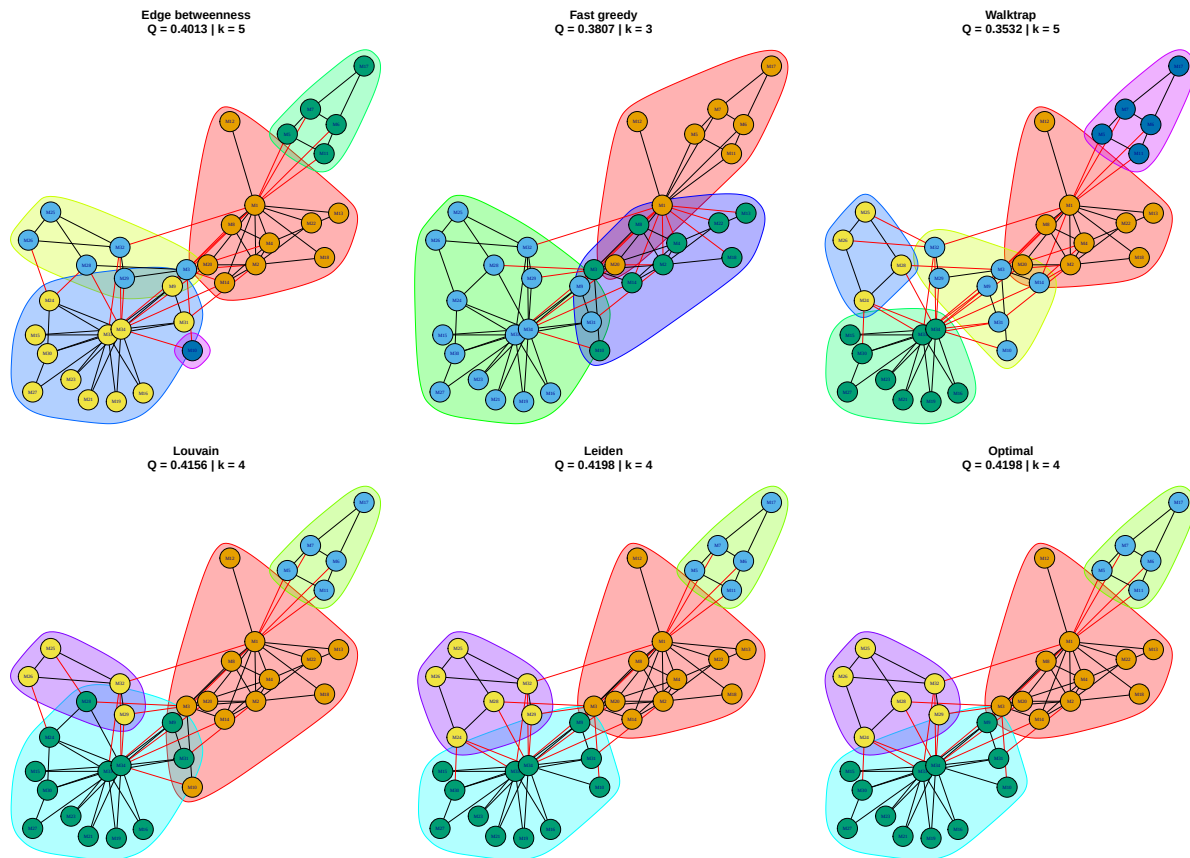


Figure 7.14: Seis algoritmos aplicados a la red de Zachary

7.6.3 Gráfico de barras: modularidad

```
comp_sorted <- comparacion[order(comparacion$Modularidad, decreasing = TRUE), ]

barplot(comp_sorted$Modularidad,
        names.arg = comp_sorted$Algoritmo,
        col = hcl.colors(nrow(comp_sorted), palette = "Zissou 1"),
        main = "Modularidad por algoritmo",
        ylab = "Modularidad (Q)",
        las = 2, cex.names = 0.7,
        ylim = c(0, max(comp_sorted$Modularidad) * 1.15))

# Línea del óptimo
q_optimo <- modularity(g, membership(algoritmos[["Optimal"]]))
```

```
abline(h = q_optimo, col = "red", lty = 2, lwd = 2)
legend("topright", "Óptimo", col = "red", lty = 2, lwd = 2, bty = "n")
```

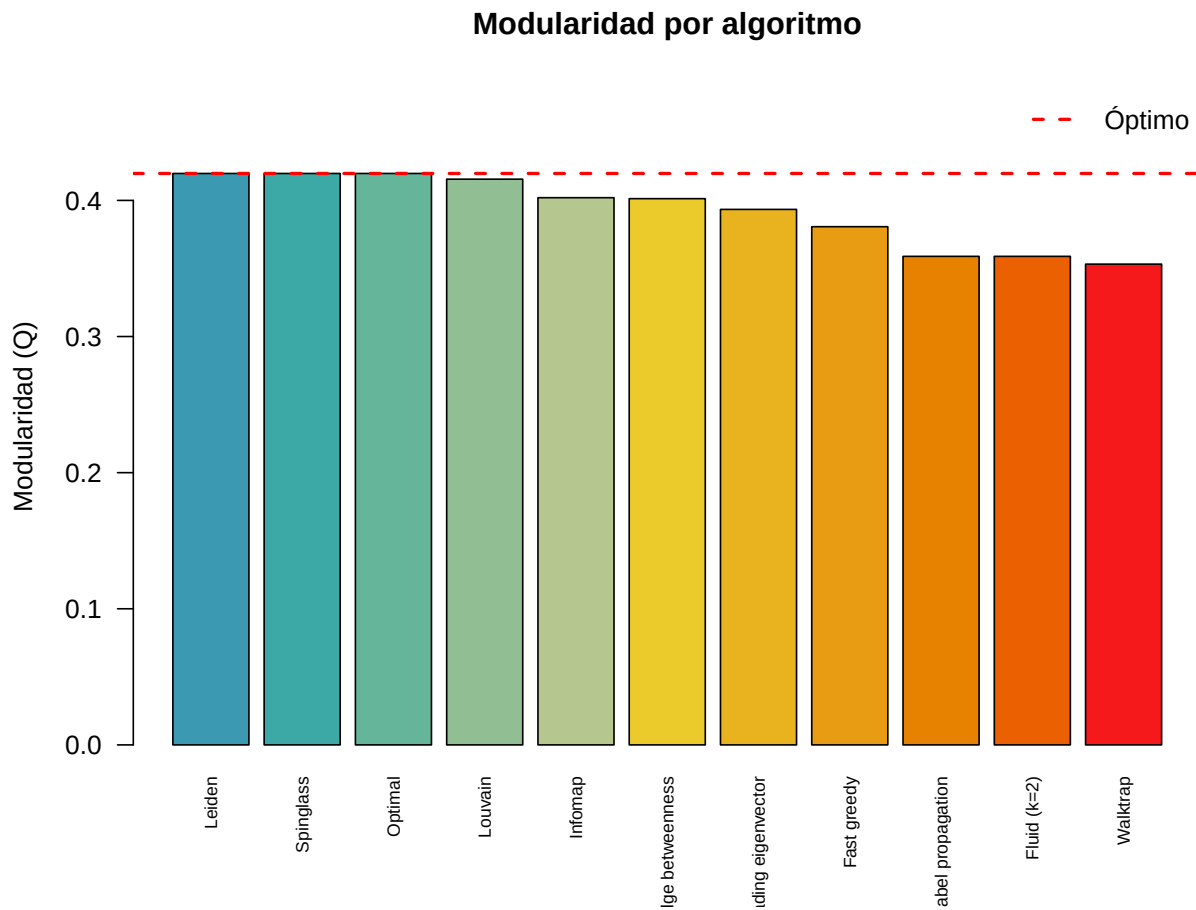


Figure 7.15: Modularidad de cada algoritmo en la red de Zachary

7.7 Comparar dos particiones: NMI y Rand Index

¿Qué tan similares son las particiones encontradas por dos algoritmos diferentes? Usamos métricas de comparación:

- **NMI** (Normalized Mutual Information): mide cuánta información comparten dos particiones. 1 = idénticas, 0 = independientes.
- **Rand Index ajustado**: mide la proporción de pares de nodos clasificados igual en ambas particiones.

```

n_alg <- length(algoritmos)
nmi_mat <- matrix(0, n_alg, n_alg)
rownames(nmi_mat) <- colnames(nmi_mat) <- names(algoritmos)

for (i in 1:n_alg) {
  for (j in 1:n_alg) {
    nmi_mat[i, j] <- compare(algoritmos[[i]], algoritmos[[j]], method = "nmi")
  }
}

heatmap(nmi_mat,
        col = hcl.colors(20, palette = "YlOrRd", rev = TRUE),
        scale = "none",
        margins = c(12, 12),
        main = "Similitud entre algoritmos (NMI)")

```

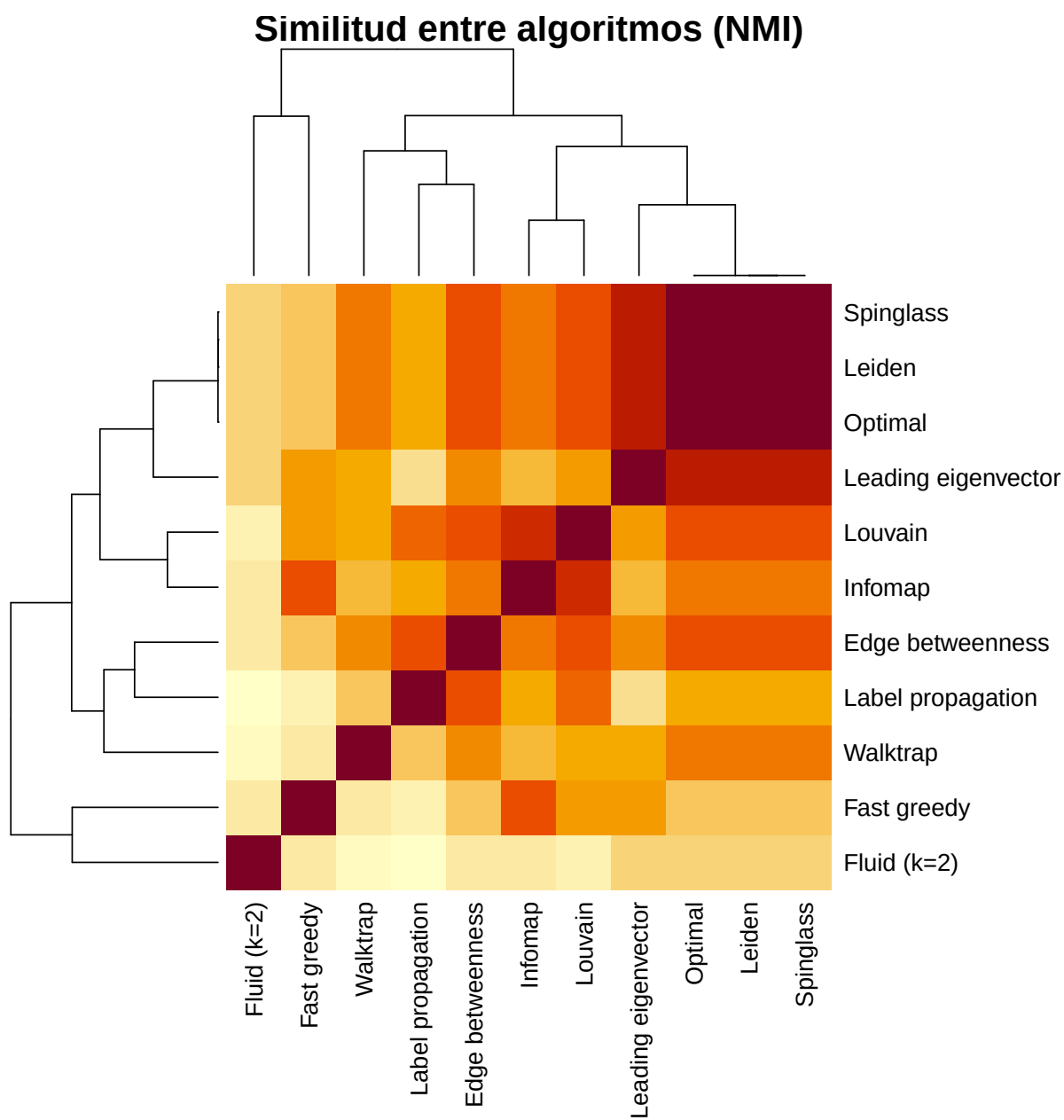


Figure 7.16: Similitud (NMI) entre las particiones de todos los algoritmos

```
knitr::kable(round(nmi_mat, 3),
              caption = "Matriz de similitud NMI entre algoritmos")
```

Table 7.2: Matriz de similitud NMI entre algoritmos

	Edge be- tween- ness	Fast greedy	Walk- trap	Leading eigenvec- tor	Lou- vain	Lei- den	Label propa- gation	Sp- in- glass	In- fomap(k=2)	Fluid	Op- ti- mal
Edge between- ness	1.000	0.634	0.731	0.739	0.830	0.832	0.825	0.832	0.759	0.550	0.832
Fast greedy	0.634	1.000	0.561	0.707	0.728	0.634	0.528	0.634	0.826	0.565	0.634
Walk- trap	0.731	0.561	1.000	0.679	0.682	0.762	0.631	0.762	0.669	0.490	0.762
Leading eigenvec- tor	0.739	0.707	0.679	1.000	0.712	0.896	0.590	0.896	0.658	0.600	0.896
Louvain	0.830	0.728	0.682	0.712	1.000	0.815	0.788	0.815	0.880	0.536	0.815
Leiden	0.832	0.634	0.762	0.896	0.815	1.000	0.704	1.000	0.772	0.609	1.000
Label propaga- tion	0.825	0.528	0.631	0.590	0.788	0.704	1.000	0.704	0.696	0.462	0.704
Spin- glass	0.832	0.634	0.762	0.896	0.815	1.000	0.704	1.000	0.772	0.609	1.000
Infomap	0.759	0.826	0.669	0.658	0.880	0.772	0.696	0.772	1.000	0.568	0.772
Fluid (k=2)	0.550	0.565	0.490	0.600	0.536	0.609	0.462	0.609	0.568	1.000	0.609
Optimal	0.832	0.634	0.762	0.896	0.815	1.000	0.704	1.000	0.772	0.609	1.000



Concepto clave

Cuando varios algoritmos **diferentes** producen particiones similares (NMI alto entre ellos), es una señal fuerte de que la estructura de comunidades es **robusta** y no un artefacto de un algoritmo particular.

7.8 Guía de selección de algoritmo

Table 7.3: Guía de selección de algoritmo

Criterio	Algoritmo recomendado
Red pequeña (< 100 nodos), quieres el óptimo	<code>cluster_optimal()</code>
Red grande, uso general	<code>cluster_louvain()</code> o <code>cluster_leiden()</code>
Quieres un dendrograma	<code>cluster_fast_greedy()</code> o <code>cluster_walktrap()</code>
Red dirigida	<code>cluster_infomap()</code> o <code>cluster_walktrap()</code>
Red con pesos	<code>cluster_louvain()</code> , <code>cluster_leiden()</code> , <code>cluster_walktrap()</code>
Sabes cuántas comunidades quieres	<code>cluster_fluid_communities()</code>
Máxima velocidad	<code>cluster_label_prop()</code> o <code>cluster_louvain()</code>
Quieres controlar la resolución	<code>cluster_leiden(resolution = ...)</code>
Red conexa, análisis fino	<code>cluster_spinglass()</code>



Error frecuente

No todos los algoritmos funcionan en todas las redes:

- `cluster_fast_greedy()` solo funciona en grafos **no dirigidos** sin aristas múltiples.
- `cluster_spinglass()` solo funciona en grafos **conexos**.
- `cluster_fluid_communities()` requiere especificar el número de comunidades y que el grafo sea **conexo**.
- `cluster_optimal()` es inviable para redes con más de ~ 100 nodos.

7.9 Ejercicios

Ejercicios resueltos

Ejercicio resuelto 1: Aplica tres algoritmos (Louvain, Walktrap y Leading eigenvector) a la red de Zachary. Compara las particiones usando NMI. ¿Cuáles dos algoritmos producen resultados más similares?

💡 Solución

```
c1 <- cluster_louvain(g)
c2 <- cluster_walktrap(g)
c3 <- cluster_leading_eigen(g)

pares <- list(
  c("Louvain", "Walktrap"),
  c("Louvain", "Leading eigen"),
  c("Walktrap", "Leading eigen")
)
comunidades_lista <- list(c1, c2, c3)
nombres <- c("Louvain", "Walktrap", "Leading eigen")

resultados <- data.frame(
  Par = sapply(pares, paste, collapse = " vs. "),
  NMI = round(c(
    compare(c1, c2, method = "nmi"),
    compare(c1, c3, method = "nmi"),
    compare(c2, c3, method = "nmi")
  ), 4),
  Rand_ajustado = round(c(
    compare(c1, c2, method = "adjusted.rand"),
    compare(c1, c3, method = "adjusted.rand"),
    compare(c2, c3, method = "adjusted.rand")
  ), 4)
)

knitr::kable(resultados,
  caption = "Similitud entre tres algoritmos",
  col.names = c("Par de algoritmos", "NMI", "Rand ajustado"))
```

Table 7.4: Similitud entre tres algoritmos

Par de algoritmos	NMI	Rand ajustado
Louvain vs. Walktrap	0.6817	0.5600
Louvain vs. Leading eigen	0.7116	0.6352
Walktrap vs. Leading eigen	0.6786	0.5398

Interpretación: El par con NMI más alto son los que producen particiones más similares. Típicamente, Louvain y Walktrap coinciden bastante en la red de Zachary porque ambos tienden a encontrar 4-5 comunidades con modularidad similar.

Ejercicio resuelto 2: Genera una red con estructura de comunidades conocida usando `sample_islands()`. Aplica `cluster_louvain()` y evalúa qué tan bien recupera las comunidades reales.

 Solución

```
set.seed(42)

# Red con 4 comunidades de 15 nodos, 5 aristas entre comunidades
g_test <- sample_islands(islands.n = 4, islands.size = 15,
                        islands.pin = 0.5, n.inter = 5)
V(g_test)$name <- paste0("n", 1:vcount(g_test))

# Comunidades reales
memb_real <- rep(1:4, each = 15)

# Detectar con Louvain
c_test <- cluster_louvain(g_test)

cat("Comunidades reales: 4\n")
```

Comunidades reales: 4

```
cat("Comunidades detectadas:", length(c_test), "\n")
```

Comunidades detectadas: 4

```
cat("NMI:", round(compare(memb_real, membership(c_test), method = "nmi"), 4), "\n")
```

NMI: 1

```
cat("Modularidad:", round(modularity(g_test, membership(c_test)), 4), "\n")
```

Modularidad: 0.6208

```

lay_test <- layout_with_fr(g_test)

par(mfrow = c(1, 2), mar = c(1, 1, 3, 1))

plot(g_test, layout = lay_test,
     vertex.color = memb_real,
     vertex.size = 10, vertex.label = NA,
     main = "Comunidades reales")

plot(c_test, g_test, layout = lay_test,
     vertex.size = 10, vertex.label = NA,
     main = paste0("Louvain (NMI = ",
                   round(compare(memb_real, membership(c_test), method = "nmi"), 3), ")")

```

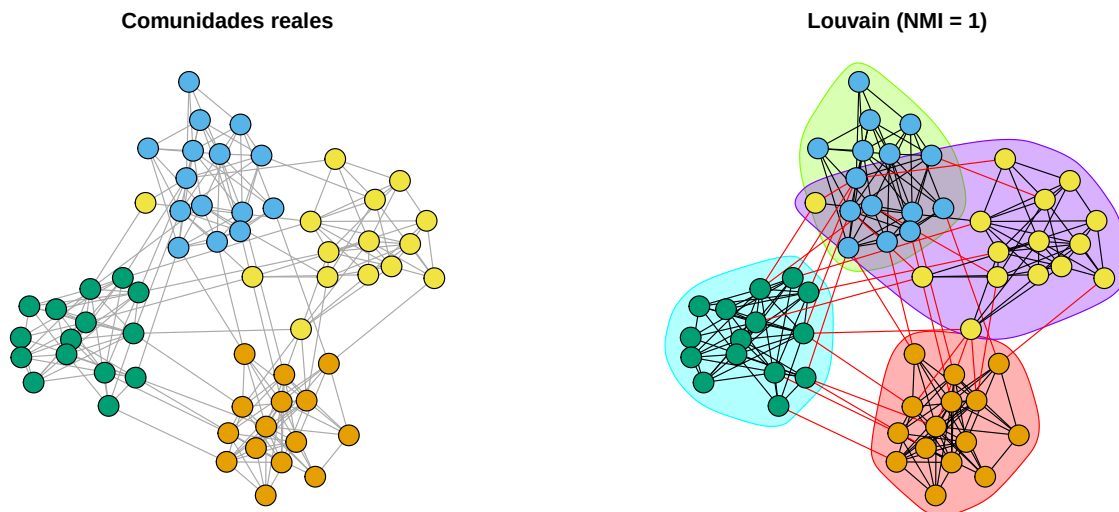


Figure 7.17: Comunidades reales (izq.) vs. detectadas por Louvain (der.)

Interpretación: Si el NMI es cercano a 1, Louvain recuperó casi perfectamente las comunidades reales. Valores menores indican que algunas comunidades se fusionaron o se dividieron.

Ejercicios con pistas

Ejercicio 1: Investiga el efecto del parámetro `steps` en `cluster_walktrap()`. Ejecuta el algoritmo con `steps = 2, 4, 6, 8, 10, 20` sobre la red de Zachary. ¿Cómo cambian el número de comunidades y la modularidad?

i Pista 1

Usa un bucle `for` sobre los valores de `steps`. Guarda `length(c)` y `modularity(c)` para cada valor.

i Pista 2

Caminatas muy cortas (`steps = 2`) pueden no capturar la estructura global. Caminatas muy largas (`steps = 20`) pueden “borrar” las diferencias entre comunidades.

i Pista 3 — Pseudocódigo

```
1. pasos <- c(2, 4, 6, 8, 10, 20)
2. resultados <- data.frame(steps = pasos, k = NA, Q = NA)
3. for (i in seq_along(pasos)) {
4.   c <- cluster_walktrap(g, steps = pasos[i])
5.   resultados$k[i] <- length(c)
6.   resultados$Q[i] <- modularity(c)
7. }
8. plot(resultados$steps, resultados$Q, type = "b")
```

Ejercicio 2: Aplica `cluster_leiden()` variando el parámetro `resolution` de 0.5 a 3.0 (en pasos de 0.5). Grafica el número de comunidades vs. resolución. ¿La relación es monótona?

i Pista 1

Mayor resolución = más comunidades más pequeñas. Pero la relación no es perfectamente lineal porque depende de la estructura de la red.

i Pista 2

Usa `objective_function = "modularity"` para que los resultados sean comparables con otros algoritmos basados en modularidad.

i Pista 3 — Pseudocódigo

```
1. res_vals <- seq(0.5, 3.0, by = 0.5)
2. resultados <- sapply(res_vals, function(r) {
3.   c <- cluster_leiden(g, objective_function = "modularity",
```

```

4.             resolution = r)
5.     c(k = length(c), Q = modularity(c))
6. })
7. plot(res_vals, resultados["k", ], type = "b",
8.       xlab = "Resolución", ylab = "Número de comunidades")

```

Ejercicio 3: Genera una red BA de 200 nodos. Aplica `cluster_louvain()` y `cluster_infomap()`. ¿Encuentran comunidades significativas? ¿Tiene sentido hablar de “comunidades” en una red BA pura?

i Pista 1

Las redes BA no tienen una estructura de comunidades clara “de fábrica”. Sin embargo, los algoritmos siempre encontrarán alguna partición — la pregunta es si la modularidad es significativamente mayor que cero.

i Pista 2

Compara la modularidad obtenida con la de redes aleatorias del mismo tamaño para saber si es significativa. Genera 100 redes ER y calcula la modularidad de `cluster_louvain()` en cada una como referencia.

i Pista 3 — Pseudocódigo

```

1. set.seed(42)
2. g_ba <- sample_pa(200, m = 2, directed = FALSE)
3. c_lv <- cluster_louvain(g_ba)
4. cat("Q (BA):", modularity(g, membership(c_lv)))
5.
6. # Referencia: 100 redes ER
7. q_ref <- sapply(1:100, function(i) {
8.     g_er <- sample_gnp(200, p = edge_density(g_ba))
9.     modularity(cluster_louvain(g_er))
10. })
11. cat("Q medio (ER):", mean(q_ref))
12. hist(q_ref); abline(v = modularity(g, membership(c_lv)), col = "red")

```

Ejercicios propuestos

Ejercicio propuesto 1

Aplica los 11 algoritmos a una red tipo anillo de 30 nodos. ¿Cuántas comunidades encuentra cada uno? ¿Tiene sentido hablar de comunidades en un anillo?

Autoevaluación: Un anillo es perfectamente simétrico, sin estructura de comunidades real. La modularidad debería ser baja ($Q < 0.3$) para la mayoría de algoritmos. Algunos podrían dividir el anillo en segmentos arbitrarios.

Ejercicio propuesto 2

Crea una red con estructura de comunidades conocida (`sample_islands()` con 5 comunidades de 20 nodos). Evalúa los 11 algoritmos con NMI contra la partición real. ¿Cuál recupera mejor las comunidades reales? Presenta un gráfico de barras con los NMI.

Autoevaluación: Los algoritmos basados en modularidad (Louvain, Leiden, Optimal) suelen tener NMI alto. Label propagation puede variar mucho entre ejecuciones.

Ejercicio propuesto 3

Investiga la **resolución límite** de la modularidad: genera una red con 10 comunidades pequeñas (5 nodos cada una) conectadas por pocas aristas. ¿Los algoritmos basados en modularidad logran distinguir las 10 comunidades o las fusionan?

Autoevaluación: Es probable que `cluster_louvain()` fusione algunas comunidades pequeñas (este es un problema conocido de la modularidad). `cluster_leiden()` con resolución alta debería encontrar más comunidades.

Ejercicio propuesto 4

Diseña un experimento: genera redes con estructura de comunidades cada vez más difusa (variando `n.inter` en `sample_islands()` de 1 a 50). Para cada nivel de “difusión”, aplica Louvain y calcula el NMI contra la partición real. Grafica NMI vs. número de aristas inter-comunidad. ¿En qué punto los algoritmos dejan de detectar las comunidades?

Autoevaluación: El NMI debería decrecer gradualmente. Cuando las aristas inter-comunidad igualan las intra-comunidad, la estructura se pierde y el NMI se acerca a 0.

Ejercicio propuesto 5

Crea una función `consenso_comunidades()` que ejecute k algoritmos diferentes, calcule la matriz de co-ocurrencia (cuántas veces cada par de nodos termina en la misma comunidad), y aplique un clustering final sobre esa matriz. Pruébala con la red de Zachary usando 5 algoritmos.

Autoevaluación: El consenso debería producir una partición más estable que cualquier algoritmo individual. La modularidad del consenso debería estar cerca del promedio o por encima.

7.10 Resumen del capítulo

Table 7.5: Resumen de algoritmos de detección de comunidades

Algoritmo	Función	Tipo	Complejidad	Dendrograma
Edge betweenness	<code>cluster_edge_betweenness()</code>	Divisivo	$O(m^2n)$	Sí
Fast greedy	<code>cluster_fast_greedy()</code>	Aditivo	$O(md \log n)$	Sí
Walktrap	<code>cluster_walktrap()</code>	Aditivo	$O(mn^2)$	Sí
Leading eigenvector	<code>cluster_leading_eigen()</code>	Divisivo	$O(n^2 + m)$	Sí
Louvain	<code>cluster_louvain()</code>	Multi-nivel	$\sim O(m)$	No
Leiden	<code>cluster_leiden()</code>	Multi-nivel+	$\sim O(m)$	No
Label propagation	<code>cluster_label_prop()</code>	Propagación	$\sim O(m)$	No
Spinglass	<code>cluster_spinglass()</code>	Simulación	$O(n^{3.2})$	No
Infomap	<code>cluster_infomap()</code>	Información	$O(m)$	No
Fluid	<code>cluster_fluid_communities()</code>	Dinámico	$O(m \cdot k)$	No

Algoritmo	Función	Tipo	Complejidad	Dendrograma
Optimal	<code>cluster_optimal()</code>	Exacto	NP-completo	No

Autoevaluación

Pregunta 1

¿Qué mide la modularidad?

- a) El número de comunidades
- b) El tamaño de la comunidad más grande
- c) La diferencia entre la densidad intra-comunidad y la esperada por azar
- d) La distancia media entre comunidades

La modularidad compara las aristas dentro de comunidades con lo esperado en una red aleatoria con la misma distribución de grado.

Pregunta 2

¿Cuál algoritmo es el más rápido para redes muy grandes (millones de nodos)?

- a) Edge betweenness
- b) Optimal
- c) Louvain o Label propagation
- d) Spinglass

Louvain y Label propagation tienen complejidad casi lineal $O(m)$, haciéndolos viables para redes masivas.

Pregunta 3

Verdadero o Falso: `cluster_optimal()` siempre da la mejor respuesta posible.

Verdadero, pero con una advertencia importante: es óptimo respecto a la **modularidad**, no respecto a la “verdad”. Si las comunidades reales no corresponden a la partición que maximiza modularidad (por ejemplo, comunidades muy pequeñas), el resultado óptimo puede no ser el correcto.

Pregunta 4

¿Qué algoritmo usarías si necesitas especificar el número de comunidades?

- a) Louvain
- b) Infomap
- c) Fluid communities
- d) Fast greedy

`cluster_fluid_communities()` requiere que el usuario especifique `no.of.communities`. Los demás algoritmos determinan el número automáticamente.

Pregunta 5

¿Qué mide el NMI (Normalized Mutual Information) entre dos particiones?

- a) La modularidad promedio
- b) Cuánta información comparten ambas particiones
- c) El número de nodos que cambian de comunidad
- d) La diferencia en número de comunidades

$NMI = 1$ significa que ambas particiones son idénticas; $NMI = 0$ significa que son completamente independientes.

8 VisNetwork y networkD3

i Objetivos de aprendizaje

Al finalizar este capítulo serás capaz de:

1. **Generar** redes interactivas usando la librería `eVisNetwork` y `networkD3`.
2. **Generar** redes h.ierárquicas usando `networkD3` y explicar su utilidad para representar relaciones de dependencia. `graph`.
3. **Visualizar y comparar** distribuciones de grado usando histogramas y gráficos log-log.
4. **Ajustar** una ley de potencia a datos de grado con el paquete `powerLaw`.
5. **Identificar** *hubs* (nodos altamente conectados) y explicar su importancia en la estructura de la red.

`visNetwork` es una librería de R que permite crear visualizaciones interactivas de redes basado en la biblioteca de javascript `vis.js`. Es especialmente útil para representar relaciones entre entidades, como conexiones entre personas, organizaciones o cualquier tipo de nodos y enlaces. A continuación, se muestra un ejemplo básico de cómo usar `visNetwork` para crear una red simple.

```
# Instalar y cargar la librería visNetwork
# install.packages("visNetwork")
library(visNetwork)
library(networkD3)
```

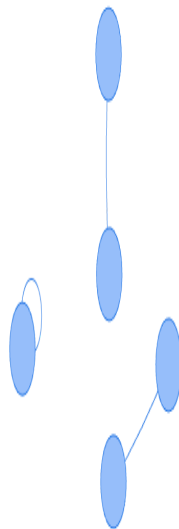
8.0.1 Redes con networkD3

`networkD3` es otra librería de R que permite crear visualizaciones interactivas de redes utilizando la biblioteca de javascript `D3.js`. Es especialmente útil para crear gráficos de redes jerárquicas, como árboles y diagramas de Sankey. A continuación, se muestra un ejemplo básico de cómo usar `networkD3` para crear una red jerárquica simple.

`visNetwork` requiere de al menos, como cualquier red, de dos elementos:

- **Nodos:** Representan las entidades en la red. Cada nodo puede tener atributos como un identificador, una etiqueta, un grupo, etc.
- **Enlaces:** Representan las conexiones entre los nodos. Cada enlace puede tener atributos como el nodo de origen, el nodo de destino, un peso, etc.

```
# Crear un data frame de nodos
nodes <- data.frame(id = 1:5)
# Crear un data frame de conexiones
edges <- data.frame(from = c(1,2,5), to = c(3,4,5))
visNetwork(nodes, edges, width = "100%")
```



En este ejemplo, hemos creado una red con 5 nodos y 3 conexiones entre ellos. La función `visNetwork()` toma los data frames de nodos y enlaces como argumentos y genera una visualización interactiva de la red. Puedes personalizar la apariencia de los nodos y enlaces utilizando diferentes atributos, como colores, tamaños, etiquetas, etc.

8.0.2 Cómo obtener ayuda en visNetwork

Si necesitas ayuda con la función `visNetwork` o cualquier otra función de la librería, puedes usar la función `help()` o el signo de interrogación `?` seguido del nombre de la función. Además, existe ayuda adicional en esta librería usando:

8.0.3 Personalización de la visualización

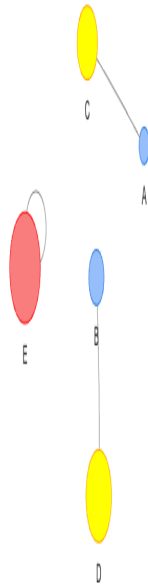
Puedes personalizar la apariencia de los nodos y enlaces utilizando diferentes atributos. Por ejemplo, puedes asignar colores a los nodos según un grupo o categoría, o ajustar el tamaño de los nodos en función de su grado (número de conexiones).

```
# Crear un data frame de nodos con atributos adicionales
nodes <- data.frame(id = 1:5,
                    label = c("A", "B", "C", "D", "E"),
                    group = c("X", "X", "Y", "Y", "Z"),
                    value = c(10, 20, 30, 40, 50))

# Crear un data frame de conexiones
edges <- data.frame(from = c(1,2,5), to = c(3,4,5))

# Visualizar la red con personalización
visNetwork(nodes, edges, width = "100%") %>%
  visNodes(color = list(background = "lightblue", border = "black")) %>%
  visEdges(color = "gray") %>%
  visOptions(highlightNearest = TRUE, nodesIdSelection = TRUE)
```

Select by id ▼



En este ejemplo, hemos asignado etiquetas a los nodos, agrupado los nodos en categorías (X, Y, Z) y asignado un valor que podría representar el grado o cualquier otra métrica. Además, hemos personalizado los colores de los nodos y enlaces, y habilitado opciones interactivas como resaltar el nodo más cercano al pasar el mouse y permitir la selección de nodos por su ID.

9 Ejercicios integradores

i Propósito de este capítulo

Estos ejercicios combinan conceptos de **todos los capítulos anteriores**. Cada uno requiere que apliques múltiples herramientas: creación de redes, métricas de grado, distancias, clustering y robustez. Están diseñados para que los resuelvas de forma autónoma, simulando un análisis de redes completo.

Guía de dificultad

Nivel	Descripción	Tiempo estimado
	Aplica funciones directamente	15–20 min
	Requiere combinar 2–3 conceptos	30–45 min
	Diseño experimental y análisis completo	60–90 min

9.1 Preparación del entorno

```
library(igraph)
library(poweRlaw)
```

Ejercicio integrador 1: Perfil completo de una red

Crea una función `perfil_red()` que reciba un grafo `g` y devuelva un *data frame* con todas las métricas que hemos aprendido:

- Número de nodos y aristas

- Grado medio, mínimo y máximo
- Densidad de aristas
- Distancia media y diámetro
- Clustering global y promedio
- Número de componentes conectados
- Tamaño del componente gigante (fracción)

Prueba tu función con las siguientes cinco redes (todas de 200 nodos):

1. Erdős–Rényi con $p = 0.03$
2. Barabási–Albert con $m = 2$
3. *Small-world* con $\text{nei} = 3$, $p = 0.05$
4. Anillo
5. Estrella

Presenta los resultados en una tabla comparativa y responde: ¿cuál red tiene la mejor combinación de distancia corta y clustering alto?



Solución

Paso 1: Definir la función.

```

perfil_red <- function(g) {
  comp <- components(g)

  # Eficiencia: promedio de 1/d(i,j)
  d <- distances(g)
  n <- vcount(g)
  inv_d <- ifelse(d == 0 | is.infinite(d), 0, 1 / d)
  eficiencia <- sum(inv_d) / (n * (n - 1))

  data.frame(
    Nodos = vcount(g),
    Aristas = ecount(g),
    Grado_medio = round(mean(degree(g)), 2),
    Grado_min = min(degree(g)),
    Grado_max = max(degree(g)),
    Densidad = round(edge_density(g), 4),
    Dist_media = round(mean_distance(g), 3),
    Diámetro = diameter(g),
    C_global = round(transitivity(g, type = "global"), 4),
    C_promedio = round(transitivity(g, type = "average"), 4),
    Componentes = comp$no,
    Comp_gigante_pct = round(max(comp$size) / vcount(g) * 100, 1),
    Eficiencia = round(eficiencia, 4)
  )
}

```

Paso 2: Generar las redes y calcular perfiles.


```

set.seed(42)

redes_perfil <- list(
  "Erdős-Rényi"      = sample_gnp(200, 0.03),
  "Barabási-Albert" = sample_pa(200, m = 2, directed = FALSE),
  "Small-world"     = sample_smallworld(1, 200, nei = 3, p = 0.05),
  "Anillo"          = make_ring(200),
  "Estrella"        = make_star(200, mode = "undirected")
)

# Calcular el perfil de cada red
perfiles <- do.call(rbind, lapply(redes_perfil, perfil_red))
perfiles <- cbind(Red = names(redes_perfil), perfiles)
rownames(perfiles) <- NULL

knitr::kable(perfiles,
              caption = "Perfil completo de cinco topologías con 200 nodos")

```

Table 9.2: Perfil completo de cinco topologías con 200 nodos

	No- dos	Aris- tas	Grado Medio	Grado Máximo	Dens- si- dad	Dist_me- dia	Diáme- tro	C_promp- to	Com- pene- ntes	Comp_gien- te	Efi- cien- cia		
Erdős- Rényi	200	578	5.78	0	14	0.0293	3.181	6	0.0267	0.0270	2	99.5	0.3436
Barabási- Albert	200	397	3.97	2	28	0.0193	3.586	6	0.0292	0.0491	1	100.0	0.3086
Small- world	200	600	6.00	3	9	0.0302	4.253	7	0.4150	0.4329	1	100.0	0.2734
Anillo	200	200	2.00	2	2	0.0101	50.251	100	0.0000	0.0000	1	100.0	0.0521
Es- trella	200	199	1.99	1	199	0.0100	1.990	2	0.0000	0.0000	1	100.0	0.5050

Interpretación: La red *small-world* ofrece la mejor combinación de distancia corta y clustering alto. La estrella tiene la menor distancia media pero clustering cero. La red completa sería ideal en ambas métricas, pero requiere $\binom{200}{2} = 19,900$ aristas, mientras que la *small-world* logra un excelente balance con solo ~600.

Ejercicio integrador 2: Red social de una clase

Diseña una red que represente las relaciones de colaboración en un grupo de 20 estudiantes. Las reglas son:

1. Cada estudiante tiene un nombre (invéntalos o usa `paste0("Est_", 1:20)`).
2. Cada estudiante pertenece a uno de 4 equipos de proyecto (5 por equipo).
3. Dentro de cada equipo, todos están conectados entre sí (clique).
4. Entre equipos, agrega 8 conexiones aleatorias (simulando amistades inter-equipo).

Con la red construida:

- a) Visualiza la red coloreando por equipo.
- b) Calcula el clustering local de cada nodo. ¿Los nodos “puente” (con conexiones inter-equipo) tienen clustering diferente?
- c) Encuentra el nodo con mayor *betweenness centrality*. ¿Es un puente?
- d) Simula la eliminación de ese nodo. ¿Cómo cambian la distancia media y el número de componentes?

Pista 1

Para crear los cliques por equipo, usa `make_full_graph(5)` cuatro veces y luego `disjoint_union()` para combinarlos. Después agrega las aristas inter-equipo con `add_edges()`.

Pista 2

Los colores por equipo se asignan fácilmente: `V(g)$color <- rep(c("tomato", "skyblue", "gold", "lightgreen"), each = 5)`. La *betweenness centrality* se calcula con `betweenness(g)`.

Pista 3 — Pseudocódigo

```
1. set.seed(42)
2. # Crear 4 cliques
3. equipos <- disjoint_union(make_full_graph(5), make_full_graph(5),
4.                           make_full_graph(5), make_full_graph(5))
5. V(equipos)$name <- paste0("Est_", 1:20)
6. V(equipos)$equipo <- rep(1:4, each = 5)
7. V(equipos)$color <- rep(c("tomato","skyblue","gold","lightgreen"), each=5)
8.
9. # Agregar 8 conexiones inter-equipo aleatorias
```

```

10. for (k in 1:8) {
11.   e1 <- sample(1:5, 1) + (sample(0:3, 1) * 5)
12.   e2 <- sample(1:5, 1) + (sample(0:3, 1) * 5)
13.   while (V(equipos)$equipo[e1] == V(equipos)$equipo[e2]) {
14.     e2 <- sample(1:5, 1) + (sample(0:3, 1) * 5)
15.   }
16.   equipos <- add_edges(equipos, c(e1, e2))
17. }
18.
19. plot(equipos)
20. betweenness(equipos)
21. transitivity(equipos, type = "local")

```

Ejercicio integrador 3: Epidemia en tres topologías

Simula una epidemia SIR (*Susceptible* $\rightarrow$ *Infectado* $\rightarrow$ *Recuperado*) simplificada en tres tipos de red con 500 nodos: ER ($p = 0.01$), BA ($m = 2$) y *small-world* ($nei = 3$, $p = 0.1$).

Reglas de la epidemia (por ronda):

1. Comienza con 1 nodo infectado al azar.
2. Cada nodo infectado contagia a cada vecino susceptible con probabilidad $\beta = 0.3$.
3. Cada nodo infectado se recupera (y se vuelve inmune) con probabilidad $\gamma = 0.1$.
4. La epidemia termina cuando no quedan infectados.

Tareas:

- a) Implementa la simulación como una función `simular_sir(g, beta, gamma, semilla)`.
- b) Ejecuta 20 realizaciones por red. Registra el número final de recuperados (= total de infectados acumulados).
- c) Presenta boxplots del total de infectados por tipo de red.
- d) ¿En qué red se propaga más rápido la epidemia? ¿En cuál infecta a más personas en total?
- e) Relaciona los resultados con las métricas de distancia y clustering de cada red.

i Pista 1

Usa tres vectores de estado: S, I, R (lógicos o numéricos). En cada ronda, para cada infectado, recorre sus vecinos con `neighbors(g, v)` y contagia con `rbinom(1, 1, beta)`.

i Pista 2

Estructura de la función:

```
simular_sir <- function(g, beta = 0.3, gamma = 0.1, semilla = 42) {  
  set.seed(semilla)  
  n <- vcount(g)  
  estado <- rep("S", n) # S, I, R  
  paciente_cero <- sample(1:n, 1)  
  estado[paciente_cero] <- "I"  
  historial <- data.frame(ronda = 0, S = n-1, I = 1, R = 0)  
  
  while (sum(estado == "I") > 0) {  
    # ... contagio y recuperación ...  
  }  
  historial  
}
```

i Pista 3 — Pseudocódigo

Para cada ronda mientras haya infectados:

1. nuevos_infectados <- c()
2. Para cada nodo infectado v:
 - a) Para cada vecino u de v:
 - Si u es susceptible y $\text{runif}(1) < \beta$:
nuevos_infectados <- c(nuevos_infectados, u)
 - b) Si $\text{runif}(1) < \gamma$: estado[v] <- "R"
3. estado[nuevos_infectados] <- "I"
4. Registrar conteos S, I, R

La red BA debería propagar más rápido (distancia corta, hubs actúan como super-propagadores). La *small-world* debería alcanzar más personas en total (clustering alto = propagación local eficiente + atajos para propagación global).

Ejercicio integrador 4: Análisis de robustez comparativo completo

Realiza un análisis de robustez exhaustivo de tres redes de 300 nodos: ER ($p = 0.02$), BA ($m = 2$) y *small-world* ($\text{nei} = 3, p = 0.1$).

Tareas:

- a) Para cada red, ejecuta:
 - 10 simulaciones de fallos aleatorios (eliminando hasta el 50% de nodos).
 - 1 simulación de ataque dirigido por grado (determinista).
 - 1 simulación de ataque dirigido por *betweenness*.
- b) Genera un panel de 6 gráficos (3 redes $\times$ 2 escenarios: fallo vs. ataque) mostrando la evolución del componente gigante. Para los fallos, incluye la media y la banda de ± 1 DE.
- c) Calcula el “índice de robustez” R (área bajo la curva del componente gigante) para cada escenario y presenta una tabla resumen.
- d) Redacta un párrafo de conclusiones comparando las tres topologías.

i Pista 1

El índice R se calcula como el área bajo la curva del componente gigante vs. fracción eliminada. Usa la regla del trapecio:

```
indice_R <- function(x, y) {  
  sum(diff(x) * (y[-length(y)] + y[-1])) / 2)  
}
```

Un $R = 0.5$ corresponde a una degradación lineal; $R > 0.5$ indica robustez; $R < 0.5$ indica fragilidad.

i Pista 2

Para el panel de 6 gráficos, usa `par(mfrow = c(2, 3))`. Fila superior: fallos aleatorios. Fila inferior: ataques. Columnas: ER, BA, SW.

Ejercicio integrador 5: Red de ingredientes

Diseña una red donde los nodos representen 15 ingredientes de cocina mexicana y las aristas indiquen que dos ingredientes aparecen juntos en al menos una receta.

Ingredientes sugeridos: chile, tomate, cebolla, ajo, cilantro, limón, aguacate, frijol, maíz, queso, crema, pollo, arroz, tortilla, sal.

Tareas:

- Define las conexiones manualmente con `graph_from_literal()` basándote en recetas reales (salsa mexicana, guacamole, tacos, enchiladas, arroz rojo, etc.).
- Visualiza la red con tamaño de nodo proporcional al grado.
- ¿Cuál es el ingrediente más “central” (mayor grado)? ¿Coincide con tu intuición culinaria?
- Calcula la distancia media. ¿Los ingredientes están “cerca” unos de otros?
- Identifica si hay comunidades usando `cluster_louvain()` o `cluster_walktrap()`. ¿Las comunidades corresponden a tipos de ingrediente (proteínas, vegetales, condimentos)?
- Elimina el ingrediente más conectado. ¿Cómo cambia la estructura de la red?

i Pista 1

Un ejemplo de definición:

```
g <- graph_from_literal(  
  chile -- tomate, chile -- cebolla, chile -- ajo,  
  tomate -- cebolla, tomate -- cilantro, tomate -- limón,  
  aguacate -- limón, aguacate -- cilantro, aguacate -- tomate, aguacate -- sal,  
  # ... continuar con más recetas  
)
```

i Pista 2

Para detectar comunidades: `com <- cluster_walktrap(g)` y luego `membership(com)` te da el grupo de cada nodo. Colorea con `V(g)$color <- membership(com)`.

Ejercicio integrador 6: Construir desde archivo

Crea dos archivos CSV: uno de nodos y otro de aristas para representar una red de colaboración entre 12 investigadores.

Archivo nodos.csv:

id	nombre	departamento	publicaciones
1	Ana	Biología	15
2	Beto	Física	22
...	...	...	...

Archivo `aristas.csv`:

from	to	peso
1	2	3
1	5	1
...	...	...

Tareas:

- Crea ambos archivos con `write.csv()` y léelos con `read.csv()`.
- Construye la red con `graph_from_data_frame(d = aristas, vertices = nodos, directed = FALSE)`.
- Limpia la red con `simplify()`.
- Visualiza con tamaño proporcional a publicaciones y color por departamento.
- Calcula el perfil completo (usa `perfil_red()` del Ejercicio 1 si lo resolviste).
- ¿Quién es el investigador más central por *betweenness*? ¿Conecta departamentos diferentes?

i Pista 1

Diseña la red pensando en que las colaboraciones inter-departamento sean menos frecuentes que las intra-departamento. Así la detección de comunidades debería encontrar los departamentos.

Ejercicio integrador 7: Power-law en el mundo real

Genera una red BA de 5000 nodos con $m = 2$. Realiza un análisis completo de su distribución de grado:

- Histograma de grado con ejes lineales.
- Histograma con eje y logarítmico.
- Gráfico log-log de la distribución de grado (CCDF).
- Ajuste de ley de potencia con `powerLaw`: reporta γ y $x_{\min}$.
- Superpón el ajuste sobre el gráfico CCDF.
- Compara con una red ER del mismo tamaño: ¿el ajuste *power-law* tiene sentido para la red ER?
- Identifica los 20 *hubs* principales. ¿Qué fracción de las aristas totales concentran?
- Elimina los 20 *hubs* y recalcula la distribución de grado. ¿Sigue siendo *power-law*?

i Pista 1

Para la fracción de aristas de los *hubs*: `sum(degree(g)[top_20]) / (2 * ecount(g))`. Se divide entre $2m$ porque cada arista contribuye al grado de dos nodos (posible sobreconteo si los *hubs* están conectados entre sí).

i Pista 2

Después de eliminar los *hubs*, la cola de la distribución debería “recortarse” pero el cuerpo debería seguir mostrando un patrón *power-law* con un $x_{\min}$ diferente. Las redes BA son *scale-free* en su estructura, no solo en unos pocos nodos.

Ejercicio integrador 8: Diseña tu propia red

Elige un sistema real que te interese (ejemplos: red de transporte de tu ciudad, red de personajes de una serie/película, red de interacciones en un ecosistema, red de citas entre artículos científicos) y modélalo como una red.

Requisitos mínimos del análisis:

1. **Construcción:** Al menos 20 nodos y aristas definidas con sentido.
2. **Visualización:** Red con atributos visuales (color, tamaño, etiquetas).
3. **Métricas básicas:** Grado, distancia media, diámetro, clustering.
4. **Distribución de grado:** Histograma y gráfico log-log.
5. **Comunidades:** Detección con al menos un algoritmo.
6. **Robustez:** Simulación de fallo aleatorio y ataque dirigido.
7. **Interpretación:** Para cada métrica, explica qué significa en el contexto de tu sistema real.

i Pista 1

Si no sabes qué sistema elegir, estas son opciones accesibles:

- **Red de personajes de Harry Potter:** Nodos = personajes, aristas = aparecen juntos en una escena.
- **Red de metro/transporte:** Nodos = estaciones, aristas = líneas directas.
- **Red de playlist:** Nodos = canciones, aristas = aparecen en la misma playlist.
- **Red de ingredientes:** Similar al Ejercicio 5 pero con tu cocina favorita.

Pista 2

Estructura sugerida del reporte:

```
## 1. Descripción del sistema
## 2. Construcción de la red
## 3. Visualización
## 4. Análisis de métricas
## 5. Distribución de grado
## 6. Detección de comunidades
## 7. Análisis de robustez
## 8. Conclusiones
```

Rúbrica de autoevaluación general

Usa esta rúbrica para evaluar tus soluciones a los ejercicios integradores:

Table 9.5: Rúbrica de autoevaluación

Criterio	Excelente (3)	Adecuado (2)	Necesita mejora (1)
Código	Funciona sin errores, bien comentado, usa funciones reutilizables	Funciona con ajustes menores, comentarios parciales	Tiene errores o no es reproducible
Visualización	Gráficos claros con títulos, ejes etiquetados y leyenda	Gráficos legibles pero incompletos en etiquetado	Gráficos difíciles de interpretar
Interpretación	Conecta resultados numéricos con el contexto del problema	Interpreta parcialmente	Solo muestra números sin explicación
Integración	Combina 3 conceptos del libro de forma coherente	Combina 2 conceptos	Aplica conceptos de forma aislada

Referencias

- Albert, R., & Barabási, A.-L. (2002). Statistical mechanics of complex networks. *Reviews of Modern Physics*, 74(1), 47–97.
- Albert, R., Jeong, H., & Barabási, A.-L. (2000). Error and attack tolerance of complex networks. *Nature*, 406, 378–382.
- Barabási, A.-L. (2016). *Network science*. Cambridge University Press. <http://networksciencebook.com/>
- Erdős, P., & Rényi, A. (1959). On random graphs I. *Publicationes Mathematicae Debrecen*, 6, 290–297.
- Watts, D. J., & Strogatz, S. H. (1998). Collective dynamics of 'small-world' networks. *Nature*, 393(6684), 440–442.